Computer Architecture

Final Topics

Mahir Shadid (MAS)

Asst. Lecturer, CSE, IIUC

Chapter – 5: The Processor: Datapath and Control

Introduction

- The datapath is the part of the processor that performs operations on data. It includes all the hardware components that actually process instructions, such as:
  - Registers
  - ALU (Arithmetic Logic Unit)
  - Multiplexers (MUXes)
  - Memory access units

How datapath works?

- When the processor executes an instruction (like add or lw), the datapath:
  1. Fetches the instruction from memory (IF)
  2. Decodes what to do (ID)
  3. Reads data from registers (RR)
  4. Performs computation or memory access (MEX)
  5. Writes back the result (WB)

Key Components in the Datapath

1. PC (Program Counter): Holds address of the next instruction
2. Instruction Memory: Stores program instructions
3. Register File: Holds 32 general-purpose registers (like $t0, $s1, etc.)
4. ALU: Performs arithmetic and logic operations
5. Data Memory: For lw and sw
6. MUX (Multiplexer): Selects between different data sources

MIPS instructions: Core operations that the datapath must support

- **Now MIPS is Microprocessor without Interlocked Pipelined Stages, a RISC-based architecture.**
- **1. Memory-Reference Instructions (lw, sw):**

  1. Use the ALU to compute the effective address (base + offset)
  2. Use Data Memory for load/store
  3. Require register file to read or write data

  Purpose in Datapath: Ensures memory connection and address computation logic is designed correctly.

MIPS instructions: Core operations that the datapath must support

- **2. Arithmetic-Logical Instructions (add, sub, and, or, slt)**
  1. Read two registers
  2. Perform an operation in the ALU
  3. Write the result back to a register
  - Purpose in Datapath: Validates that ALU connections, control signals, and register read/write paths work properly.

MIPS instructions: Core operations that the datapath must support

- **3. Branch Instructions (beq, j)**

1. Beq compares registers and possibly updates the PC (Program Counter)
2. j directly sets the PC to a target address

Purpose in Datapath: Ensures the control logic can change the flow of execution, which is essential for loops, conditionals, etc.

An overview of the implementation

For every instruction, the first two steps are identical:

1. Sent the PC to the memory that contains the code and fetch the instruction from that memory.
2. Read one or two registers, using fields of the instruction to select the registers to read.

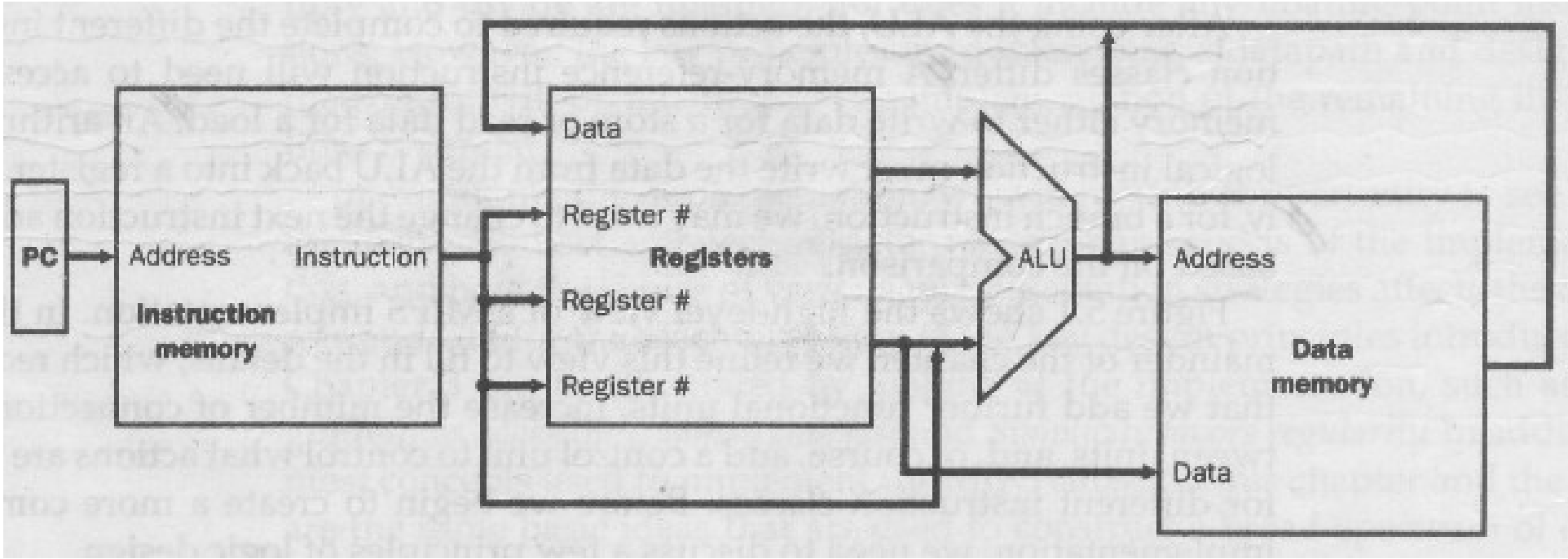
An overview of the implementation (Cont.)

Even across different instruction classes there are some similarities:

Example:

- All instruction classes use the ALU after reading the registers.
  - Memory-reference – address calculation
  - Arithmetic-logic – operation execution
  - Branch – comparison.
- After using the ALU, the actions differ.

An overview of the implementation (Cont.)



- **Fig.** Abstract view of a simplified datapath for a CPU. This diagram shows a single-cycle implementation, where each instruction completes in one clock cycle using this datapath.

A word about Logic Convention and Clocking

The datapath in a processor (like MIPS) is made up of two types of logic elements:

- Combinational - ALU
- State Element / Sequential – Instruction and Data Memory, Registers

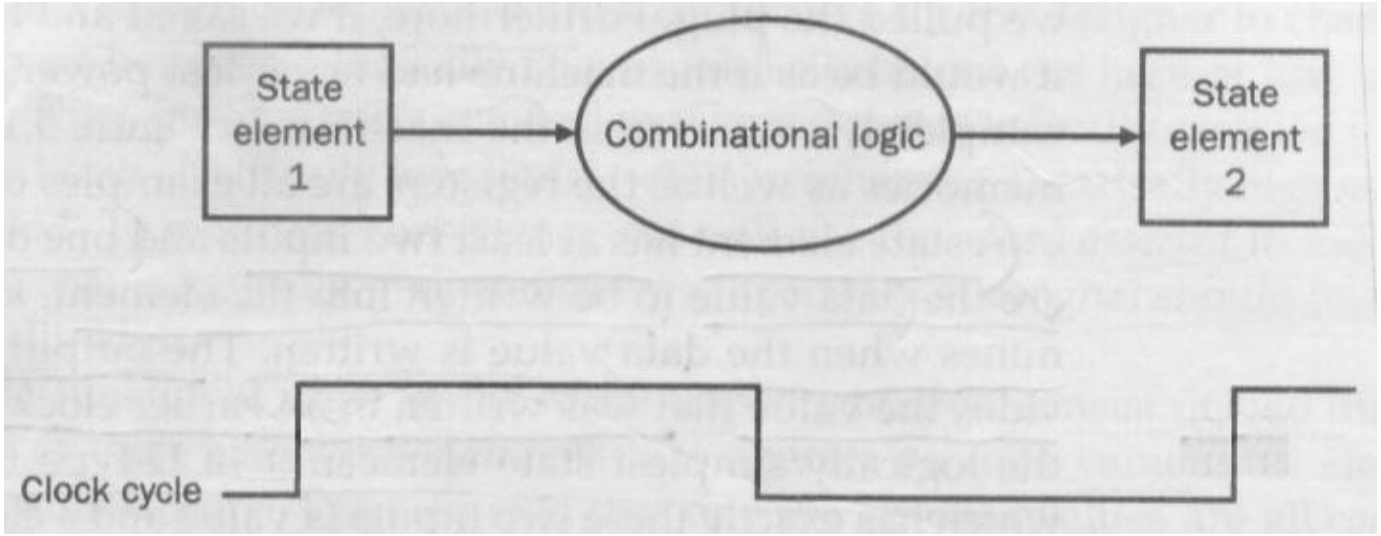
State element has at least two inputs and one output.

- Examples:

1. Instruction Memory (stores instructions)
2. Data Memory (stores data)
3. Register File (stores register values)
4. Program Counter (PC)

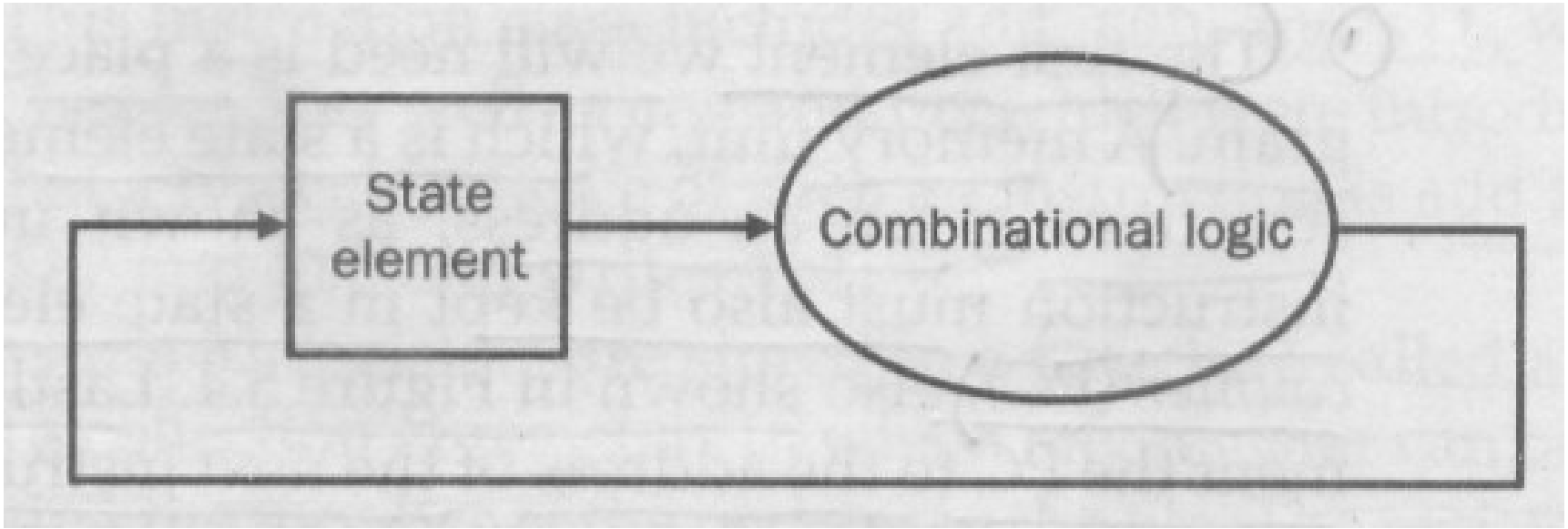
Clocking Methodology

- A clocking methodology defines when signal can be read and when they can be written. We assume ***edge-triggered clocking***.
- Combinational logic, which operate in a single clock cycle.



Clocking Methodology (Cont.)

- An *edge triggered methodology* allows us to read the contents of a register, send the value through some combinational logic, and write the register in the same clock cycle.



Building a Datapath

- Datapath elements:

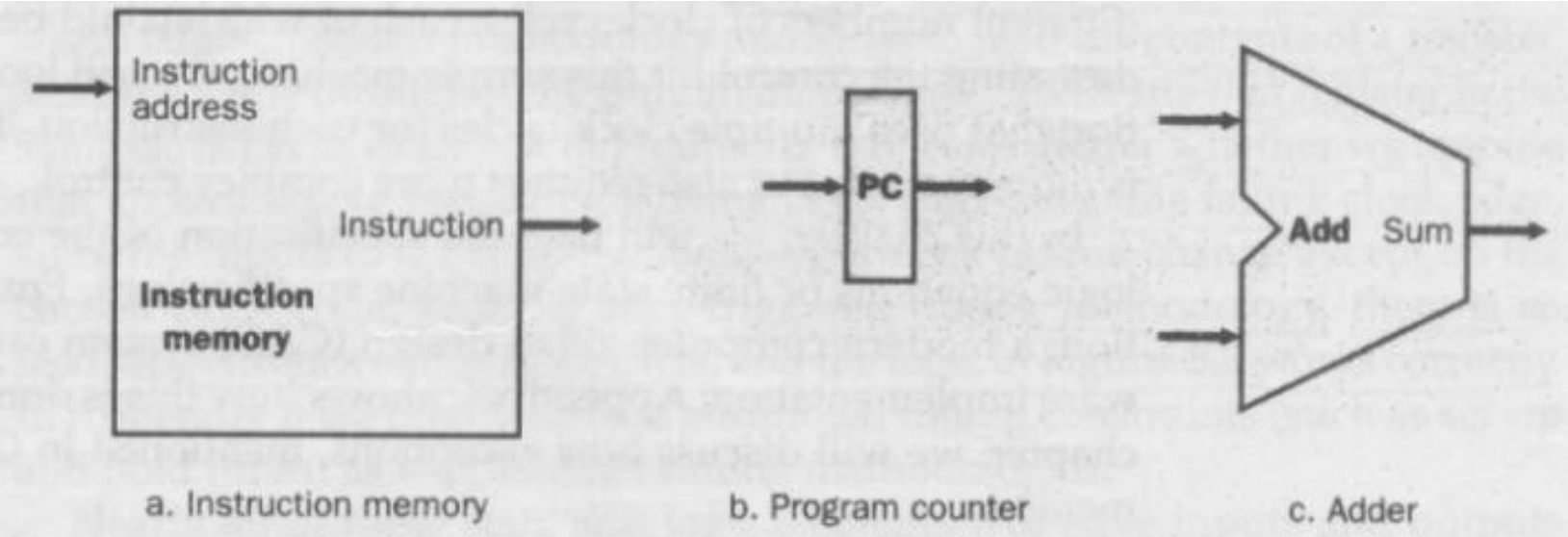
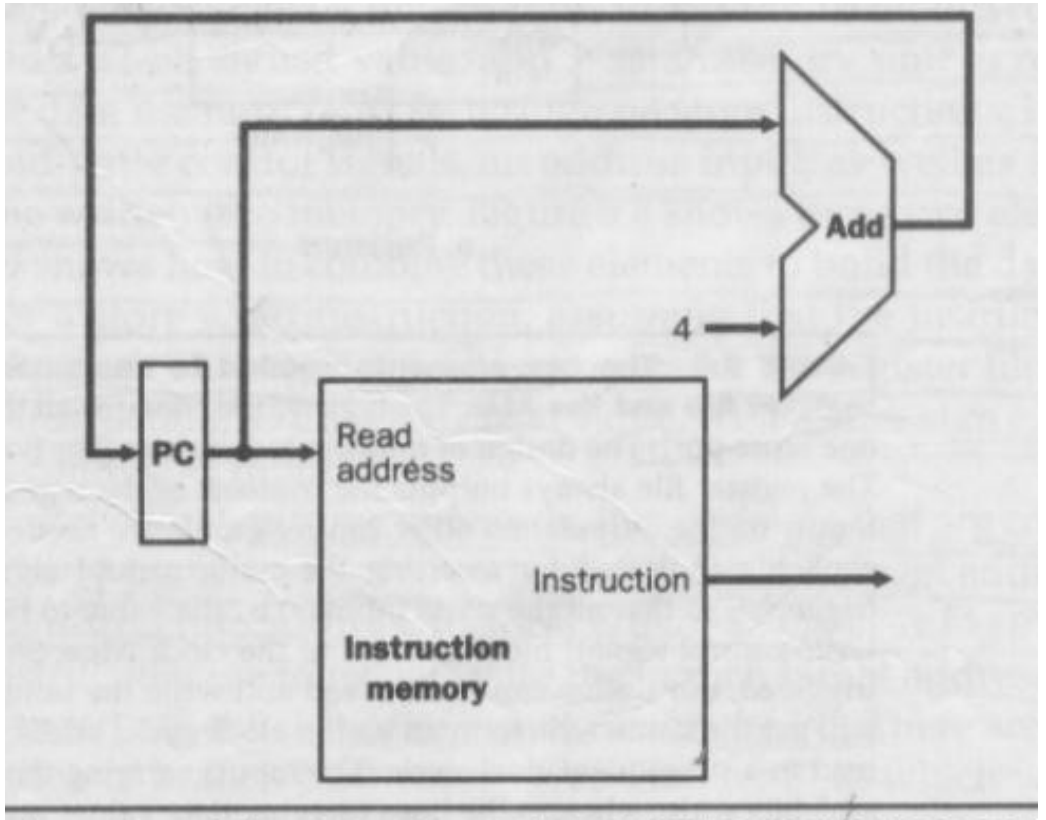


FIGURE 5.4 Two state elements are needed to store and access instructions, and an adder is needed to compute the next instruction address.

Building a Datapath

- For fetching instructions:



A portion of the datapath used for fetching instructions and incrementing the program counter.

Building a Datapath

Name	Fields					
Field Size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-format	op	rs	rt	rd	shamt	funct
I-format	op	rs	rt	address/immediate		
J-format	op	target address				

Building a Datapath

- **R-format:** ***add*** $t1, $t2, $t3
- **Register file:** is a collection of registers in which any resister can be read or written by specifying the number of the register in the file.

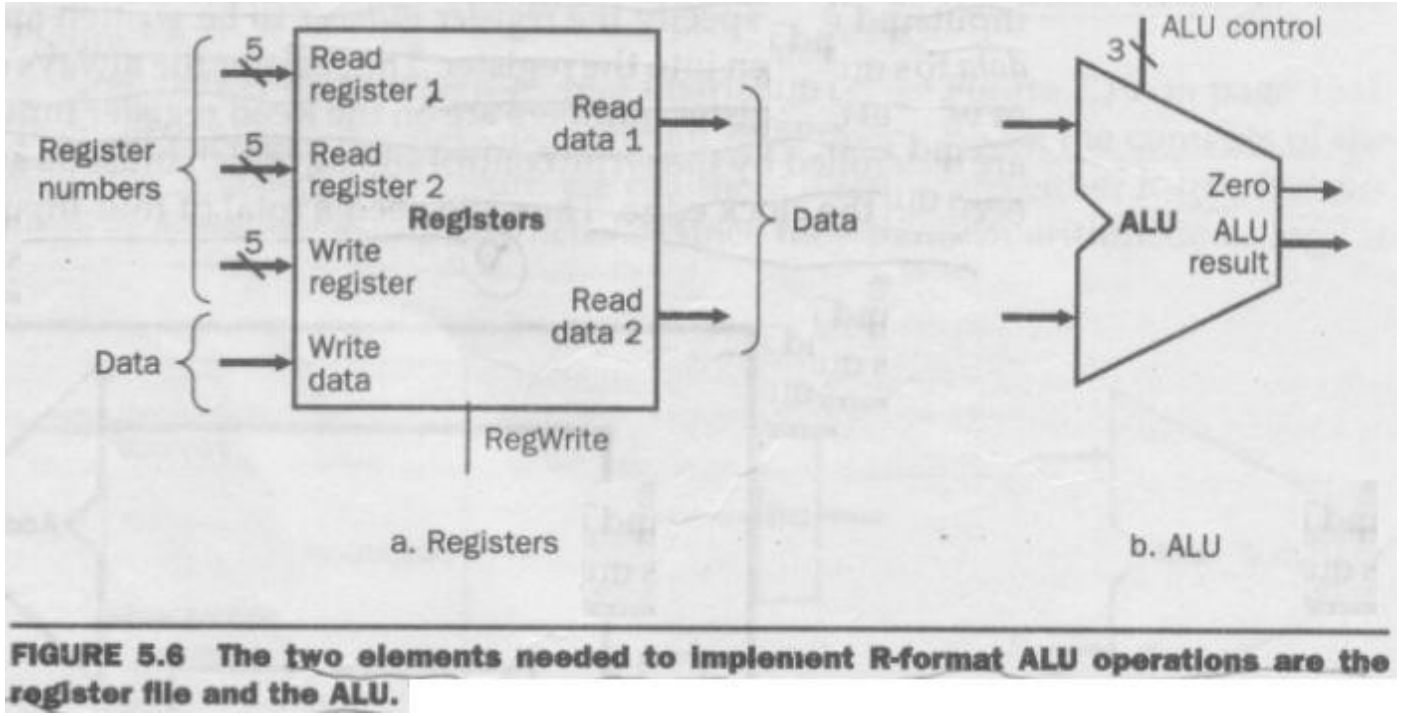
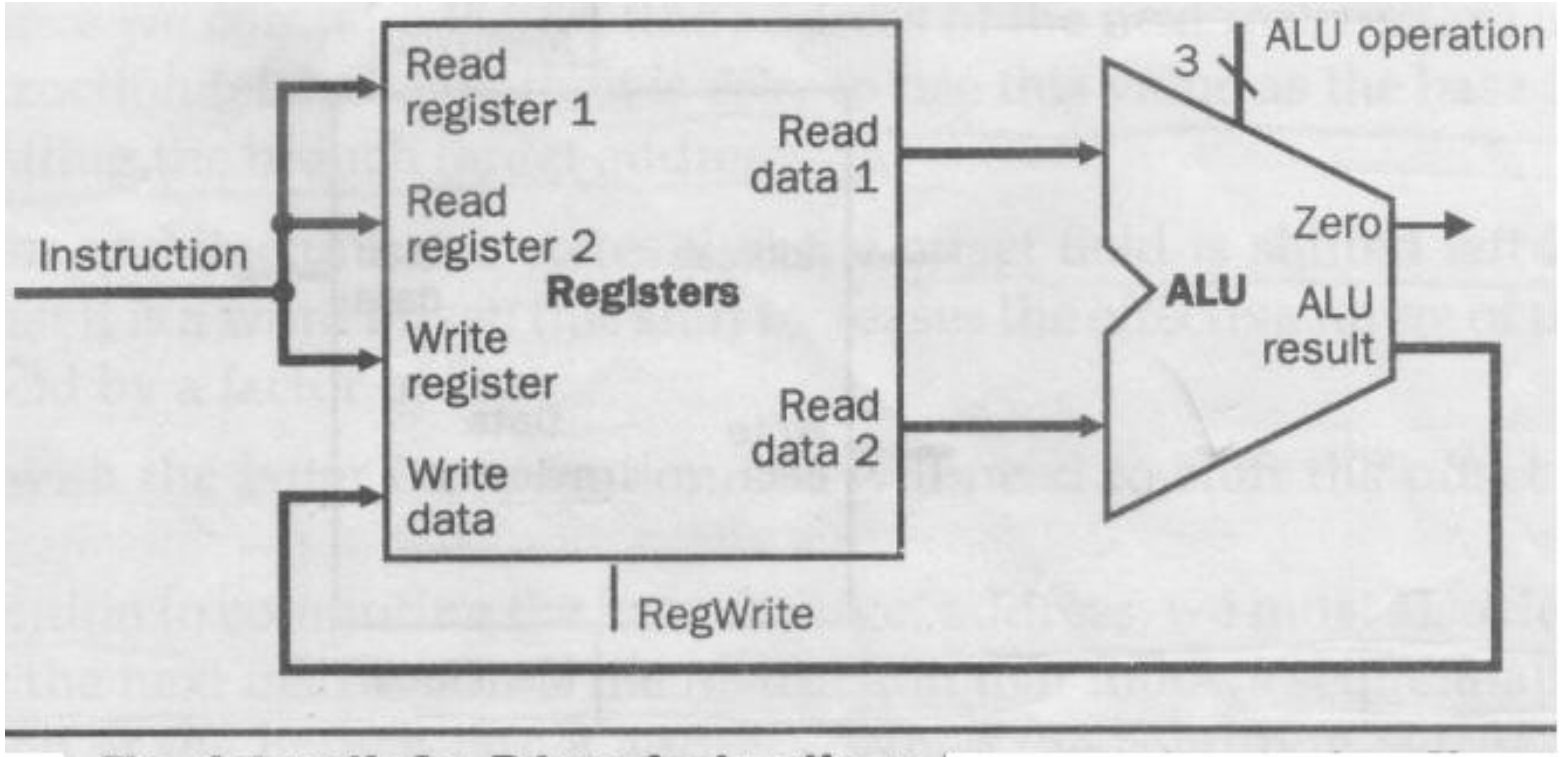


FIGURE 5.6 The two elements needed to implement R-format ALU operations are the register file and the ALU.

Building a Datapath

- Datapath for R-type instruction.



The datapath for R-type instructions.

Building a Datapath

- **I-format:**
  1. *lw* $t1, offset_value ($t2)
  2. *sw* $t1, offset_value ($t2)
- Compute a memory address by adding the base register ($t2) to the 16 bit signed offset field.

Building a Datapath

- Two elements needed for lw & sw instructions.

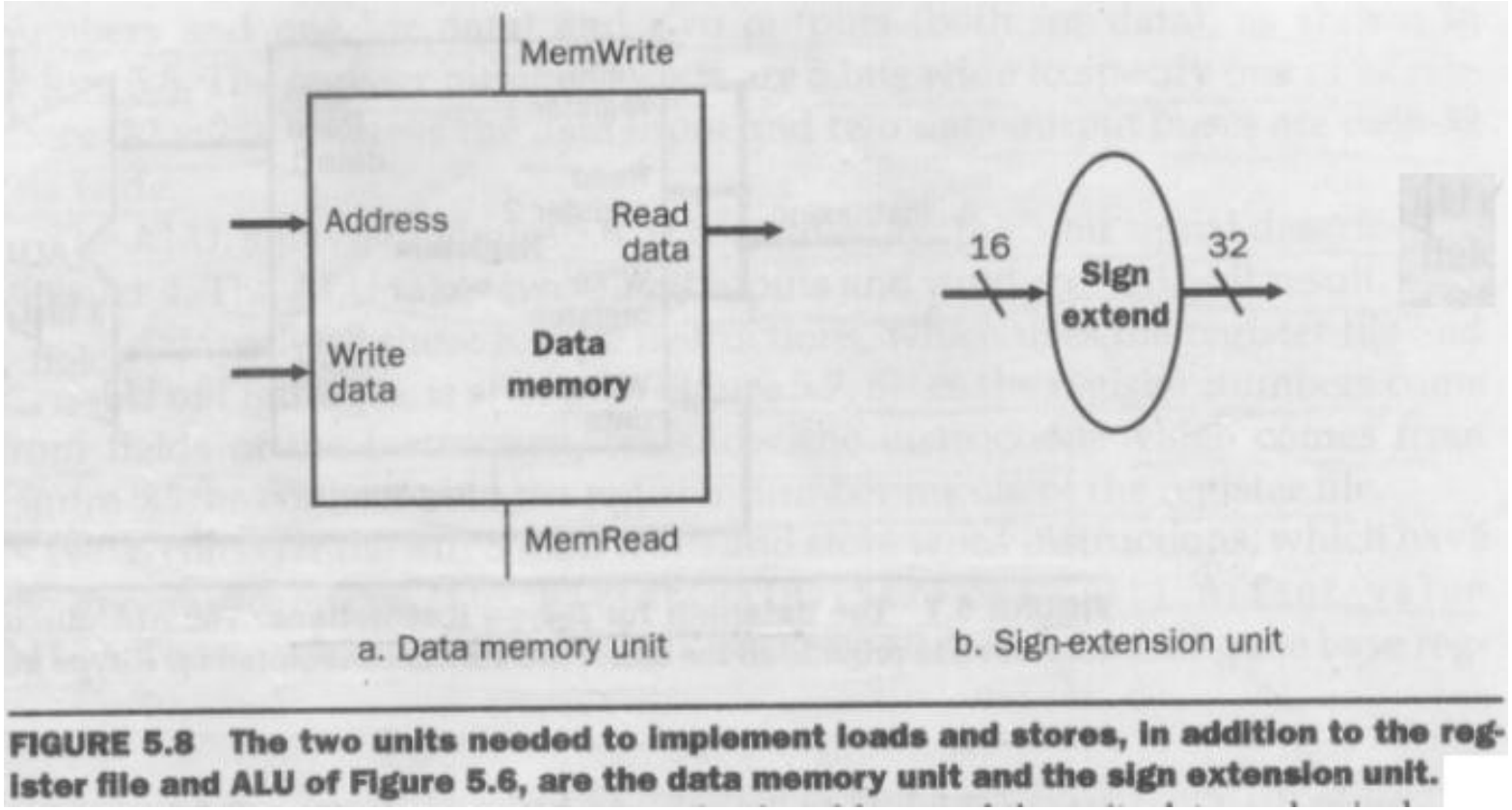
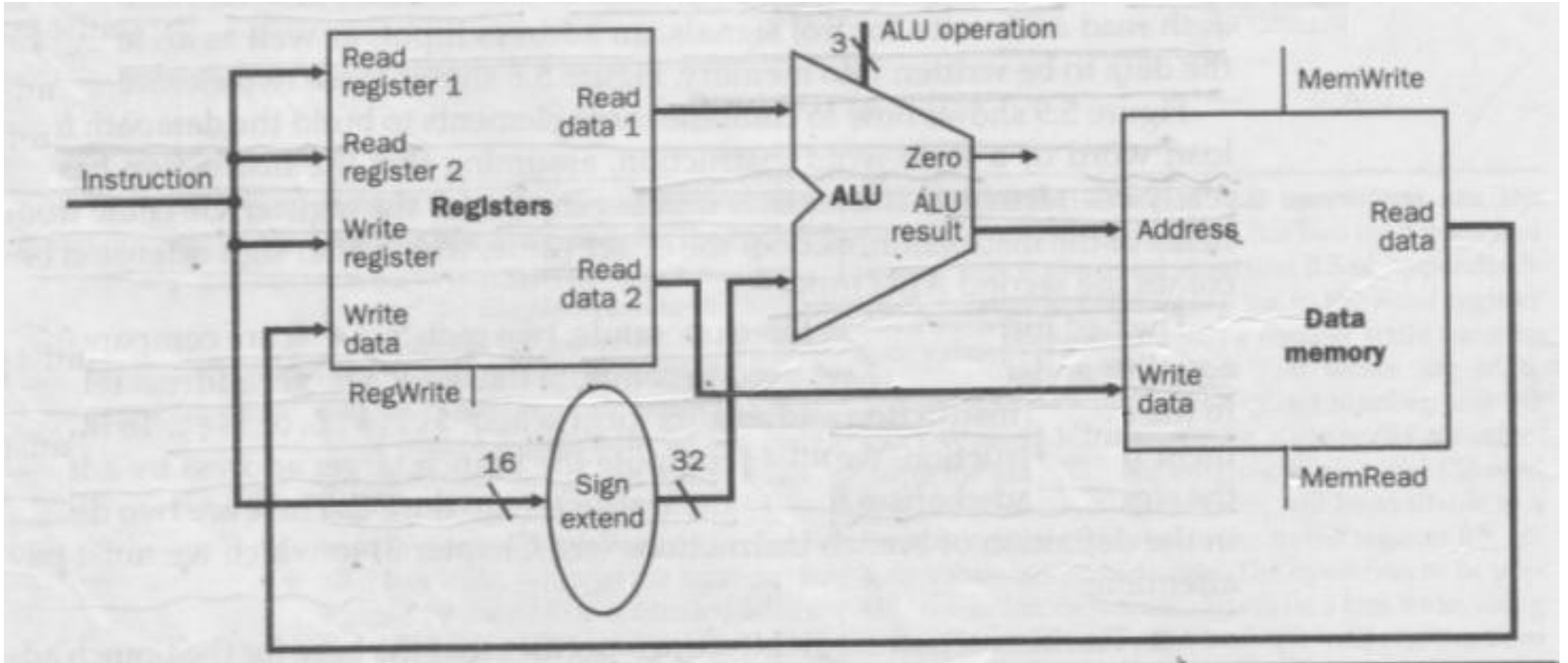


FIGURE 5.8 The two units needed to implement loads and stores, in addition to the register file and ALU of Figure 5.6, are the data memory unit and the sign extension unit.

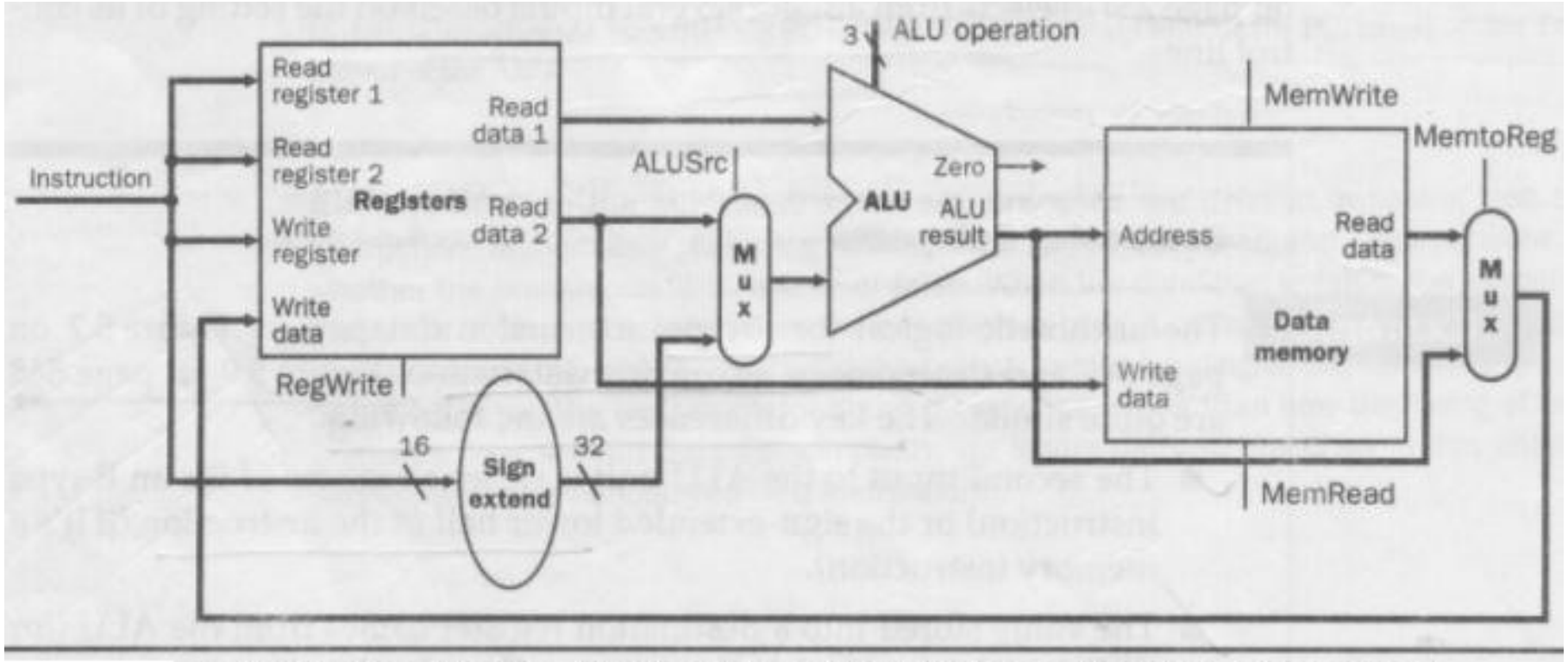
Building a Datapath

- Datapath for lw (read data) or sw (write data) instructions.



The datapath for a load or store does a register access, followed by a memory address calculation, then a read or write from memory, and a write into the register file if the instruction is a load.

Creating a Single Datapath



Combining the datapaths for the memory instructions and the R-type instructions.

Creating a Single Datapath

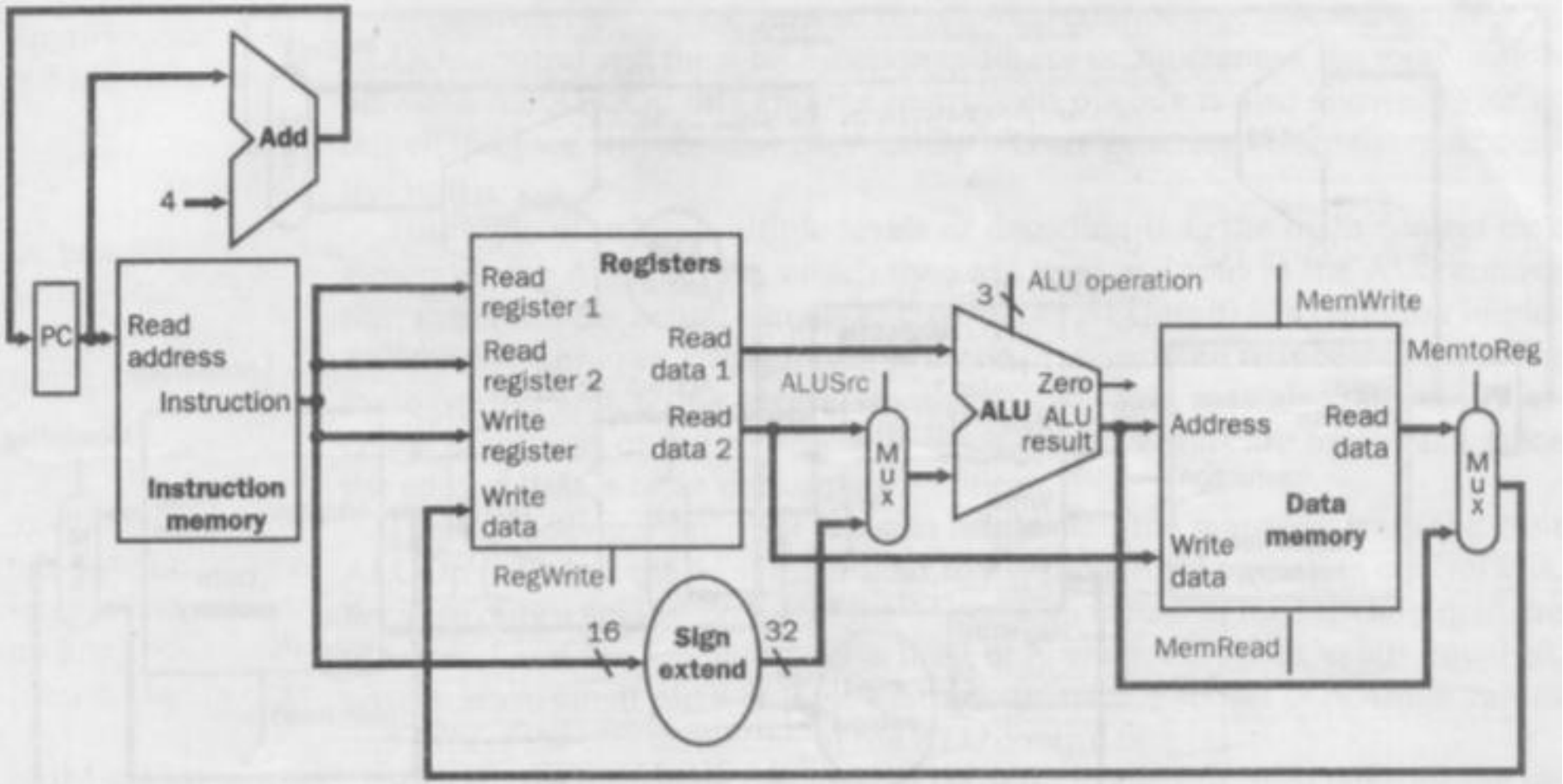


FIGURE 5.12 The instruction fetch portion of the datapath from Figure 5.5 is appended to the datapath of Figure 5.11 that handles memory and ALU instructions.

Creating a Single Datapath

<<2: Instruction count to byte distance. MIPS works with 4 bytes, so it basically multiplies 4 with an instruction count. Like in branching if we say, jump 3 steps ahead, the actual jump would be to 3x4=12 bytes ahead.

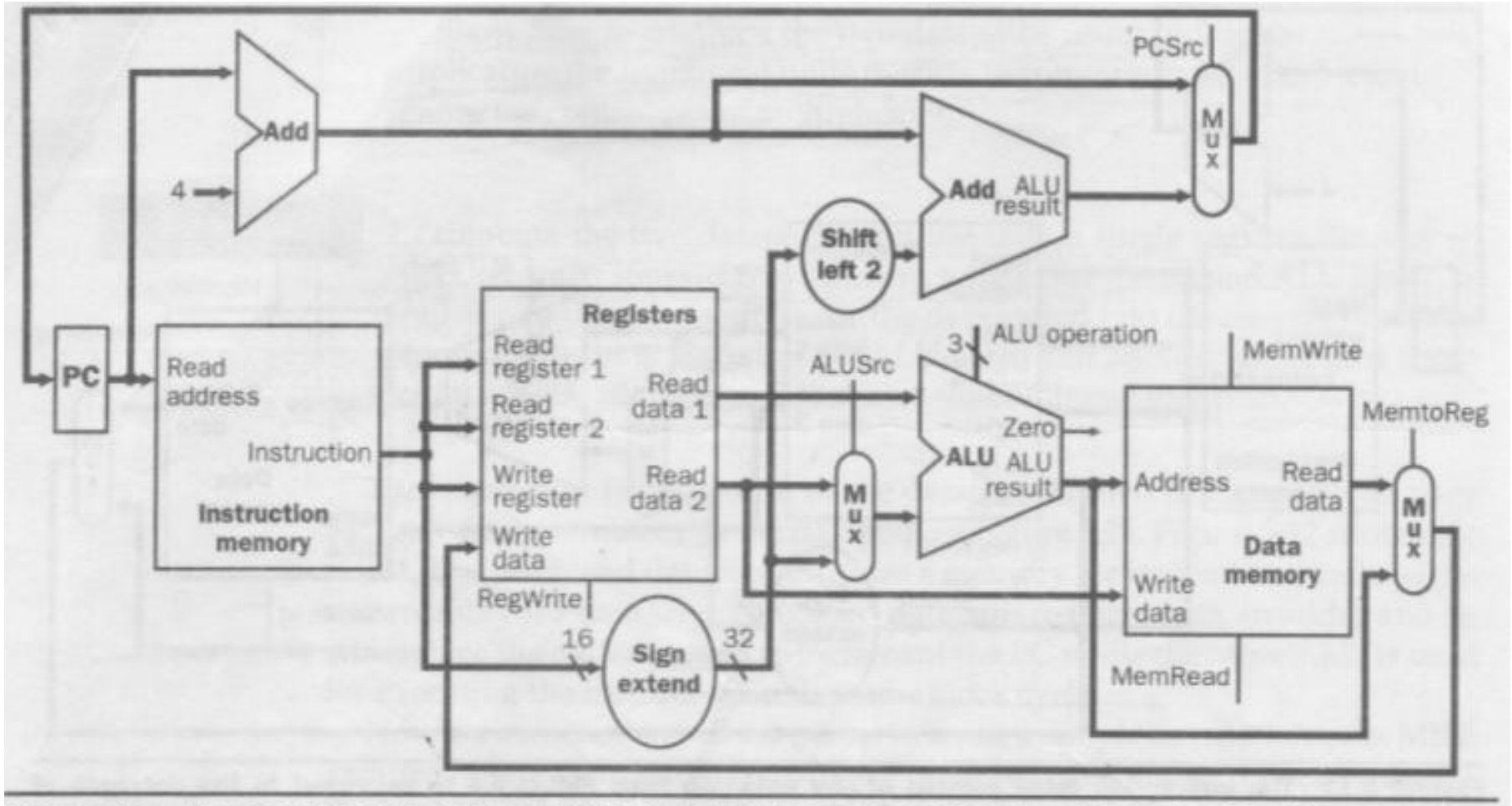


FIGURE 5.13 The simple datapath for the MIPS architecture combines the elements required by different instruction classes.

Designing the Main Control Unit

Name	Fields					
Field Size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-format	op (0)	rs	rt	rd	shamt	funct
I-format	op (35/43/4)	rs	rt	address/immediate		

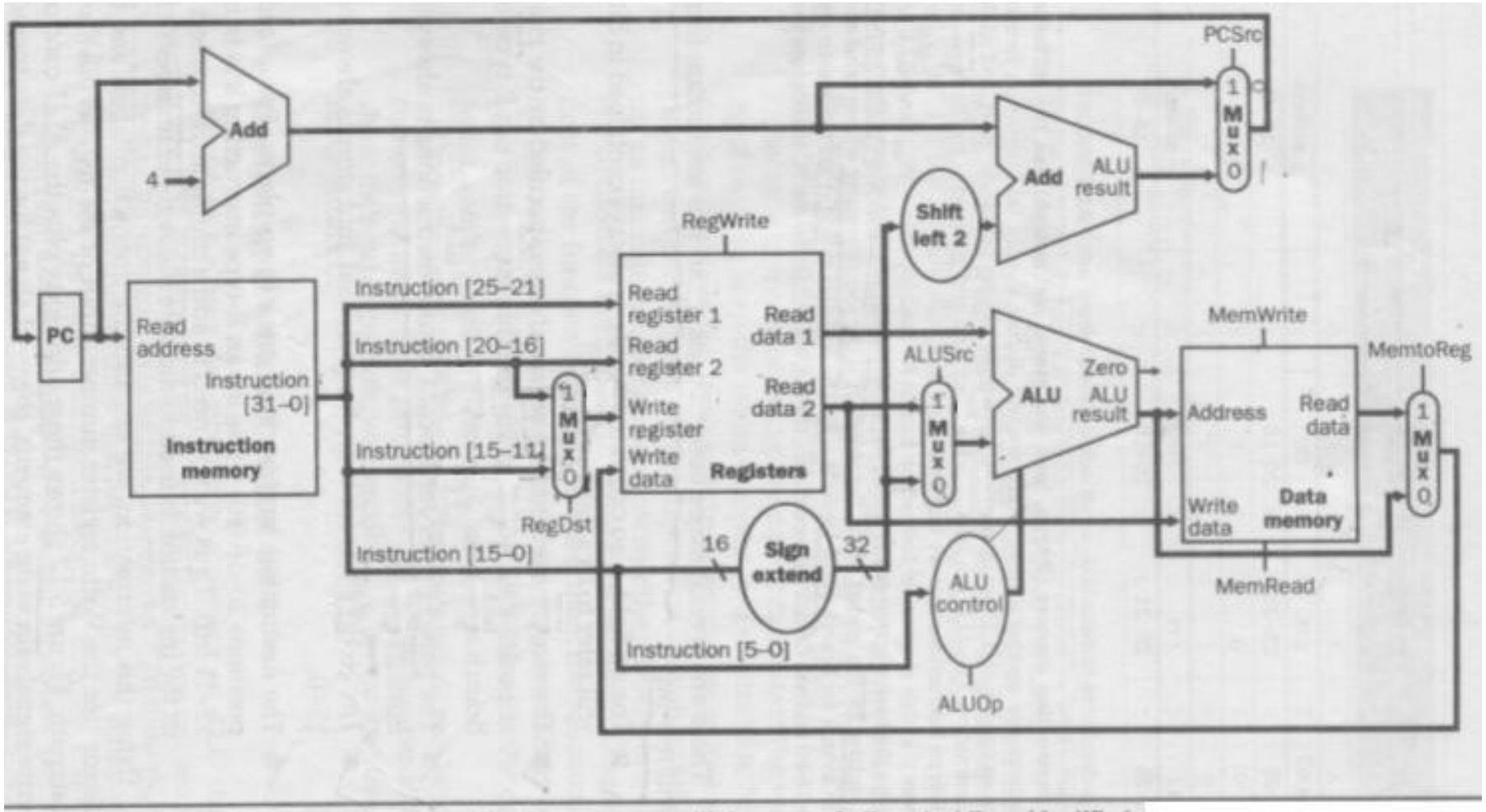
I-format : lw, sw, beq

Designing the Main Control Unit

- **Major observations:**
  1. op field always contained in bits 31-26
  2. Two registers to be read are always specified at positions 25-21 and 20-16.
  3. Base register for lw and sw is always in position 25-21.
  4. 16-bit offset for beq, lw and sw is always in position 15-0.
  5. Destination register is in one of two places. 20-16 for lw and 15-11 for R-type.

Designing the Main Control Unit

- **25-21**: rs, **20-16**: rt, **15-11**: rd, **15-0**: offset, **5-0**: funct, **31-26:** Opcode



The datapath of Figure 5.13 with all necessary multiplexors and all control lines identified.

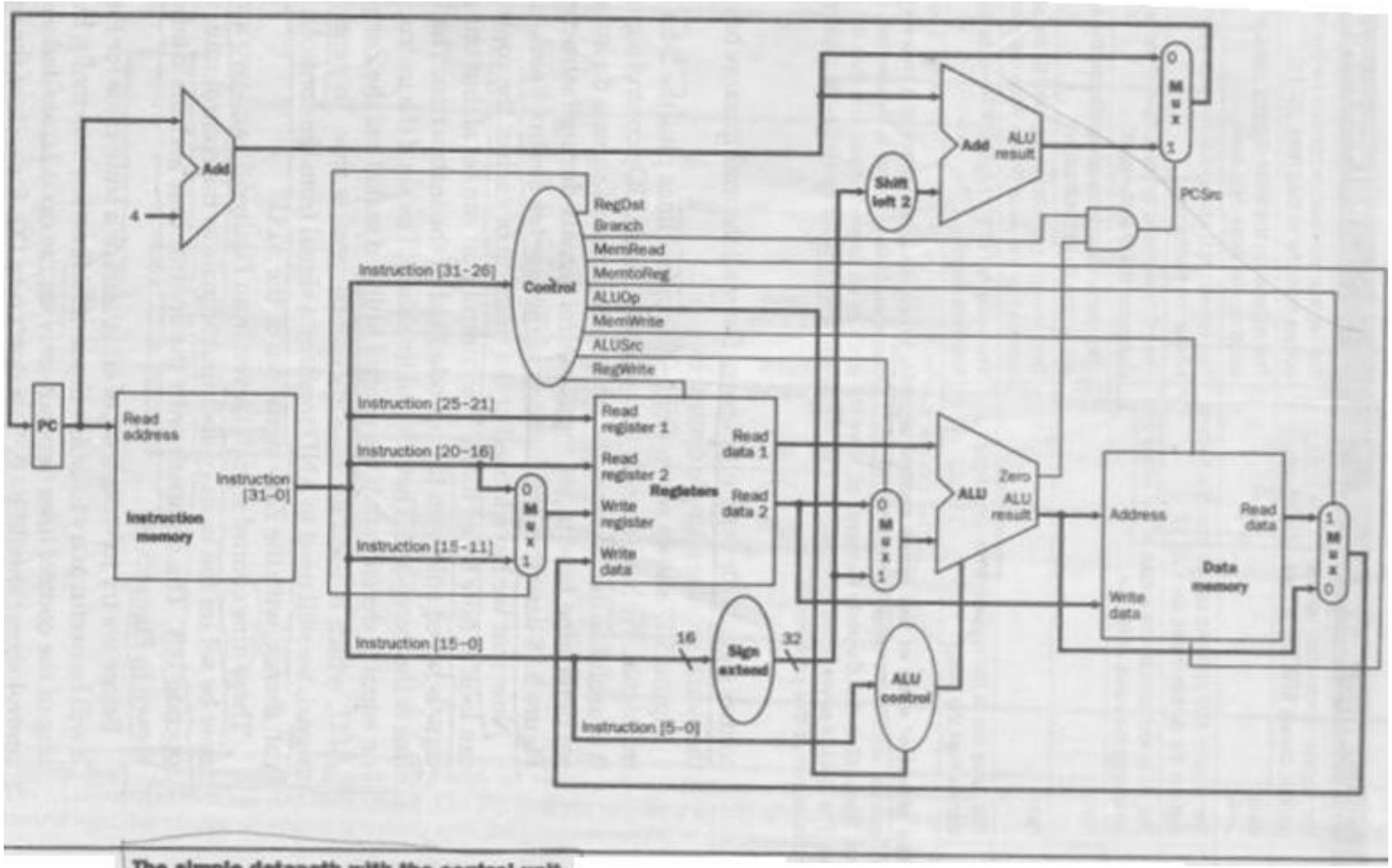
Designing the Main Control Unit

- 7-single bit control lines
- 2-bit ALUOp control signal

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register destination number for the Write register comes from the rt field (bits 20–16).	The register destination number for the Write register comes from the rd field (bits 15–11).
RegWrite	None	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, lower 16 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

FIGURE 5.18 The effect of each of the seven control signals.

Designing the Main Control Unit



The simple datapath with the control unit.

- 9 control signals can be set on the basis of six opcode bits.

Designing the Main Control Unit

- Defining how the control signals should be set for each opcode.

Instruction	RegDst	ALUSrc	Memto-Reg	Reg Write	Mem Read	Mem Write	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

FIGURE 5.20 The setting of the control lines is completely determined by the opcode fields of the instruction.

Operation of the Datapath

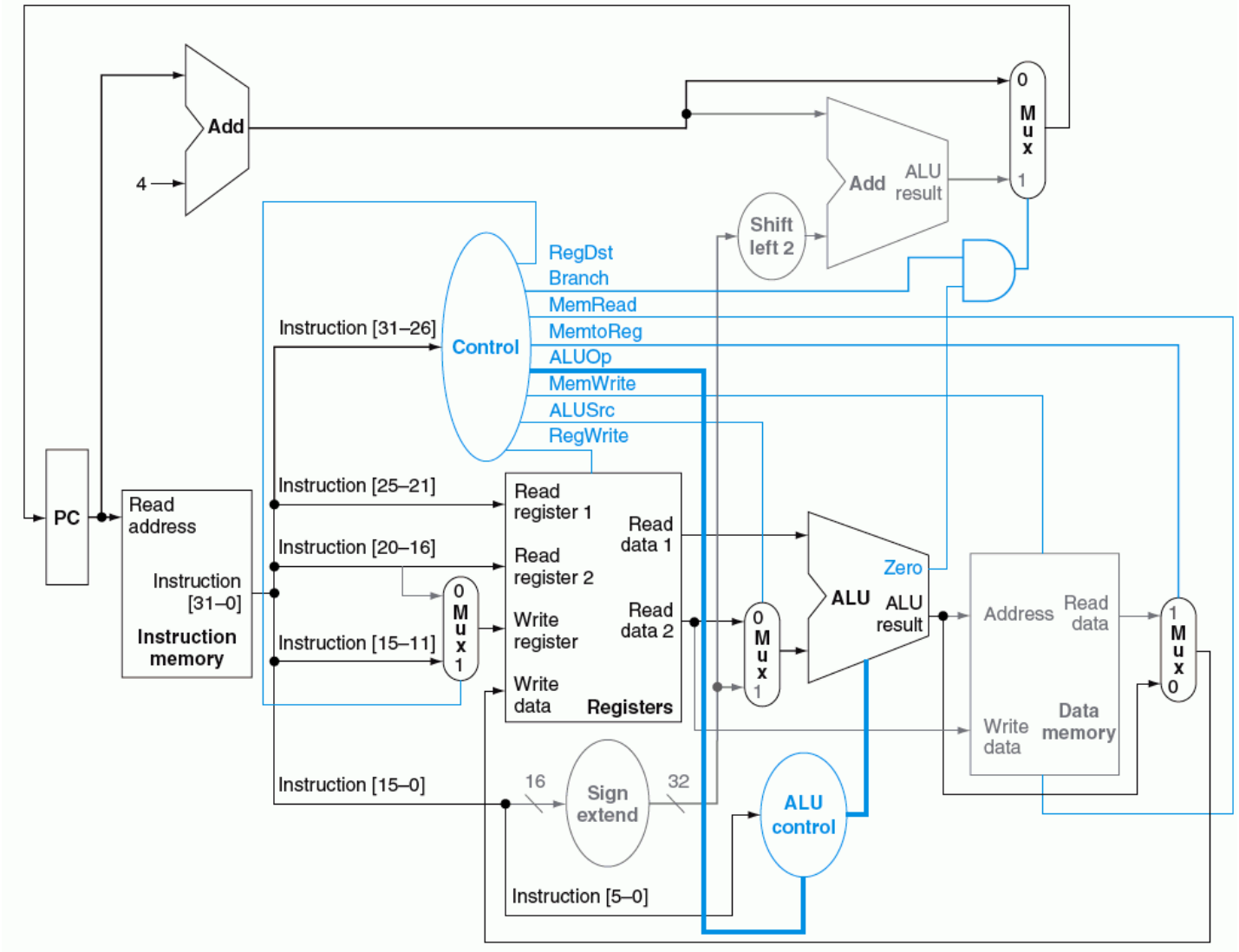


FIGURE 5.19 **The datapath in operation for an R-type instruction such as add** `$t1,$t2,$t3`. The control lines, datapath units, and

Operation of the Datapath

Figure 5.19 shows the operation of the datapath for an R-type instruction, such as `add $t1,$t2,$t3`. Although everything occurs in 1 clock cycle, we can think of four steps to execute the instruction; these steps are ordered by the flow of information:

1. The instruction is fetched, and the PC is incremented.
2. Two registers, `$t2` and `$t3`, are read from the register file, and the main control unit computes the setting of the control lines during this step also.
3. The ALU operates on the data read from the register file, using the function code (bits 5:0, which is the funct field, of the instruction) to generate the ALU function.
4. The result from the ALU is written into the register file using bits 15:11 of the instruction to select the destination register (`$t1`).

Operation of the Datapath

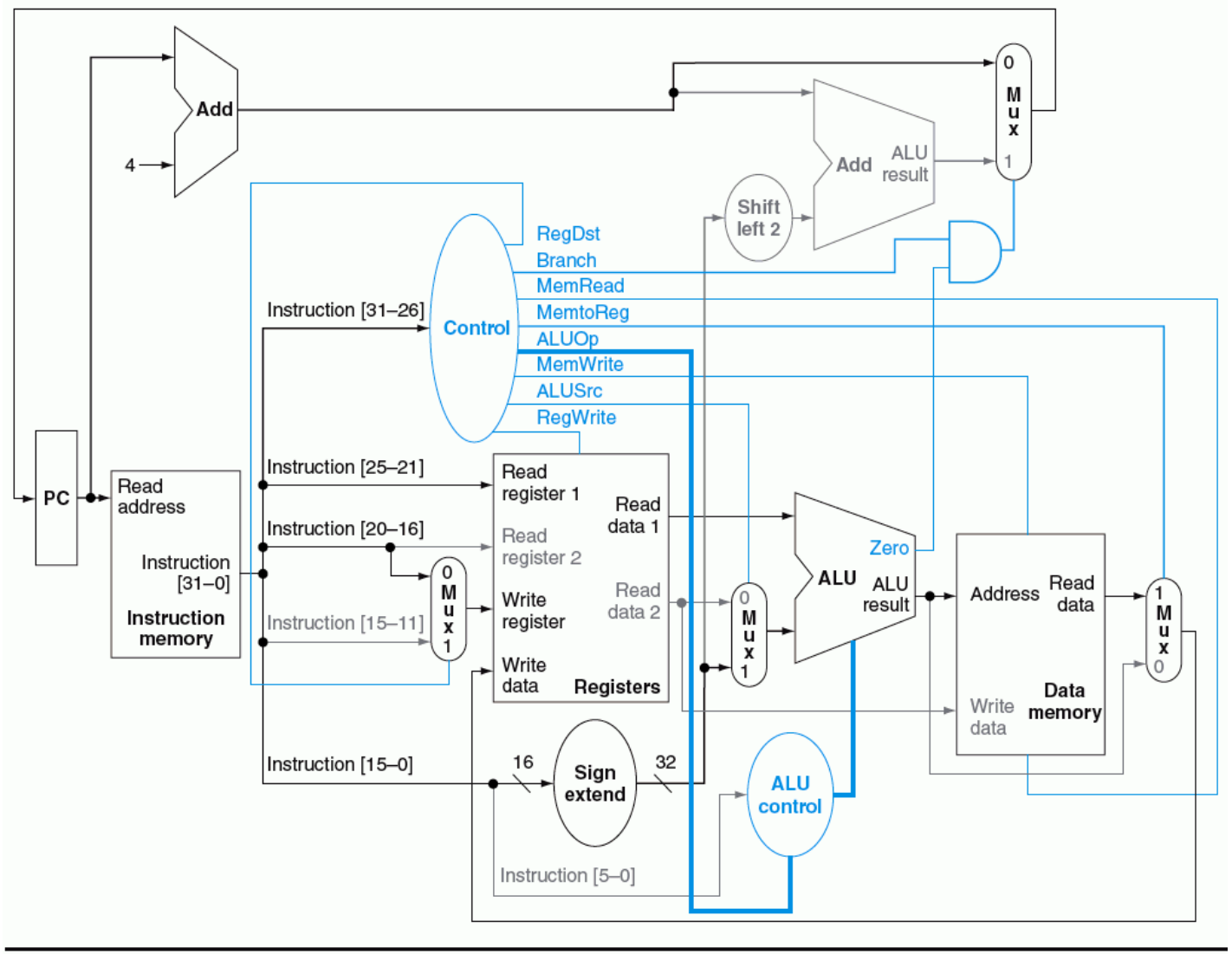


FIGURE 5.20 The datapath in operation for a load instruction. The control lines, datapath units, and connections that are active are high-

Operation of the Datapath

```
lw $t1, offset($t2)
```

1. An instruction is fetched from the instruction memory, and the PC is incremented.
2. A register (`$t2`) value is read from the register file.
3. The ALU computes the sum of the value read from the register file and the sign-extended, lower 16 bits of the instruction (`offset`).
4. The sum from the ALU is used as the address for the data memory.

Why a single-cycle implementation is not used?

In a single-cycle CPU, every instruction (simple or complex) must complete in 1 clock cycle. This means:

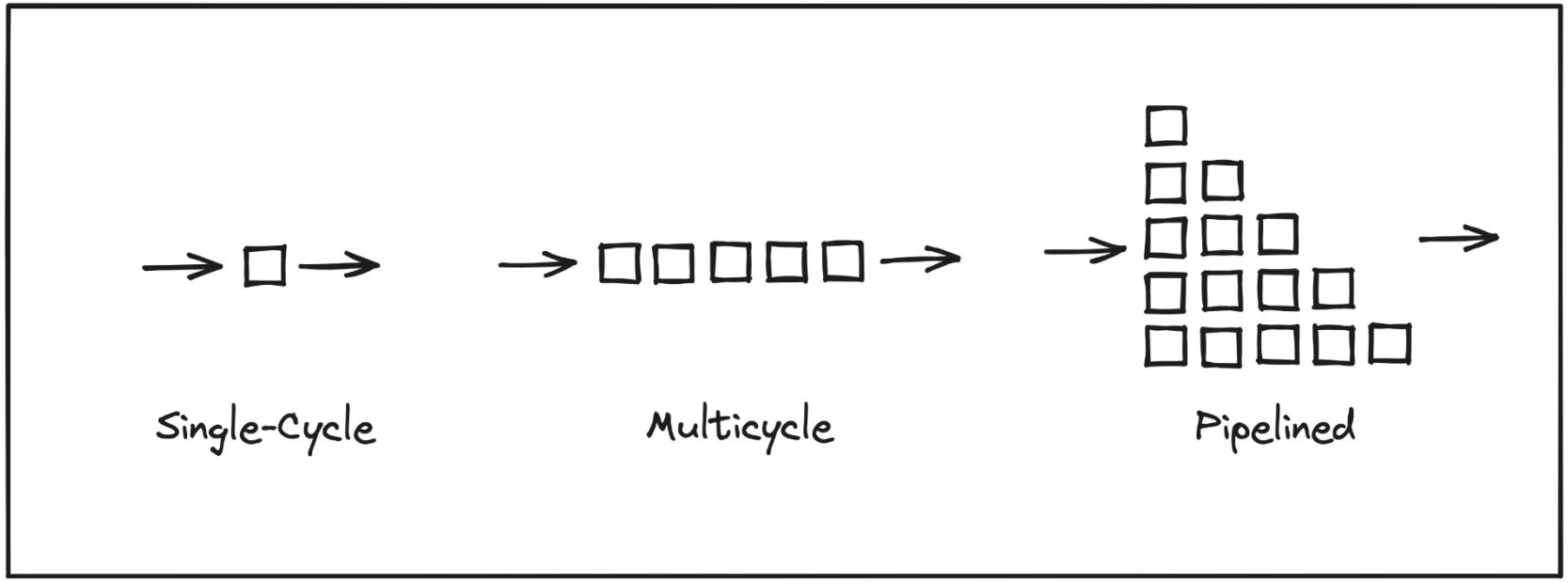
- The clock cycle must be long enough for the slowest instruction (e.g., lw which uses memory access).
- Faster instructions (e.g., add) waste time waiting for the cycle to end.

Result: The CPU runs at the speed of its slowest instruction.

A Multicycle Implementation

Advantages?

- Each step (fetch, decode, execute, memory access, write-back) gets one cycle, so the clock can run much faster.
- Hardware is reused across cycles. Only the needed hardware is active in each cycle.



A Multicycle Implementation

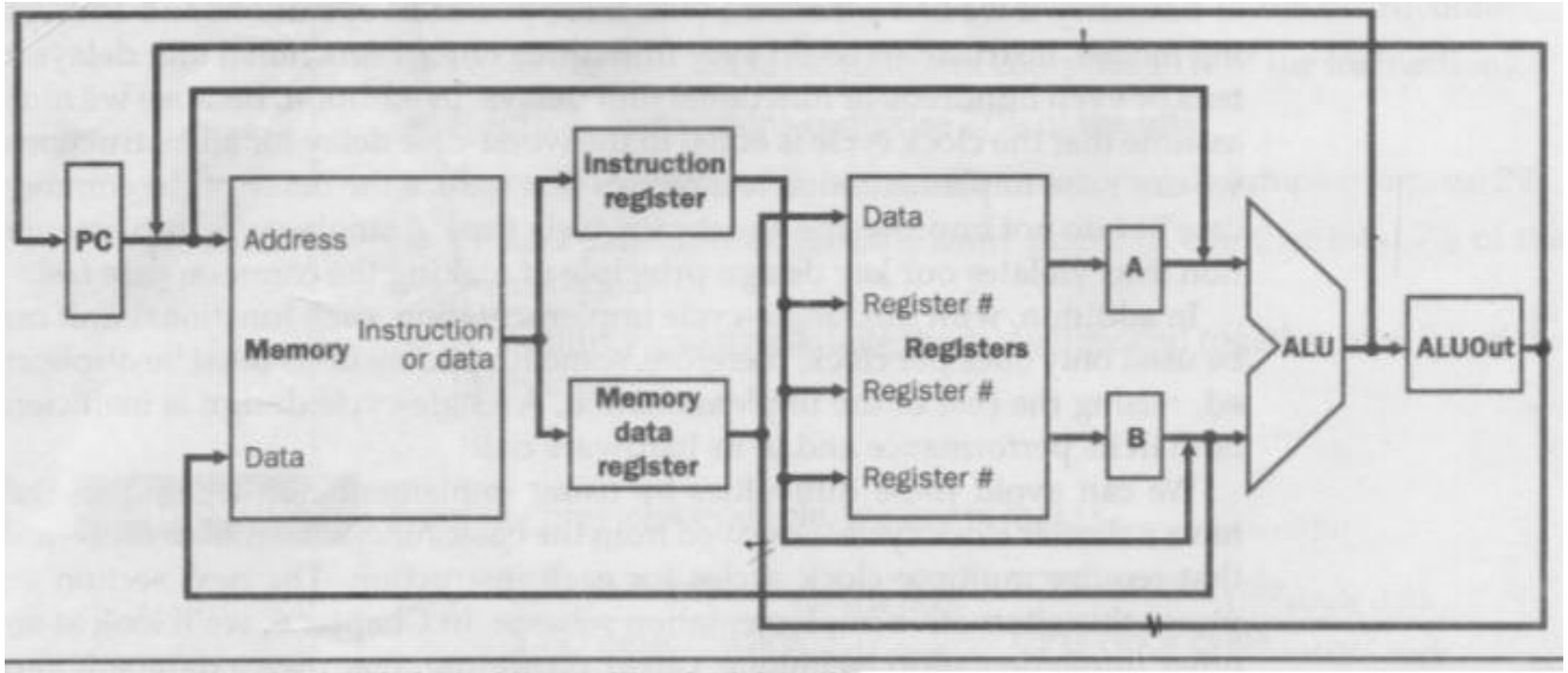


FIGURE 5.30 The high-level view of the multicycle datapath.

Why the temporary registers are used?

A Multicycle Implementation

The following temporary registers are added to meet these requirements:

- The Instruction register (IR) and the Memory data register (MDR) are added to save the output of the memory for an instruction read and a data read, respectively. Two separate registers are used, since, as will be clear shortly, both values are needed during the same clock cycle.
- The A and B registers are used to hold the register operand values read from the register file.
- The ALUOut register holds the output of the ALU.

A Multicycle Implementation

Replacing the three ALUs of the single-cycle datapath by a single ALU means that the single ALU must accommodate all the inputs that used to go to the three different ALUs. Handling the additional inputs requires two changes to the datapath:

1. An additional multiplexor is added for the first ALU input. The multiplexor chooses between the A register and the PC.
2. The multiplexor on the second ALU input is changed from a two-way to a four-way multiplexor. The two additional inputs to the multiplexor are the constant 4 (used to increment the PC) and the sign-extended and shifted offset field (used in the branch address computation).

A Multicycle Implementation

With the additional multiplexers.

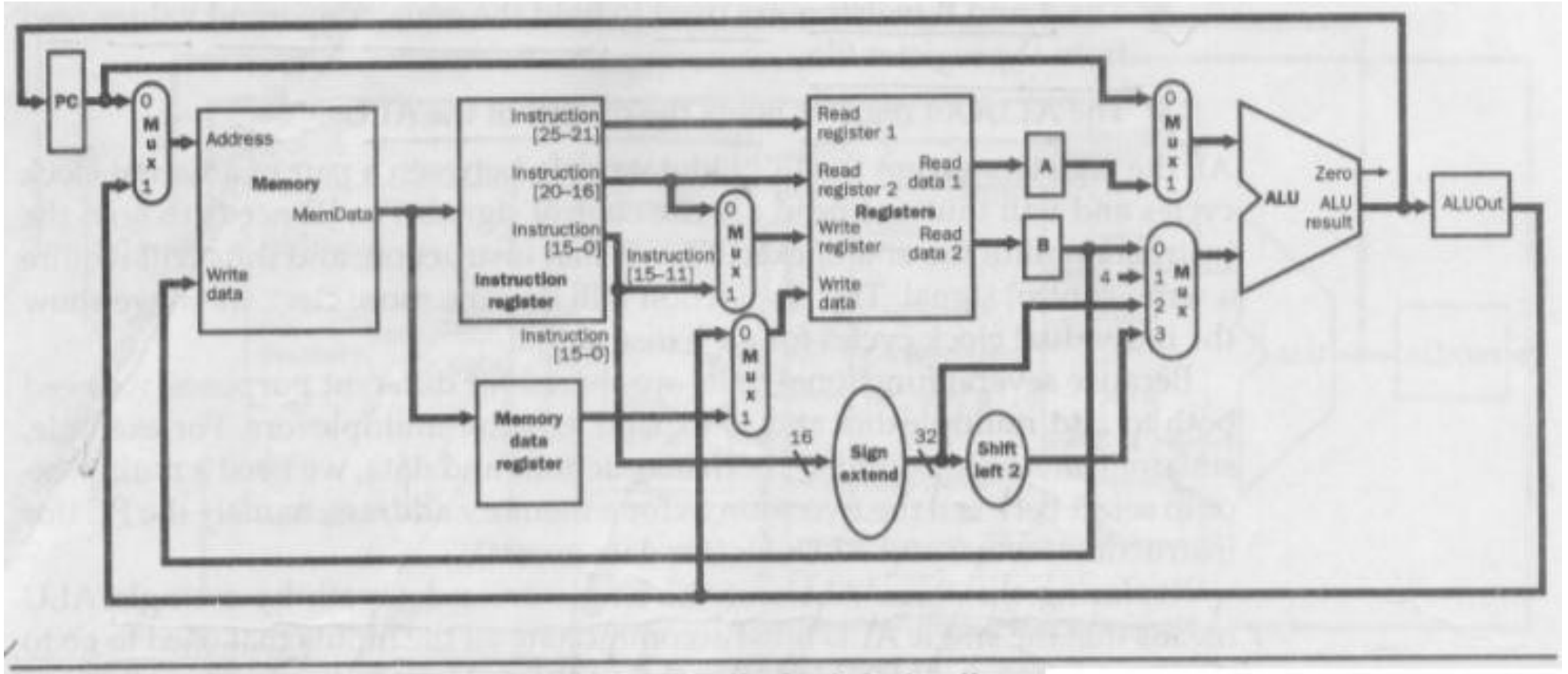


FIGURE 5.31 Multicycle datapath for MIPS handles the basic instructions.

A Multicycle Implementation

With the jump instruction and branch instruction, there are three possible sources for the value to be written into the PC:

1. The output of the ALU, which is the value PC + 4 during instruction fetch. This value should be stored directly into the PC.
2. The register ALUOut, which is where we will store the address of the branch target after it is computed.
3. The lower 26 bits of the Instruction register (IR) shifted left by two and concatenated with the upper 4 bits of the incremented PC, which is the source when the instruction is a jump.

A Multicycle Implementation

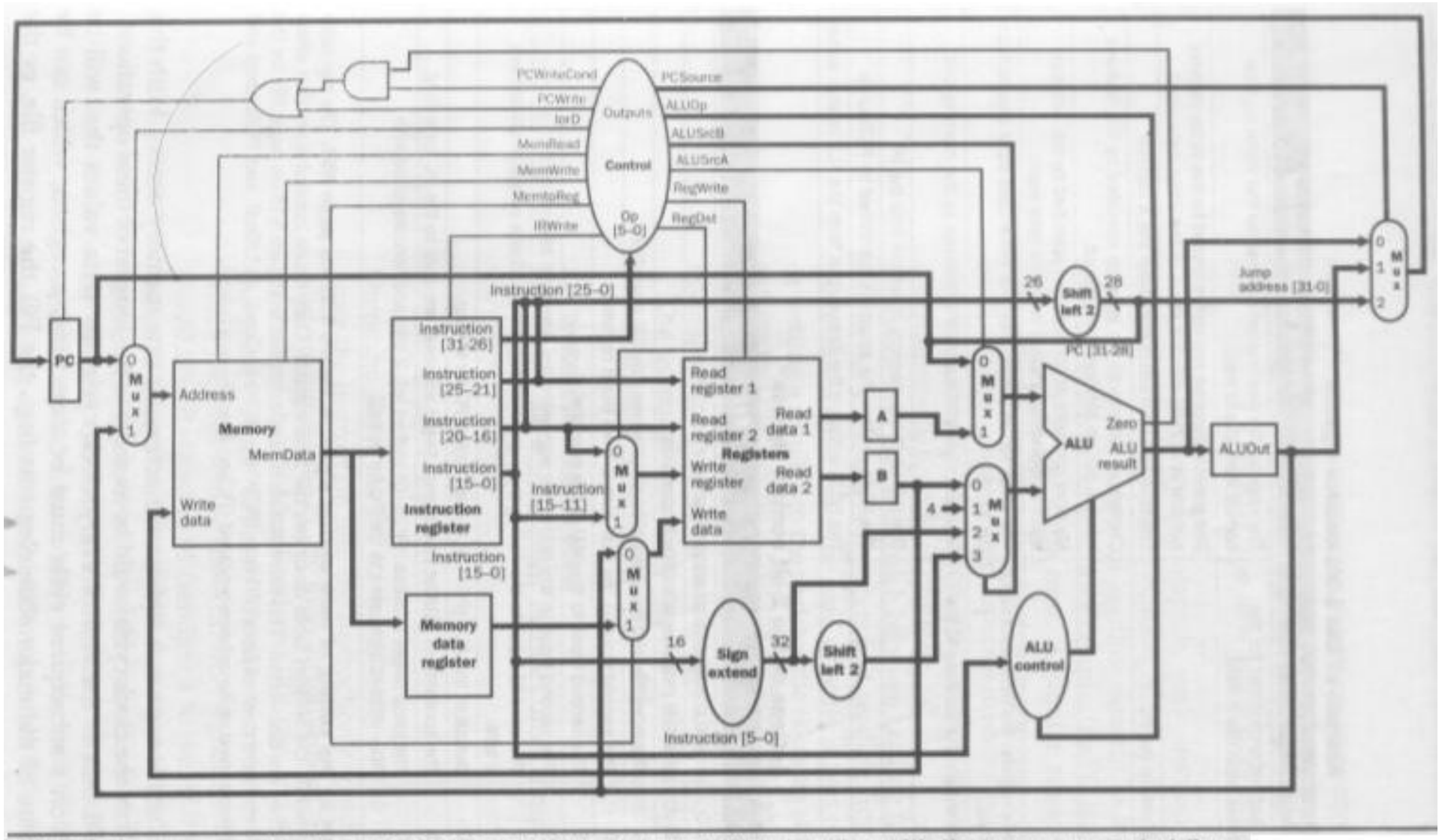


FIGURE 5.33 The complete datapath for the multicycle implementation together with the necessary control lines.

A Multicycle Implementation

Actions of the 1-bit control signals

Signal name	Effect when deasserted	Effect when asserted
RegDst	The register file destination number for the Write register comes from the rt field.	The register file destination number for the Write register comes from the rd field.
RegWrite	None	The general-purpose register selected by the Write register number is written with the value of the Write data input.
ALUSrcA	The first ALU operand is the PC.	The first ALU operand comes from the A register.
MemRead	None	Content of memory at the location specified by the Address input is put on Memory data output.
MemWrite	None	Memory contents at the location specified by the Address input is replaced by value on Write data input.
MemtoReg	The value fed to the register file Write data input comes from ALUOut.	The value fed to the register file Write data input comes from the MDR.
IorD	The PC is used to supply the address to the memory unit.	ALUOut is used to supply the address to the memory unit.
IRWrite	None	The output of the memory is written into the IR.
PCWrite	None	The PC is written; the source is controlled by PCSource.
PCWriteCond	None	The PC is written if the Zero output from the ALU is also active.

Exceptions

An exception is an unexpected event from within the processor.

- Types: due to an undefined instruction and an arithmetic overflow.
- Interrupt

How Exceptions are Handled

(1) Detect the Exception

- The CPU notices something unusual (e.g., div $t0, $t1, $0 - division by zero). Hardware checks for exceptions after each instruction.

(2) Save the Current State -> The CPU backs up two things:

- Current PC (Program Counter): To know where to return after handling the exception.
- Status (Cause Register): Records what went wrong (e.g., "division by zero").

(3) Jump to Exception Handler

- The CPU stops normal execution and jumps to a special exception handler (a fixed memory address, e.g., 0x80000180 in MIPS).
- This is a predefined OS routine (like 911 for emergencies).

(4) Handle the Exception -> The OS decides what to do:

- Fix & Continue: If possible (e.g., page fault: load missing data from disk).
- Terminate Program: If unrecoverable (e.g., invalid instruction).

(5) Return (if possible)

- After fixing, the CPU restores the saved PC and resumes execution.

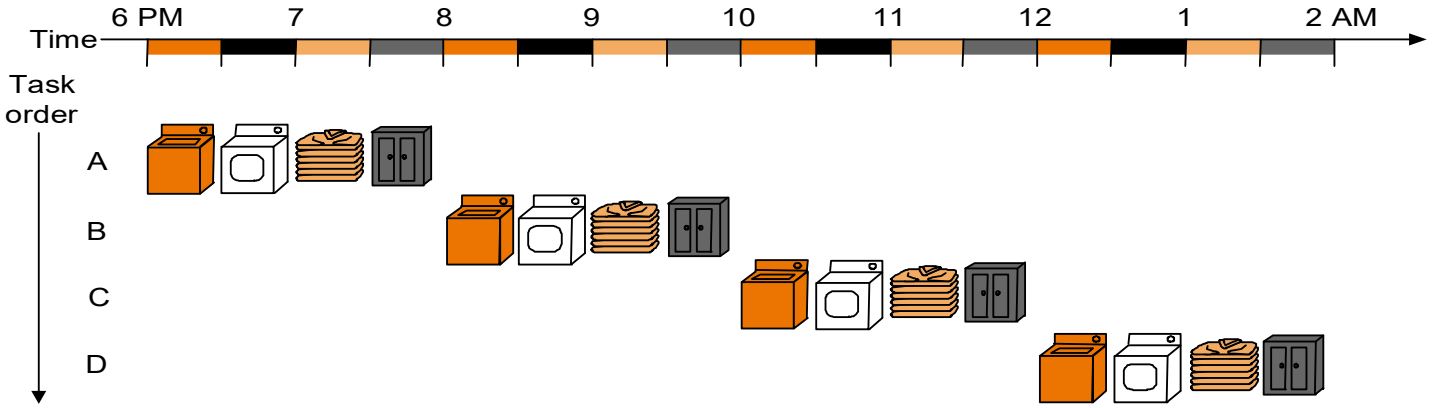
How Exceptions are Handled: Key Components

- EPC (Exception Program Counter): A 32-bit register that saves the address of the faulty instruction.
- Cause Register: Records the exception type (e.g., "interrupt” vs. "invalid instruction").
- Status Register: Tracks if exceptions are enabled/disabled.

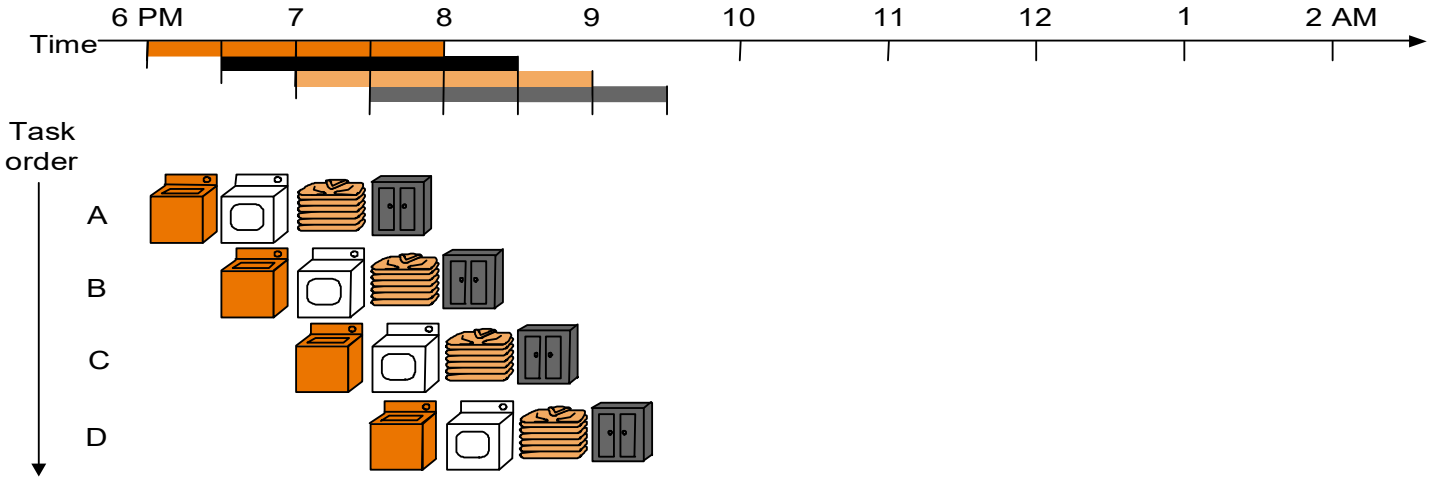
Chapter – 6: Pipeline Hazard

Pipelining

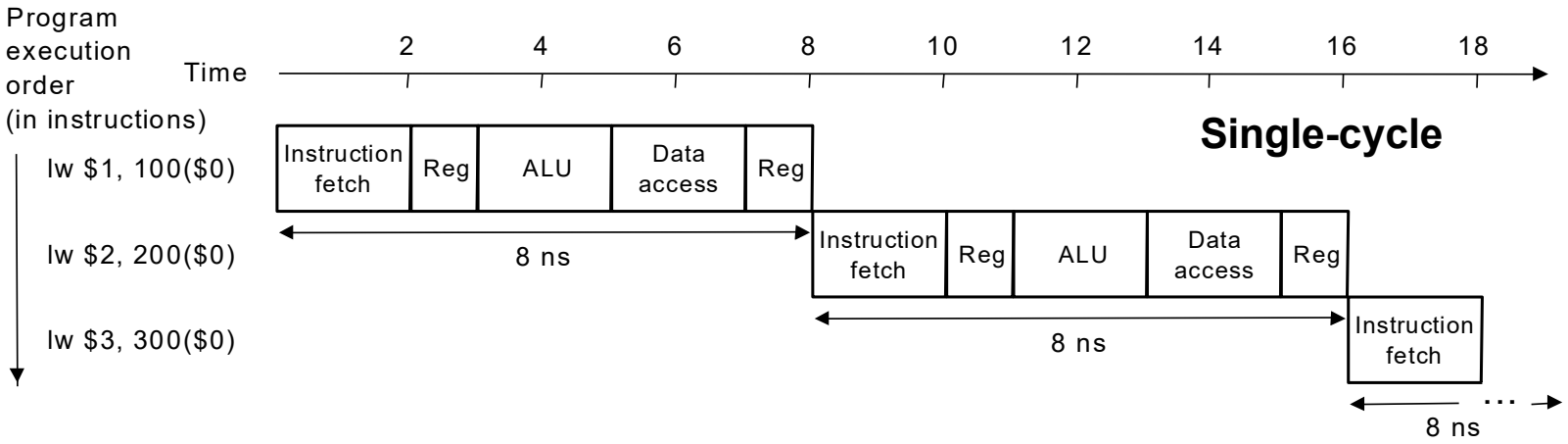
- Start work ASAP!! Do not waste time!



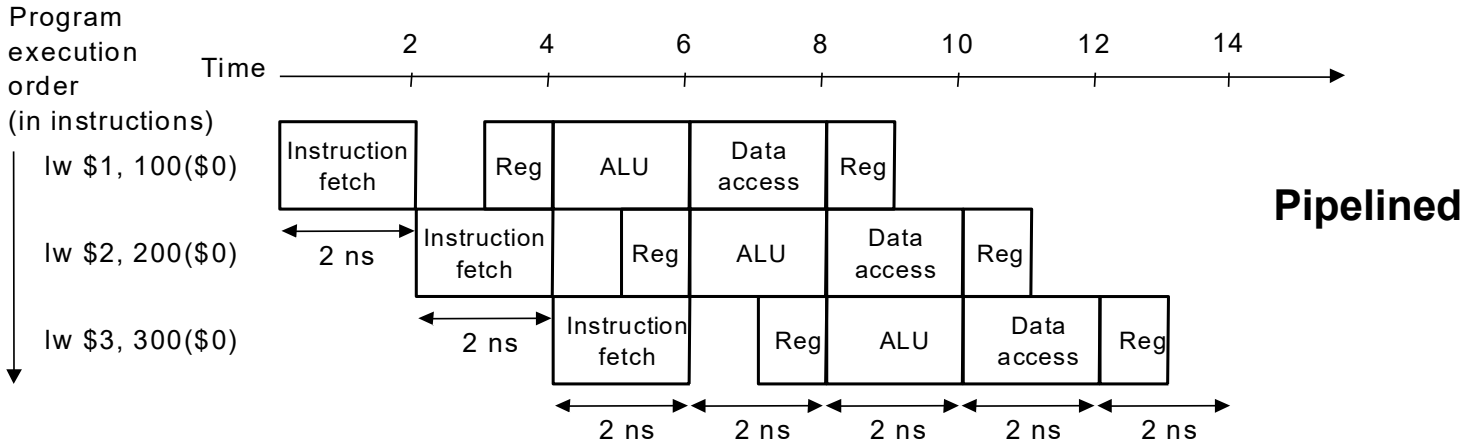
Assume 30 min. each task (wash, dry, fold, store) and that separate tasks use separate hardware and so tasks can be overlapped



Pipelined vs. Single-Cycle Instruction Execution: the Plan



Assume 2 ns for memory access, ALU operation; 1 ns for register access: therefore, single cycle clock 8 ns; pipelined clock cycle 2 ns.



Pipelining: Keep in Mind

- Pipelining *does not reduce latency* of a single task, which means pipelining does NOT reduce the time for a single task. It remains same but total time consumption lessens.
- Pipeline rate *limited by longest stage*
  - *Potential* speedup = Number of Stages
  - *Unbalanced lengths* of pipe stages reduces speedup (Thus, ideal case is all steps takes same time)
- Time to *fill* pipeline and time to *drain* it – when there is *slack* in the pipeline – reduces speedup:
  - In the beginning, you're still filling the pipeline – no output yet.
  - At the end, you're draining it – not all workers are busy.
  - This "slack time" reduces speedup, especially for small jobs.

Example Problem

- *Problem: for the laundry, fill in the following table when*
  1. *the stage lengths are 30, 30, 30 30 min., resp.*
  2. *the stage lengths are 20, 20, 60, 20 min., resp.*

Person	Unpipelined finish time	Pipeline 1 finish time	Ratio unpipelined to pipeline 1	Pipeline 2 finish time	Ratio unpiplelined to pipeline 2
1					
2					
3					
4					
n					

Example Problem: Answer

- *Problem: for the laundry, fill in the following table when*
  1. *the stage lengths are 30, 30, 30 30 min., resp.*
  2. *the stage lengths are 20, 20, 60, 20 min., resp.*

Person	**Unpipelined** finish time	Pipeline 1 finish time	Ratio unpipelined to pipeline 1	Pipeline 2 finish time	Ratio unpiplelined to pipeline 2
1	120	120	1	120	1
2	240	150	1.6	180	1.33
3	360	180	2	240	1.5
4	480	210	2.29	300	1.60
n	120 x n	……..	……	60 x (n+1)	……

Pipelining MIPS

- What makes it easy with MIPS?
  - all *instructions are same length*
    - so fetch and decode stages are similar for all instructions
  - just a *few instruction formats*
    - simplifies instruction decode and makes it possible in one stage
  - *memory operands appear only in load/stores*
    - so memory access can be deferred to exactly one later stage
  - *operands are aligned in memory*
    - one data transfer instruction requires one memory access stage

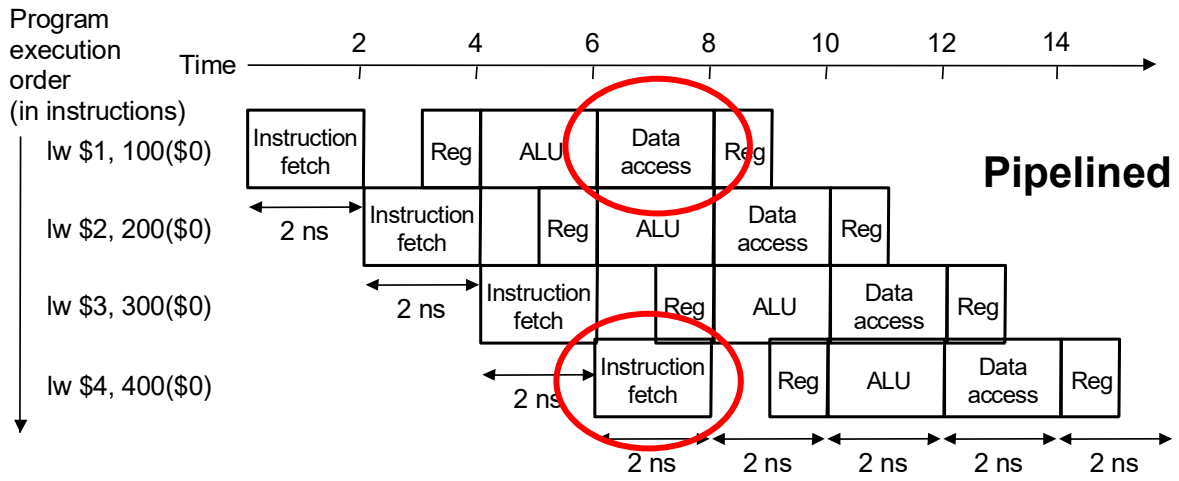
Pipelining MIPS

- What makes it hard?
  - *structural hazards*: different instructions, at different stages, in the pipeline want to use the same hardware resource
  - *control hazards*: succeeding instruction, to put into pipeline, depends on the outcome of a previous branch instruction, already in pipeline
  - *data hazards*: an instruction in the pipeline requires data to be computed by a previous instruction still in the pipeline

- Before actually building the pipelined datapath and control we first briefly examine these potential hazards individually…

Structural Hazards

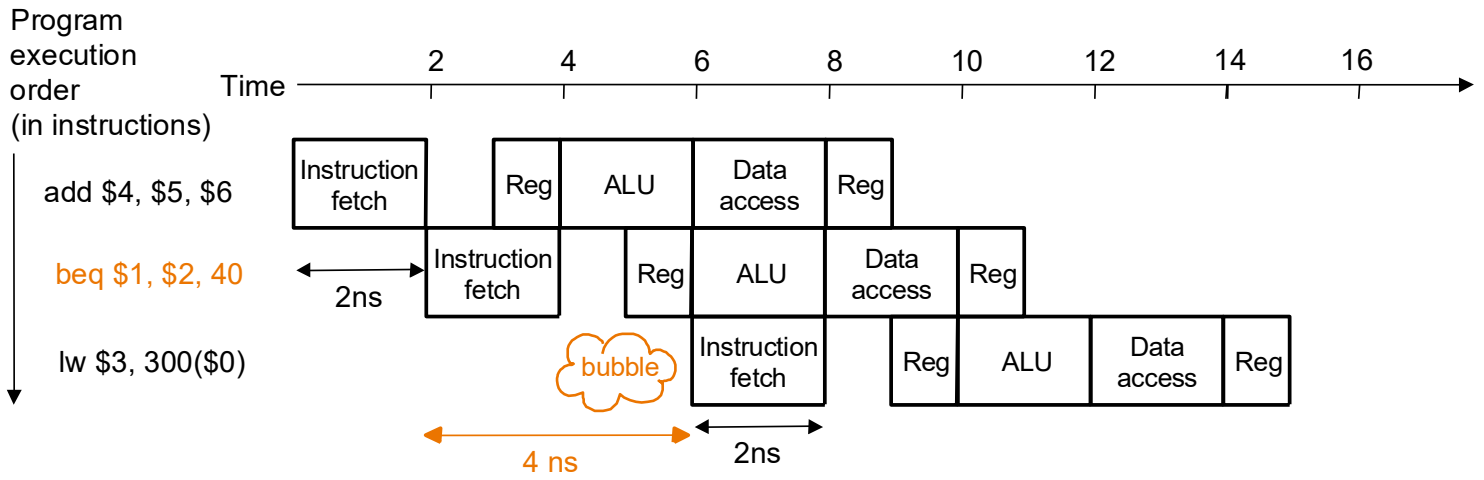
- *Structural hazard*: inadequate hardware to simultaneously support all instructions in the pipeline in the same clock cycle
- E.g., suppose *single – not separate* – instruction and data memory in pipeline below with *one read port*
  - then a structural hazard between first and fourth `lw` instructions



- *MIPS was designed to be pipelined*: structural hazards are easy to avoid!

Control Hazards

- *Control hazard*: need to make a decision based on the result of a previous instruction still executing in pipeline
- Solution 1 *Stall* the pipeline



Pipeline stall

Data Hazard

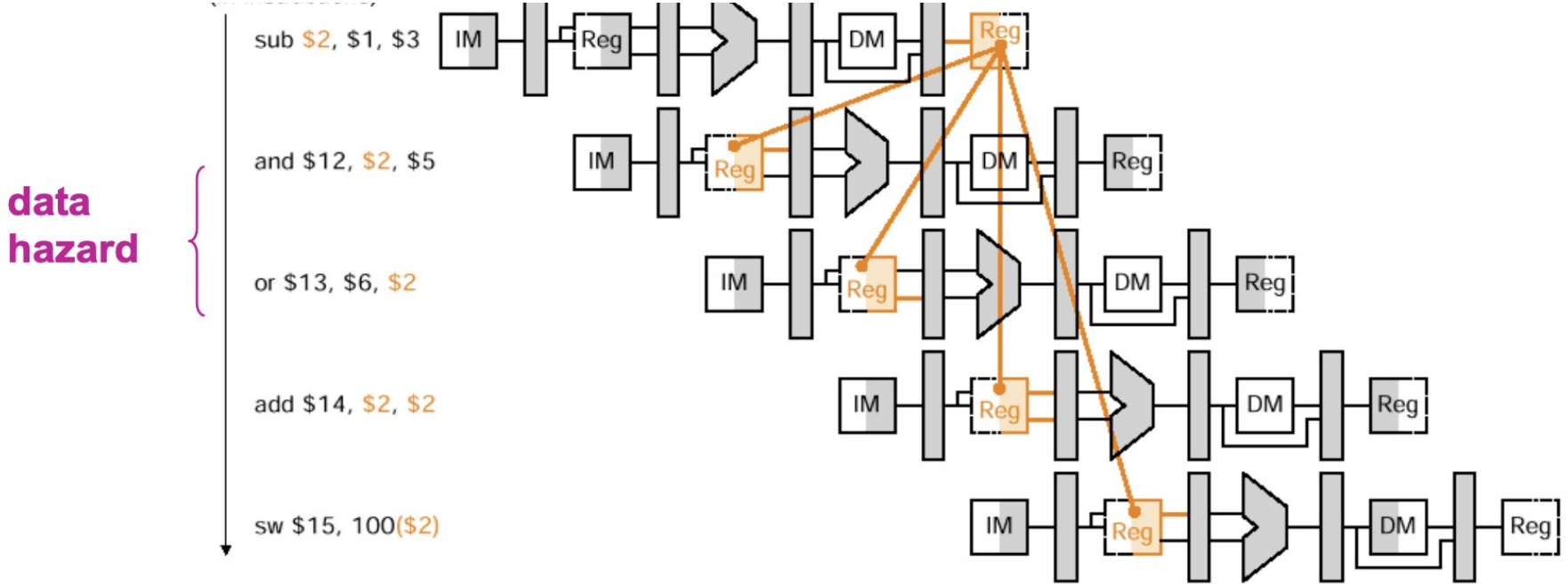
- An instruction depends on the results of a previous instruction still in pipeline.

ex) add $s0, $t0, $t1

sub $t2, $s0, $t3

- For example, if we have these instructions, $s0 register cannot be used in sub instruction.
- Problem with starting next instruction before first is finished.

Data Hazard

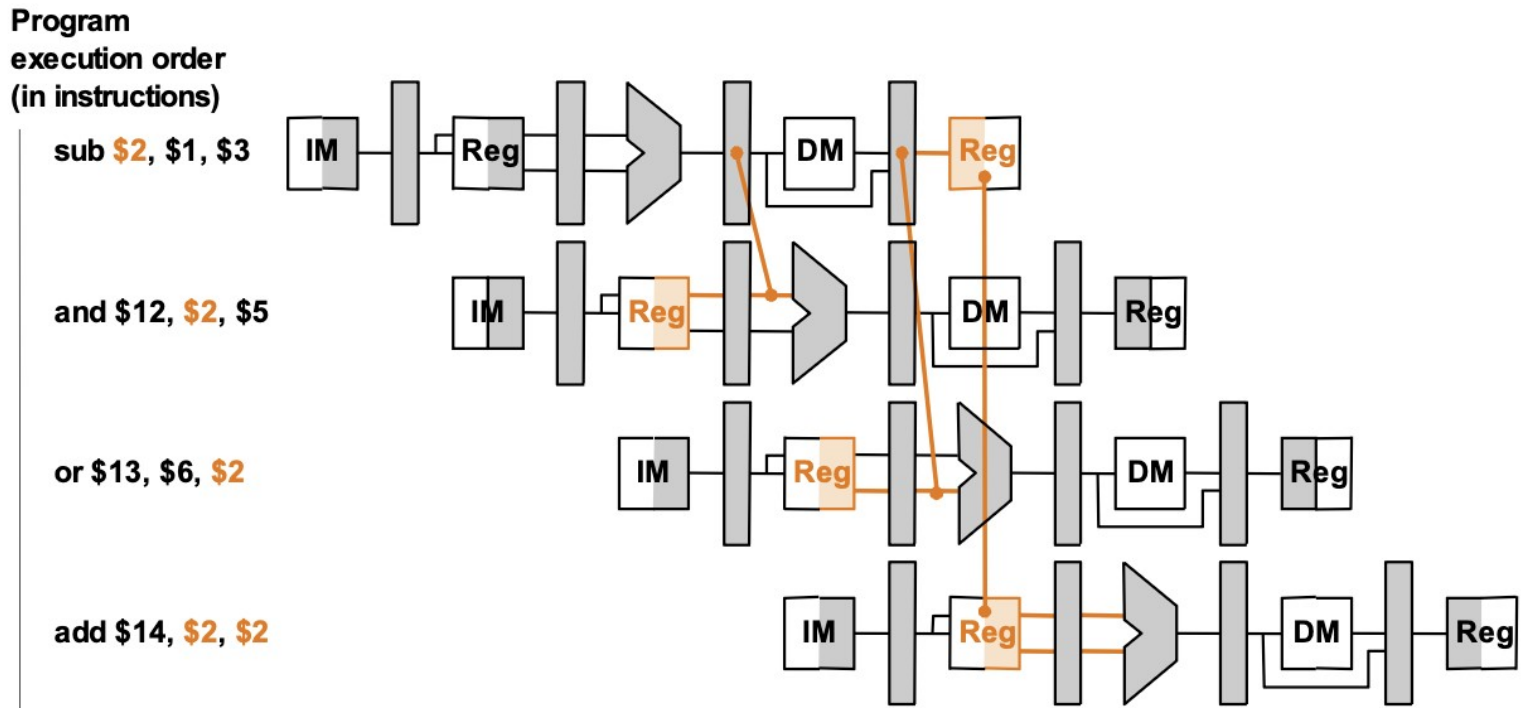


Solutions for Data Hazards

Stalling: We can stall other instructions until previous instruction executed.

Forwarding: We can create the path to send the result of instruction or calculation.

Data Hazard: Forwarding



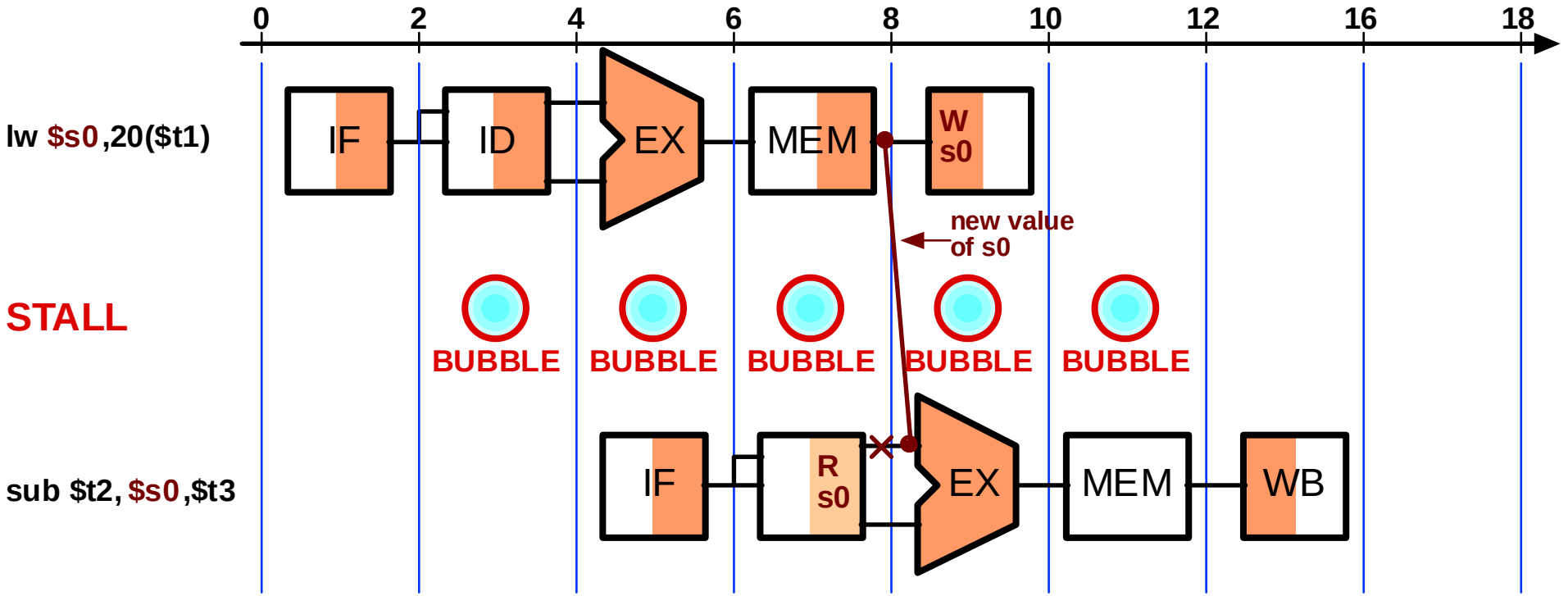
Pipeline stages of MIPS instruction

The 5 steps in instruction execution

1. Instruction Fetch & PC Increment (IF)
2. Instruction Decode and Register Read (ID)
3. Execution or calculate address (EX)
4. Memory access (MEM)
5. Write result into register (WB)

Data Hazards - Stalling

- STALL still required for load - data avail. after



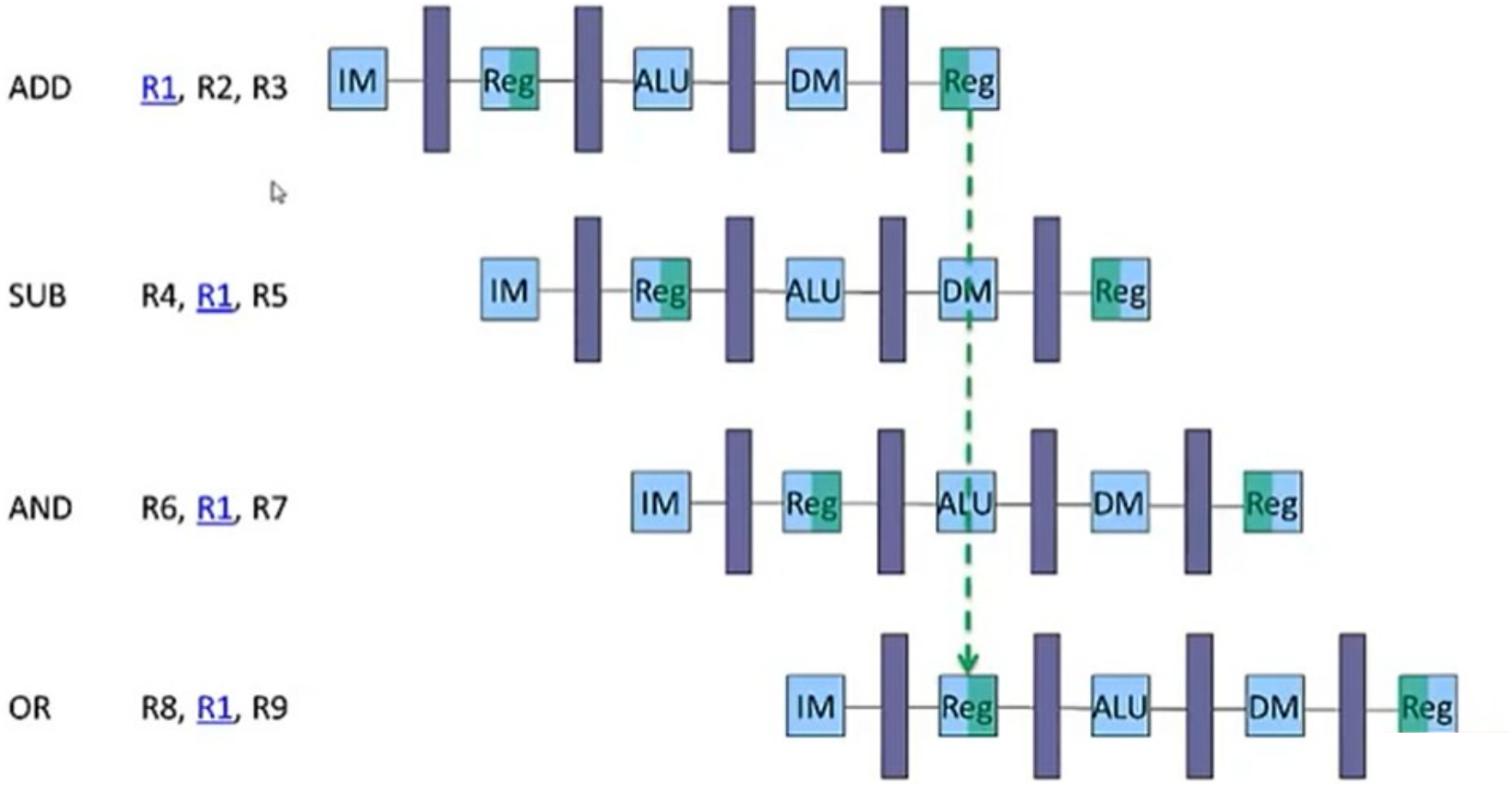
Data Hazards: Another representation of the stall

LW R1, 0(R2)	IF	ID	EX	MEM	WB			
SUB R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
OR R8, R1, R9				IF	ID	EX	MEM	WB

LW R1, 0(R2)	IF	ID	EX	MEM	WB				
SUB R4, R1, R5		IF	ID	stall	EX	MEM	WB		
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB	
OR R8, R1, R9				stall	IF	ID	EX	MEM	WB

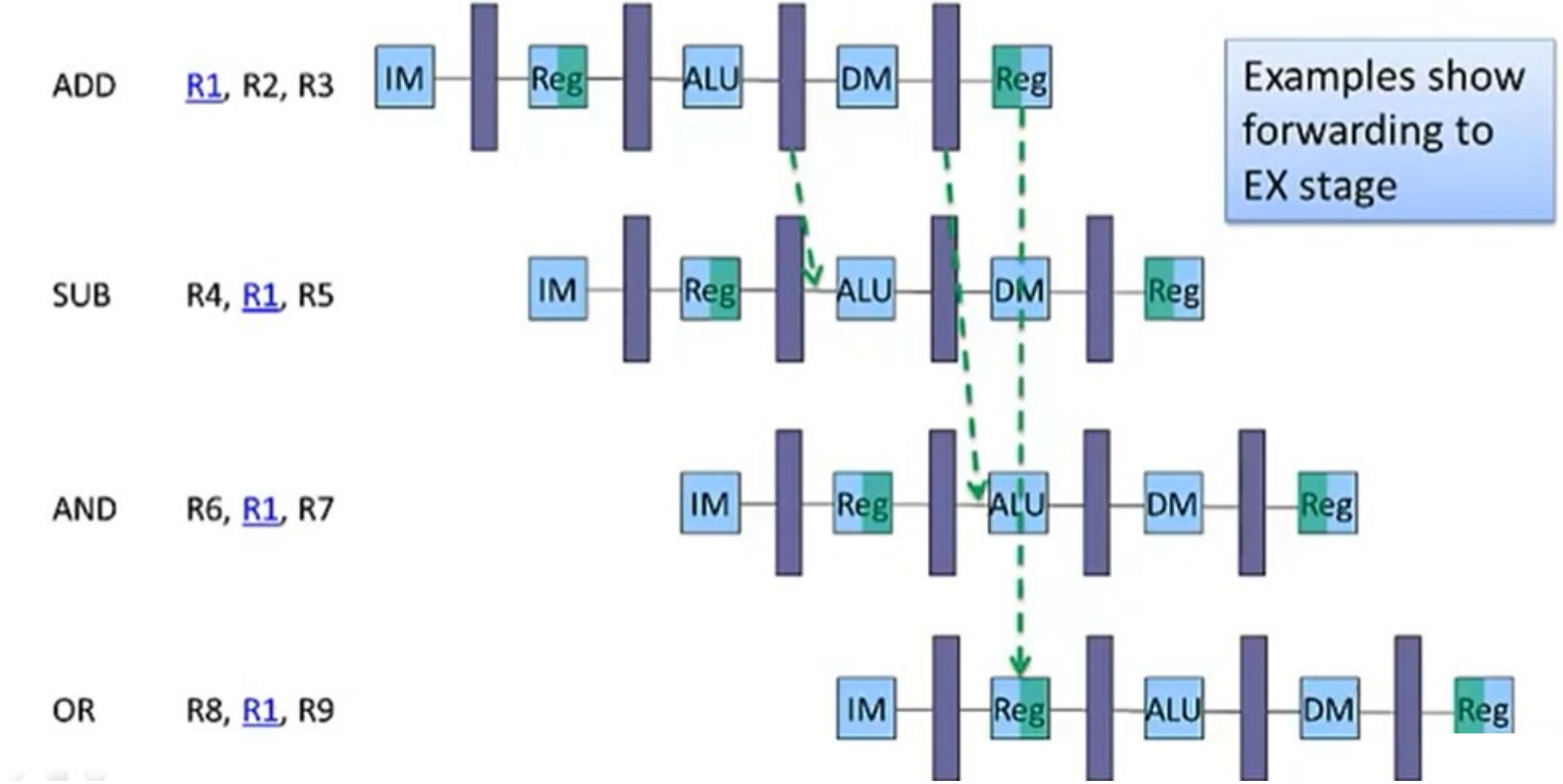
Data Hazard: Can we do better?

One solution can be Read after Write, but it can't solve all the data hazard in every stage



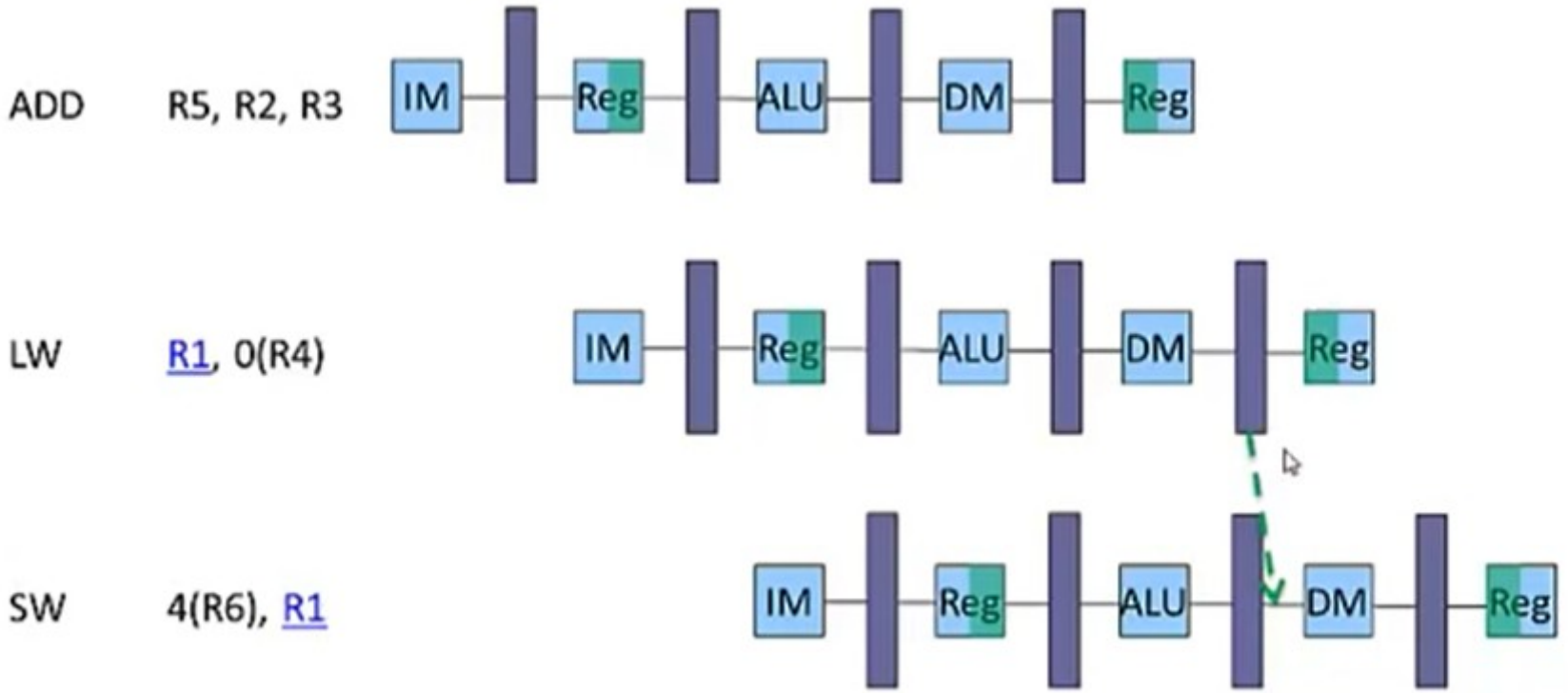
Data Hazard: Can we do better?

- We can minimize stalls by data forwarding that involves all the instructions!



Data Hazard: What if we have lw and sw?

- Data forwarded to Memory stage:



Reordering Code to Avoid Pipeline Stall (Software Solution)

- Example:

```
lw $t0, 0($t1)
lw $t2, 4($t1)
sw $t2, 0($t1)
sw $t0, 4($t1)
```



- Reordered code:

```
lw $t0, 0($t1)
lw $t2, 4($t1)
sw $t0, 4($t1)
sw $t2, 0($t1)
```

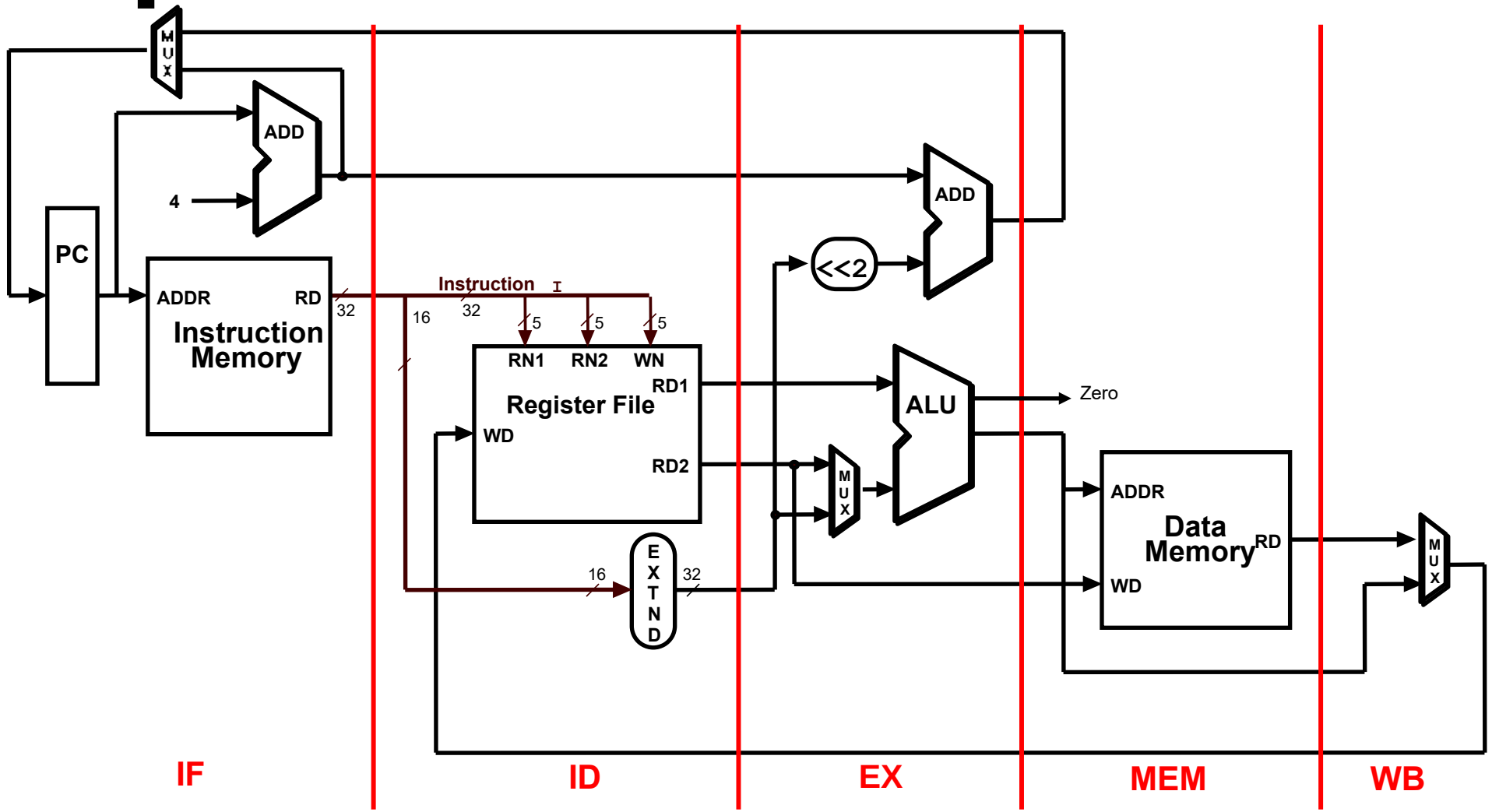


Pipelined Datapath

- We now move to actually building a pipelined datapath
- First recall the 5 steps in instruction execution
  1. Instruction Fetch & PC Increment (IF)
  2. Instruction Decode and Register Read (ID)
  3. Execution or calculate address (EX)
  4. Memory access (MEM)
  5. Write result into register (WB)
- Review: single-cycle processor
  - all 5 steps done in a single clock cycle
  - dedicated hardware required for each step

- *What happens if we break the execution into multiple cycles, but keep the extra hardware?*

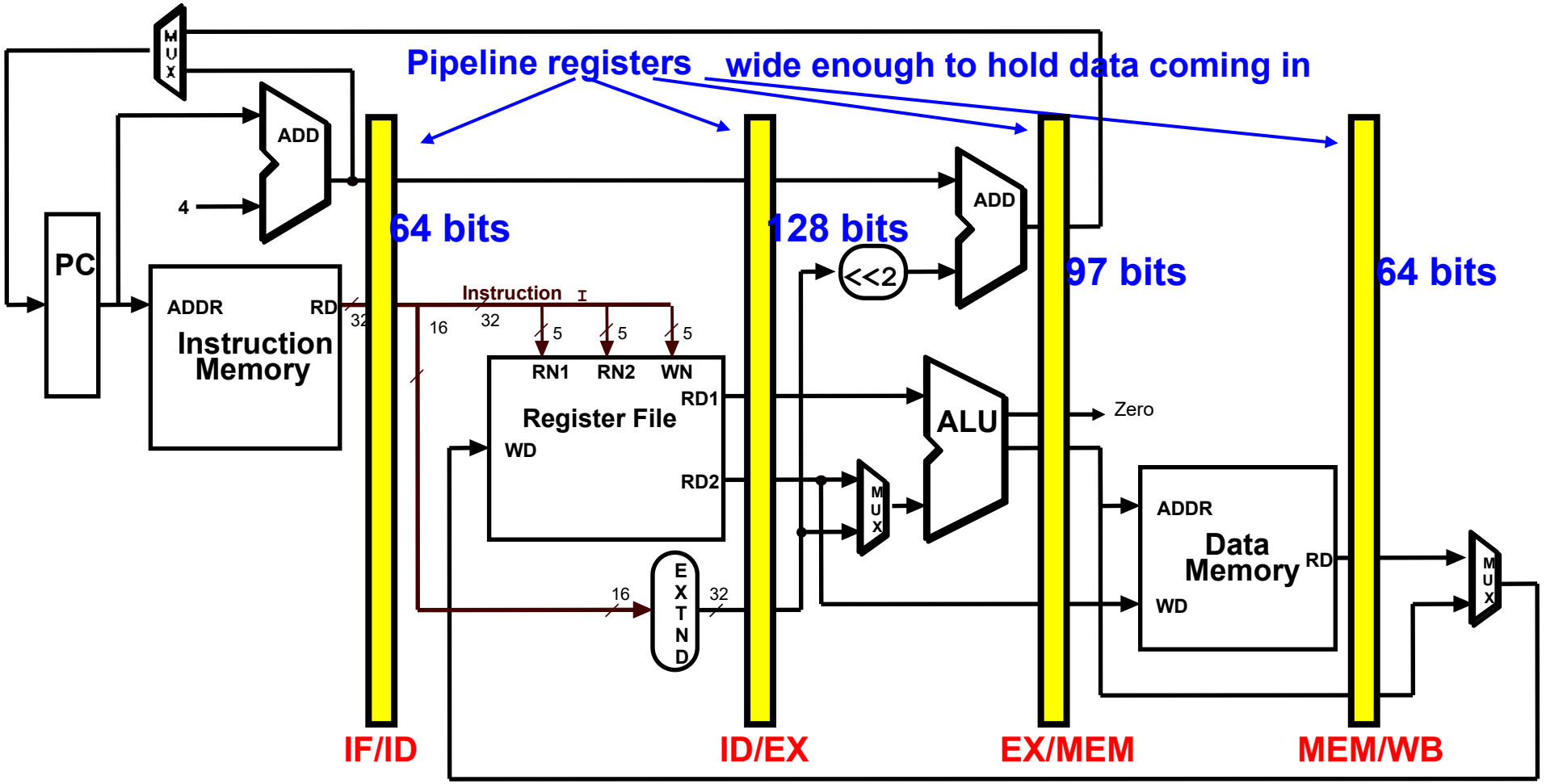
Review - Single-Cycle Datapath "Steps"



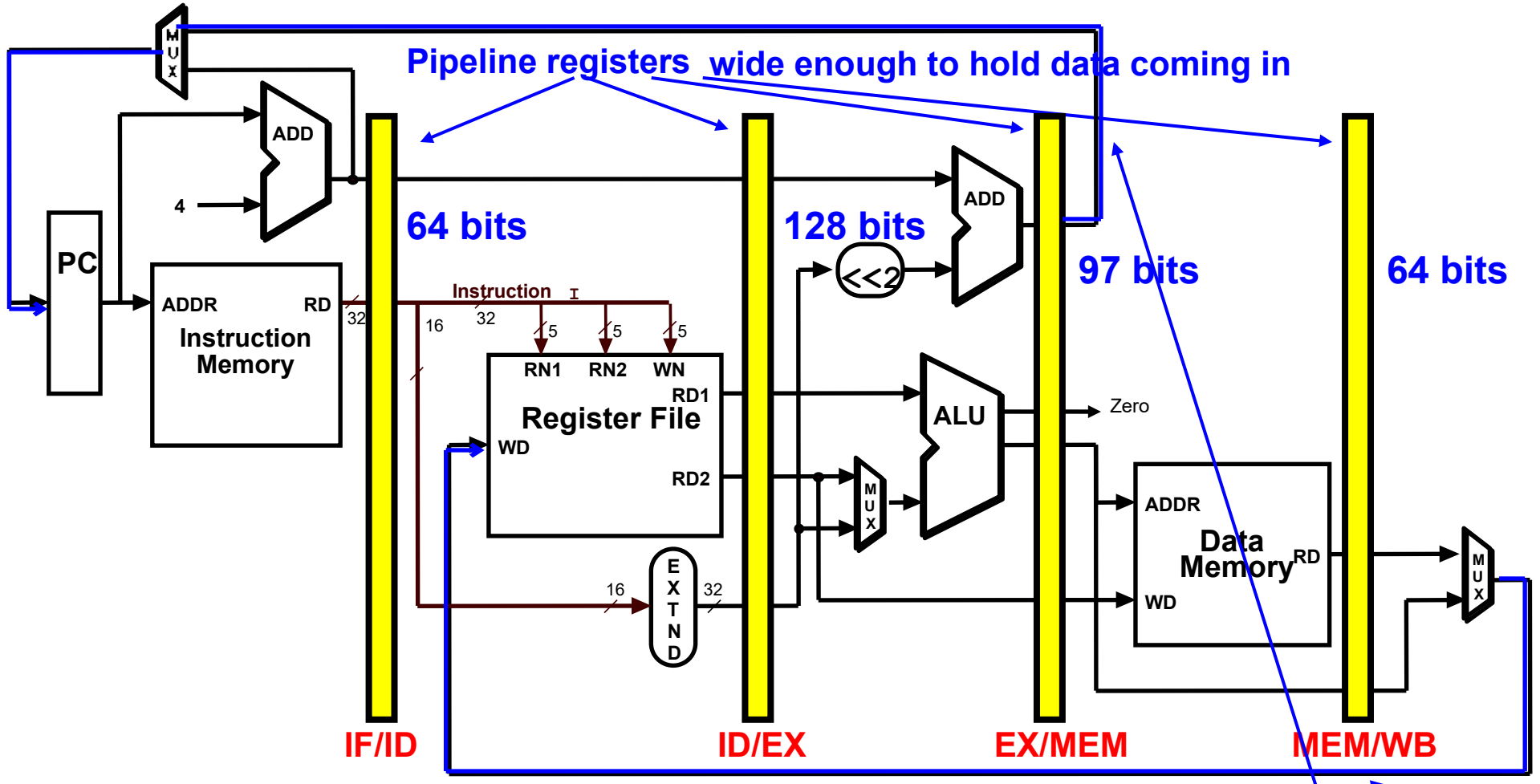
Pipelined Datapath – Key Idea

- *What happens if we break the execution into multiple cycles, but keep the extra hardware?*
  - Answer: *We may be able to start executing a new instruction at each clock cycle* - pipelining
- …but we shall need *extra* registers to hold data between cycles – *pipeline registers*

Pipelined Datapath

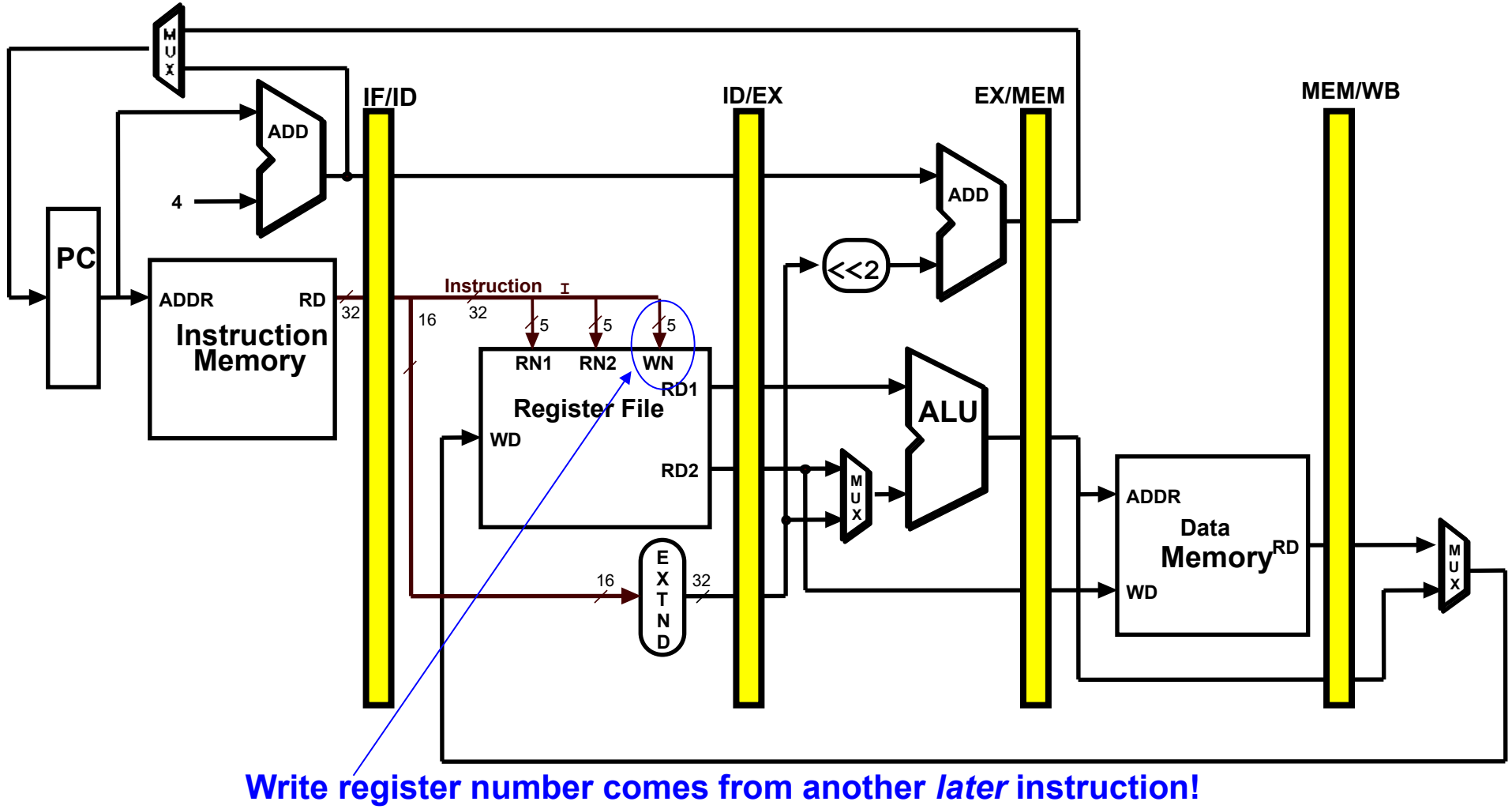


Pipelined Datapath

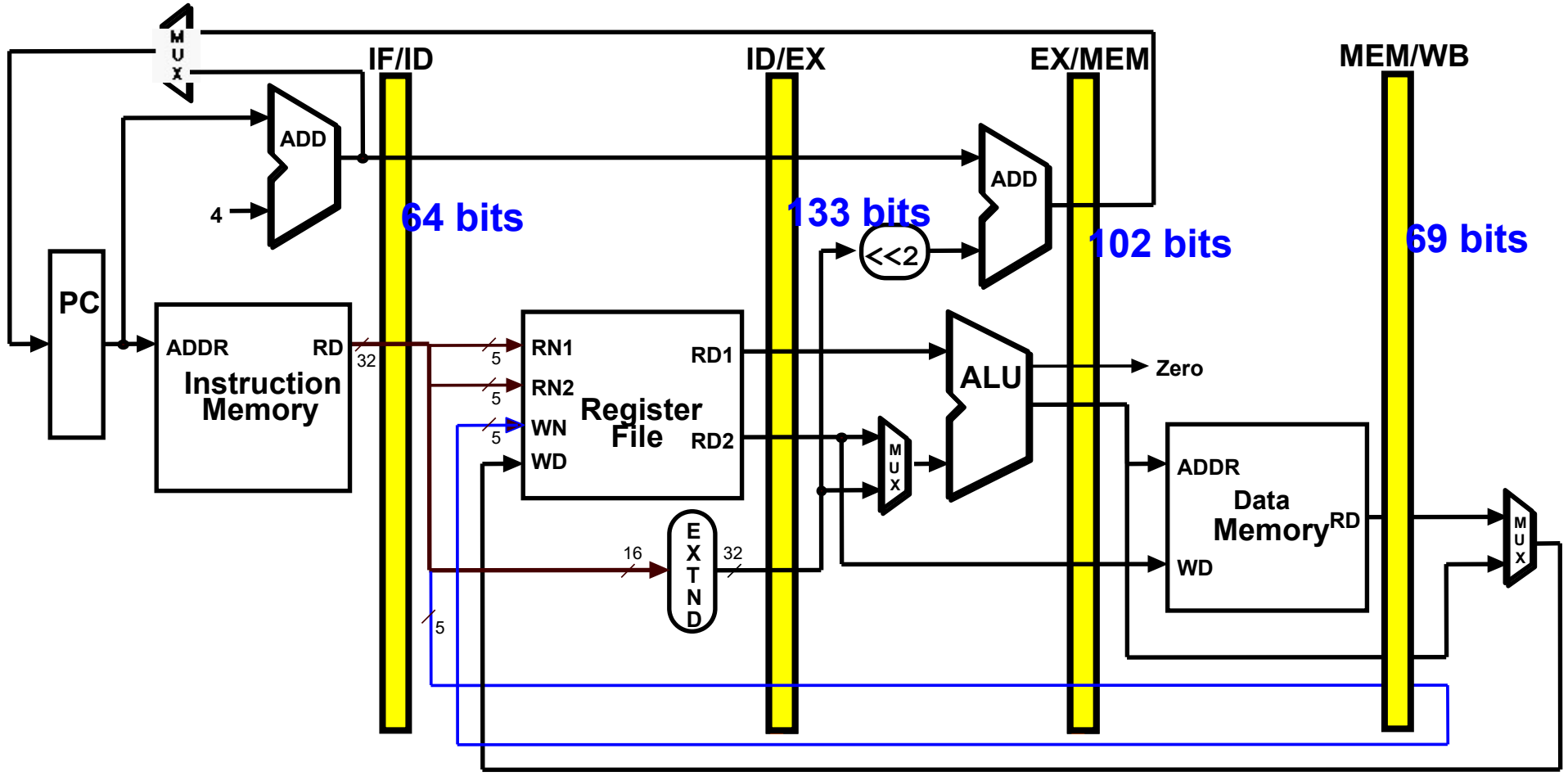


Only data flowing right to left may cause hazard…, why?

Bug in the Datapath



Corrected Datapath



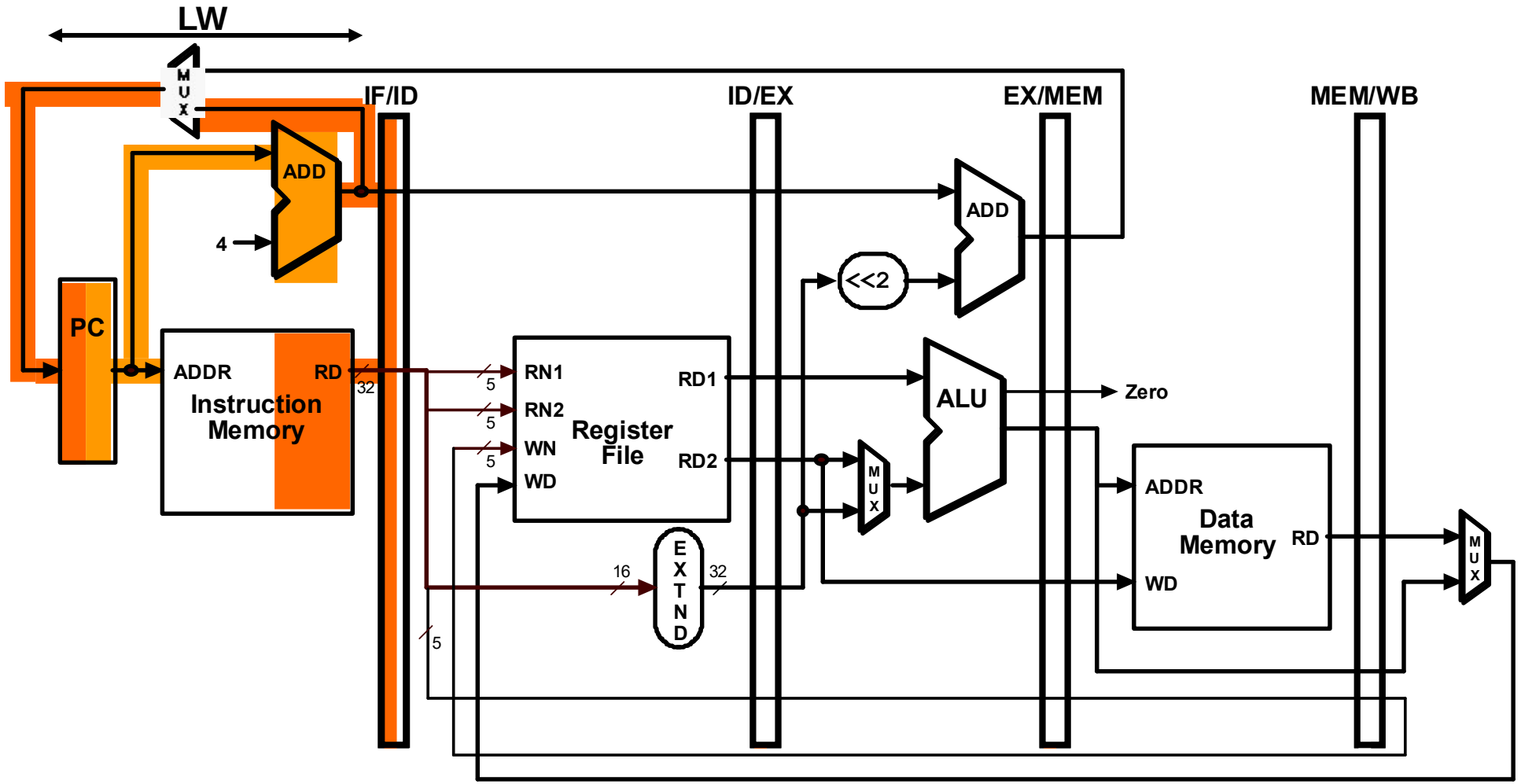
Destination register number is also passed through ID/EX, EX/MEM and MEM/WB registers, which are now wider by 5 bits

Pipelined Example

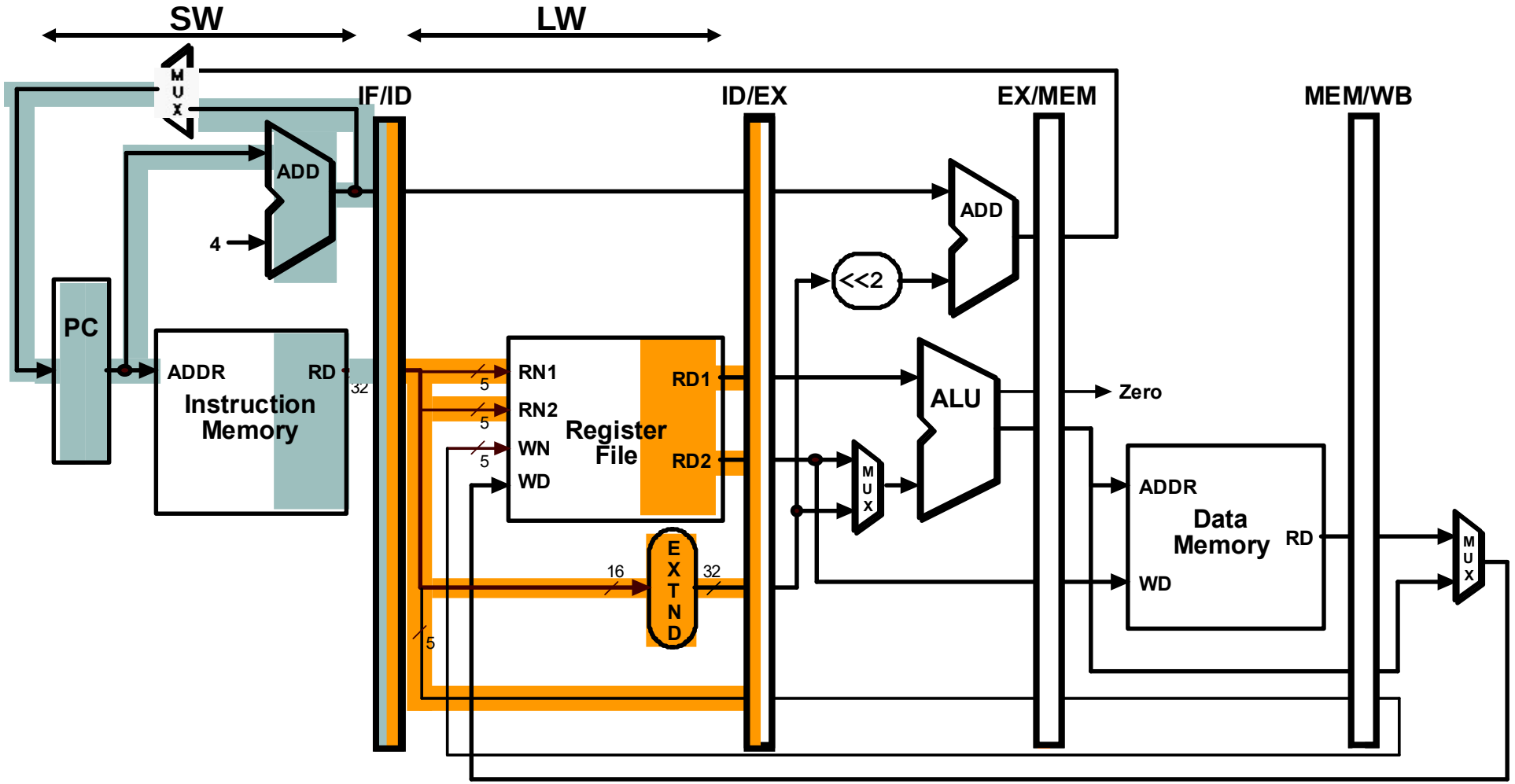
- Consider the following instruction sequence:

```
lw  $t0,  10($t1)
sw  $t3, 20($t4)
add $t5,  $t6,  $t7
sub $t8,  $t9,  $t10
```

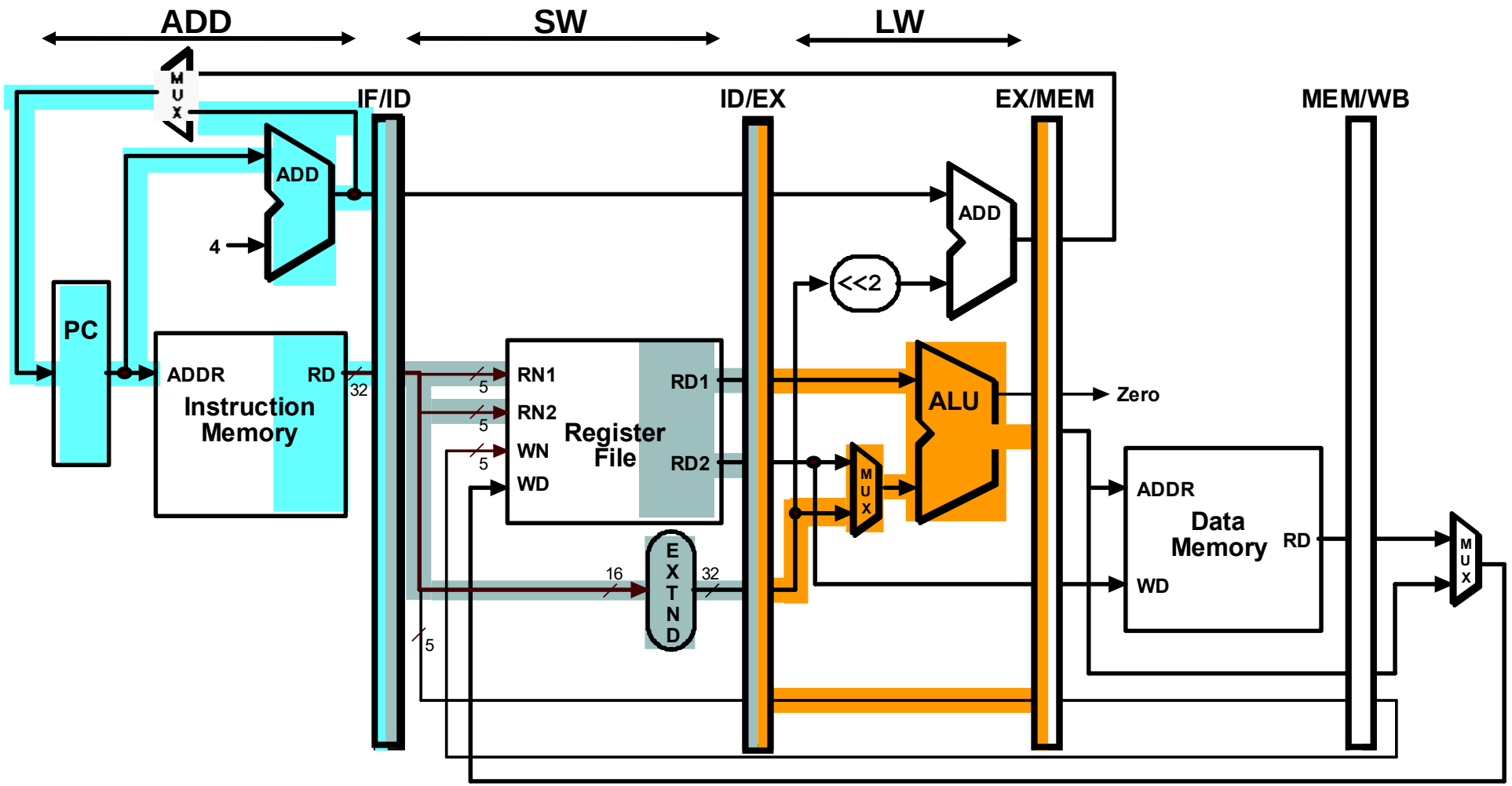
Single-Clock-Cycle Diagram: Clock Cycle 1



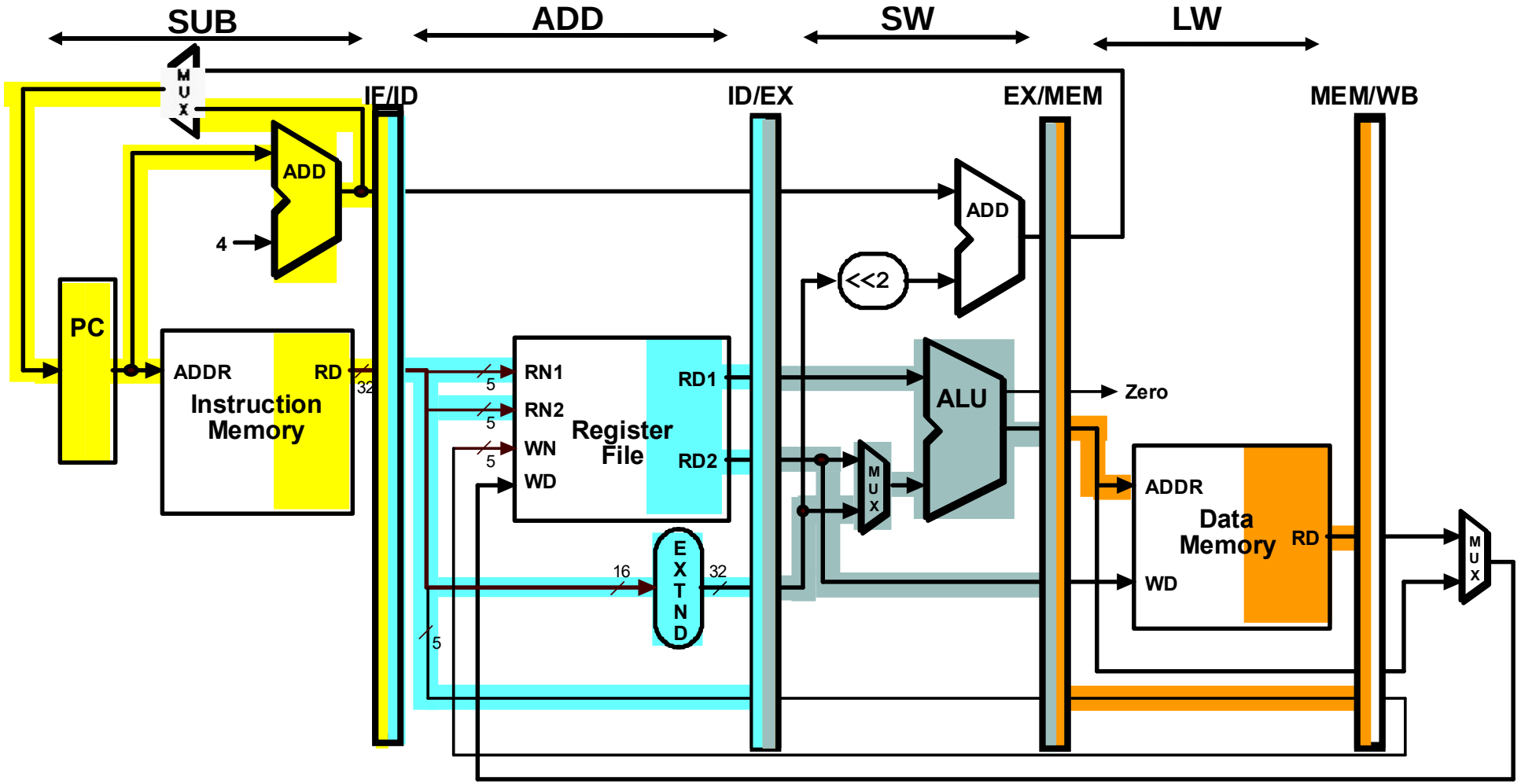
Single-Clock-Cycle Diagram: Clock Cycle 2



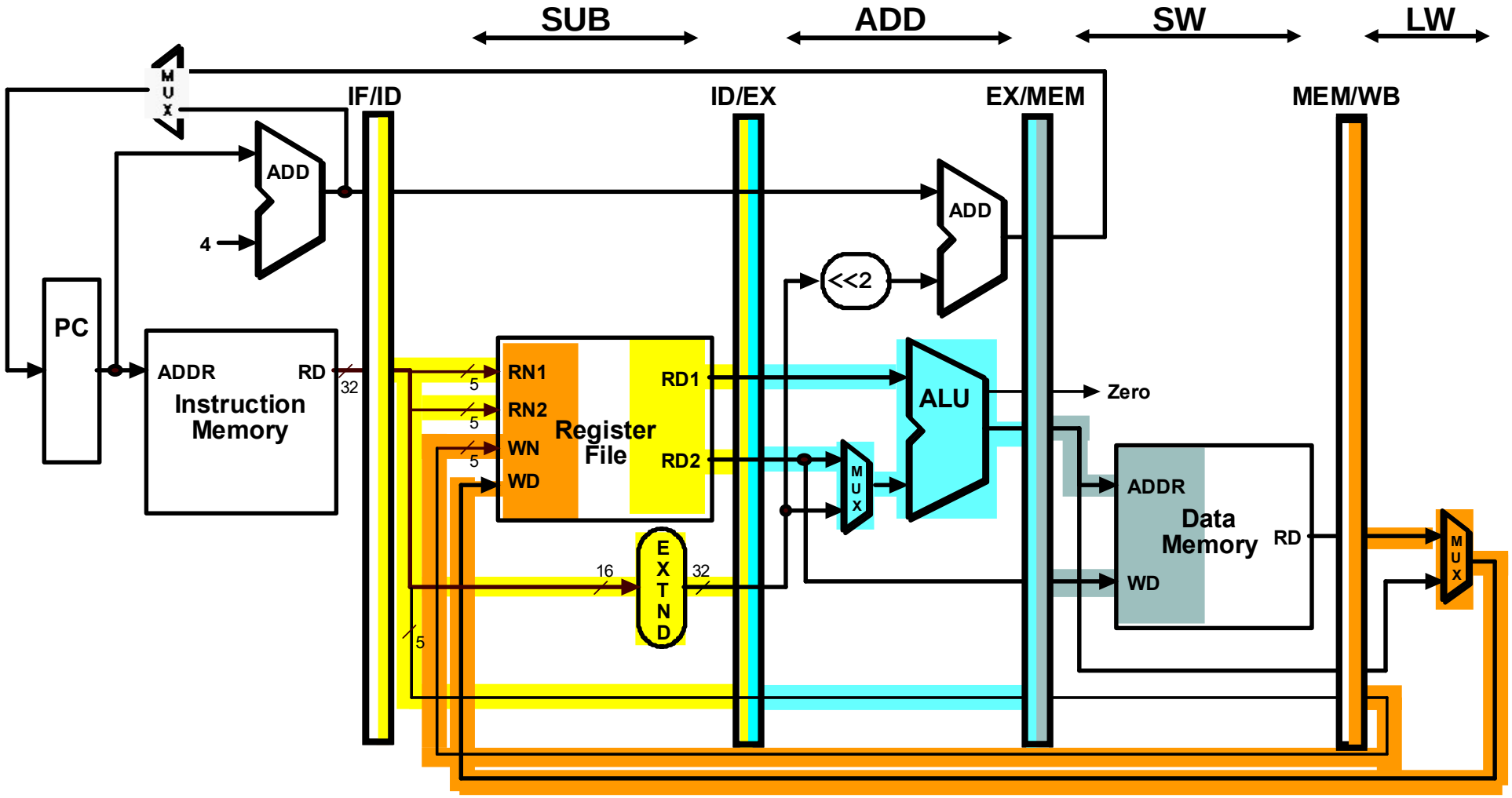
Single-Clock-Cycle Diagram: Clock Cycle 3



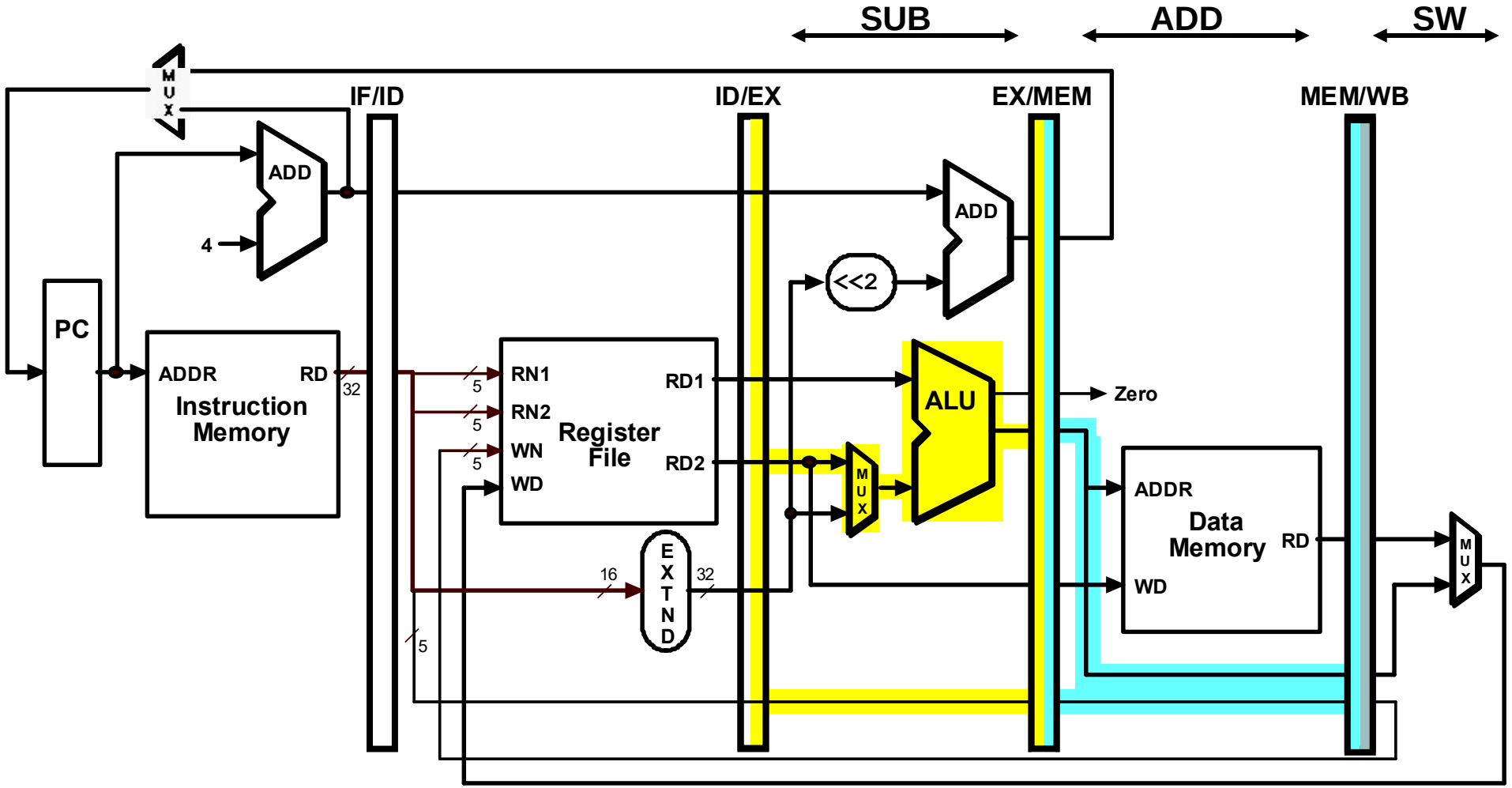
Single-Clock-Cycle Diagram: Clock Cycle 4



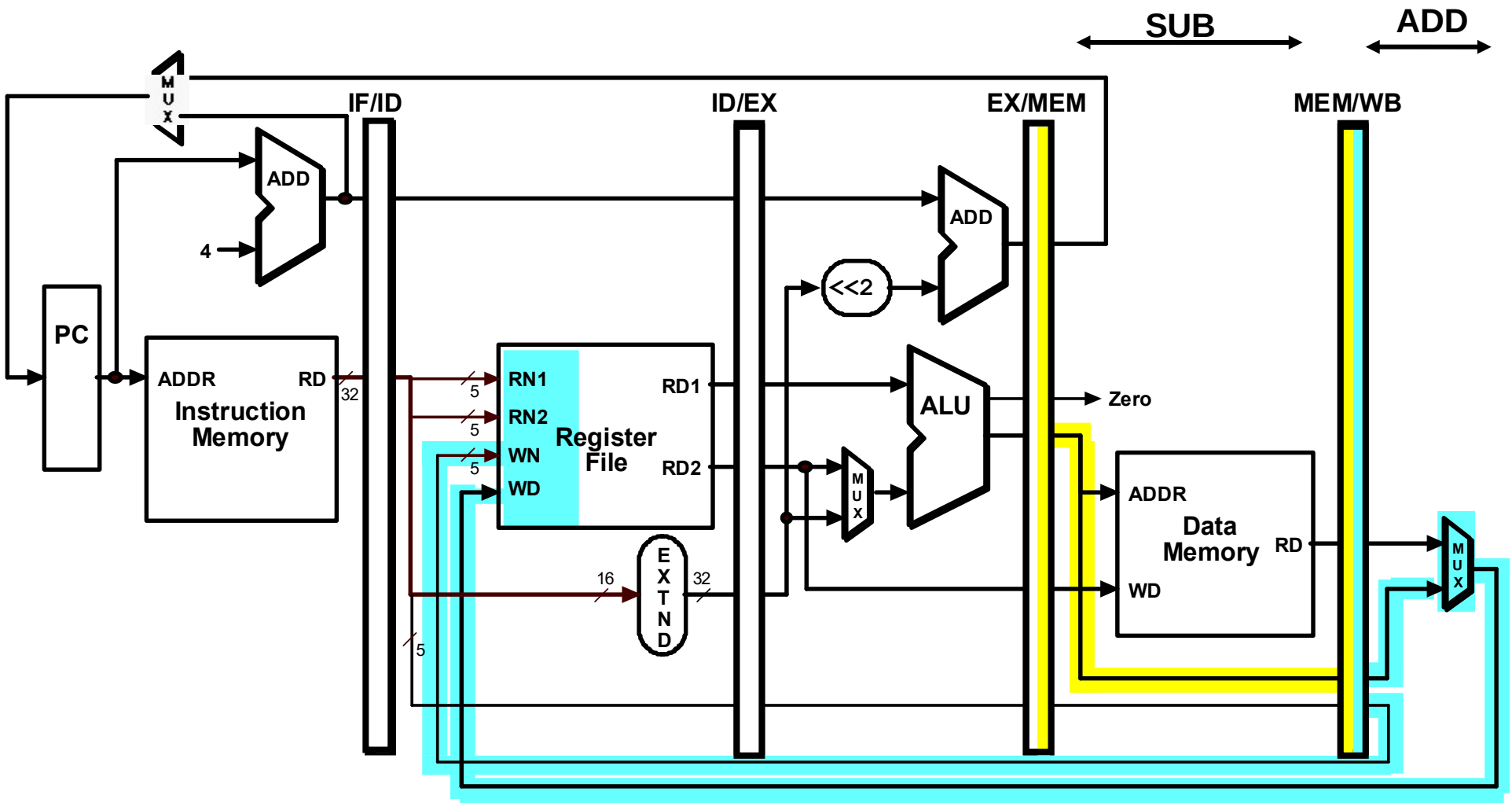
Single-Clock-Cycle Diagram: Clock Cycle 5



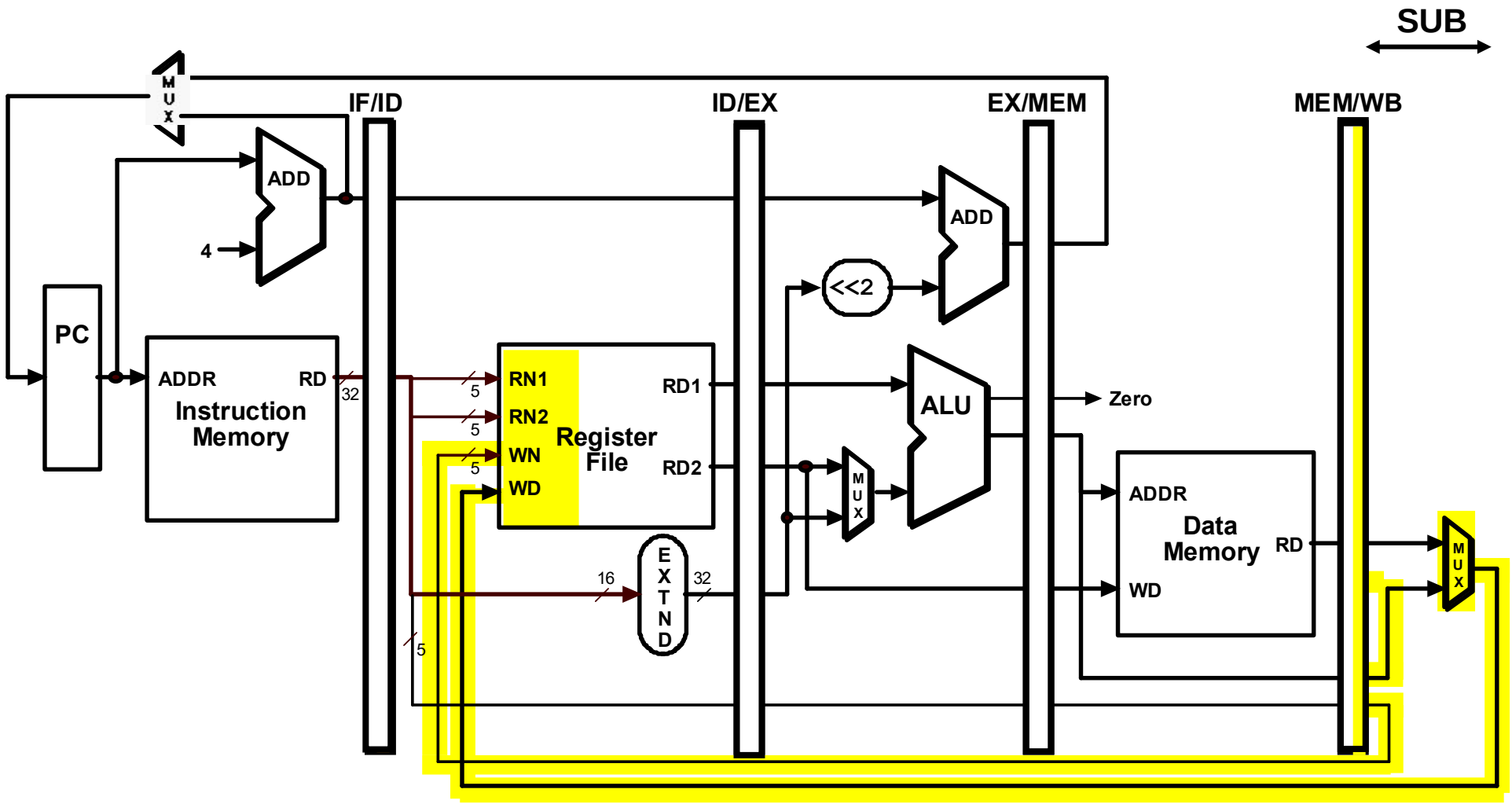
Single-Clock-Cycle Diagram: Clock Cycle 6



Single-Clock-Cycle Diagram: Clock Cycle 7



Single-Clock-Cycle Diagram: Clock Cycle 8



Superscalar and Dynamic Pipelining

- In the interest of even faster processors, there have been three major directions:
- Super pipelining: It simply means longer pipelines.
  - Since the ideal maximum from pipelining is related to the number of pipeline stages, some recent microprocessors speedup have gone to pipelines with eight or more stages.
- Superscalar: Usage of duplicate components to replicate the internal components of the computer so that it can launch multiple instructions in every pipeline stage.
- Dynamic pipeline scheduling / Dynamic pipelining: Dynamic pipelining is normally combined with extra hardware resources so later instructions can proceed in parallel.
  - Divided into three major units:
    - Instruction fetch and issue unit
    - Execute units
    - Commit unit

Dynamic Pipelining Scheduling

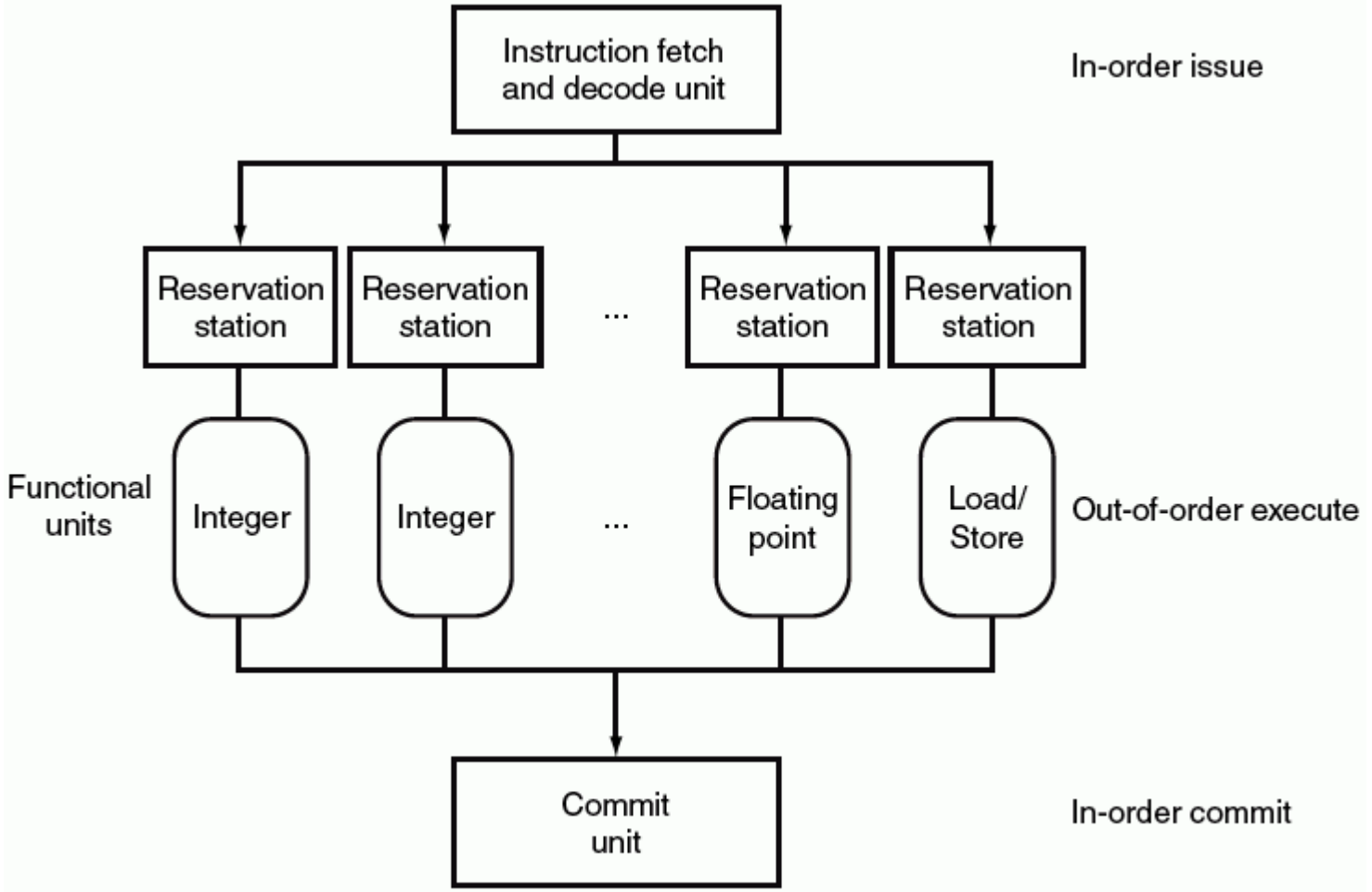


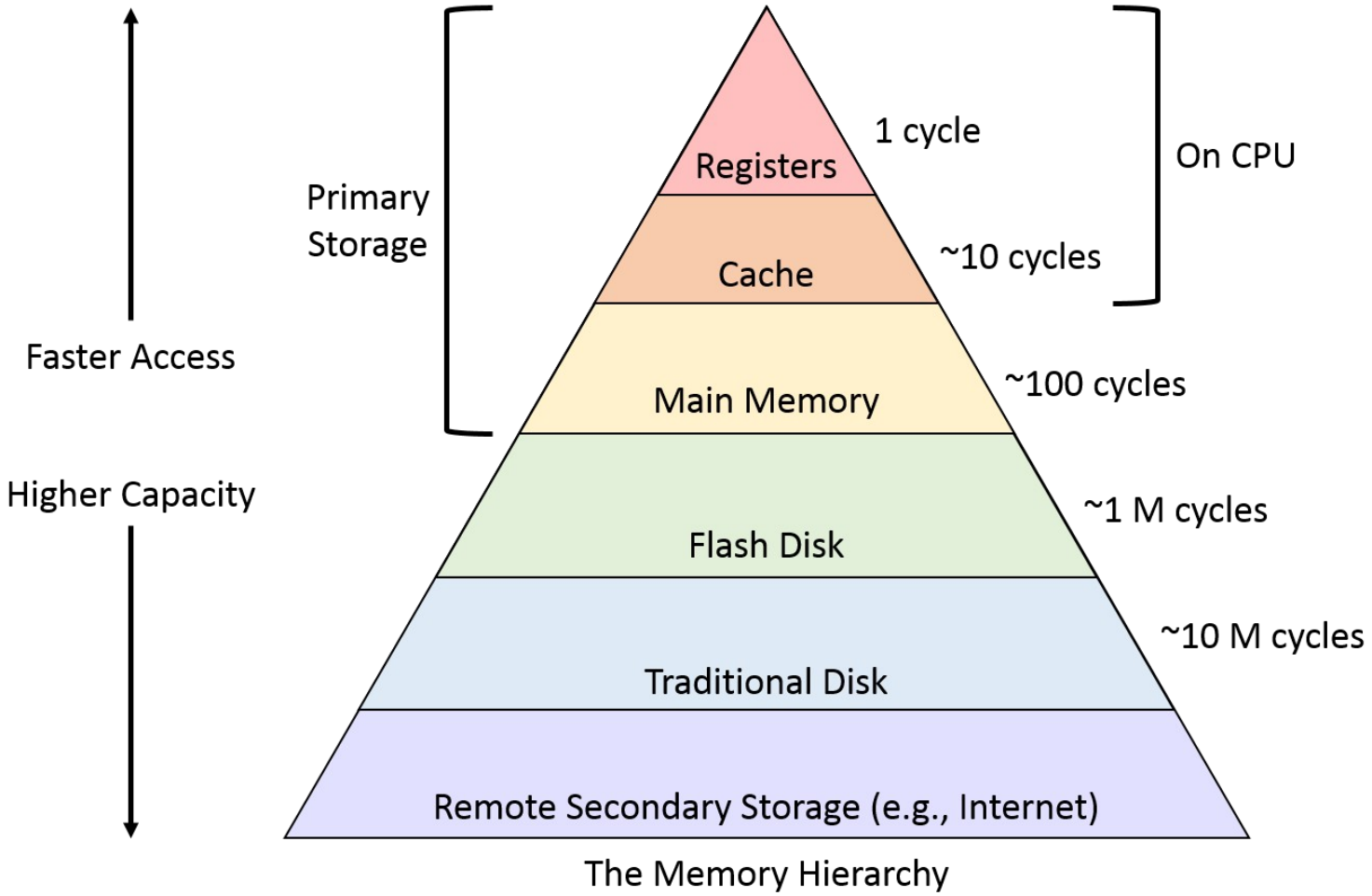
FIGURE 6.49 The three primary units of a dynamically scheduled pipeline.

Superscalar and Dynamic Pipelining

- In-order completion
- Out-of-order completion
- Dynamic pipelining is more complicated than the traditional or static pipelining – why?
  - Dynamic scheduling is normally combined with branch prediction, so the commit unit must be able to discard all the results in the execution units that were due to instructions executed after a mispredicted branch.
  - Dynamic scheduling is also typically combined with superscalar execution, so each unit may be issuing or committing four to six instructions each clock cycle.

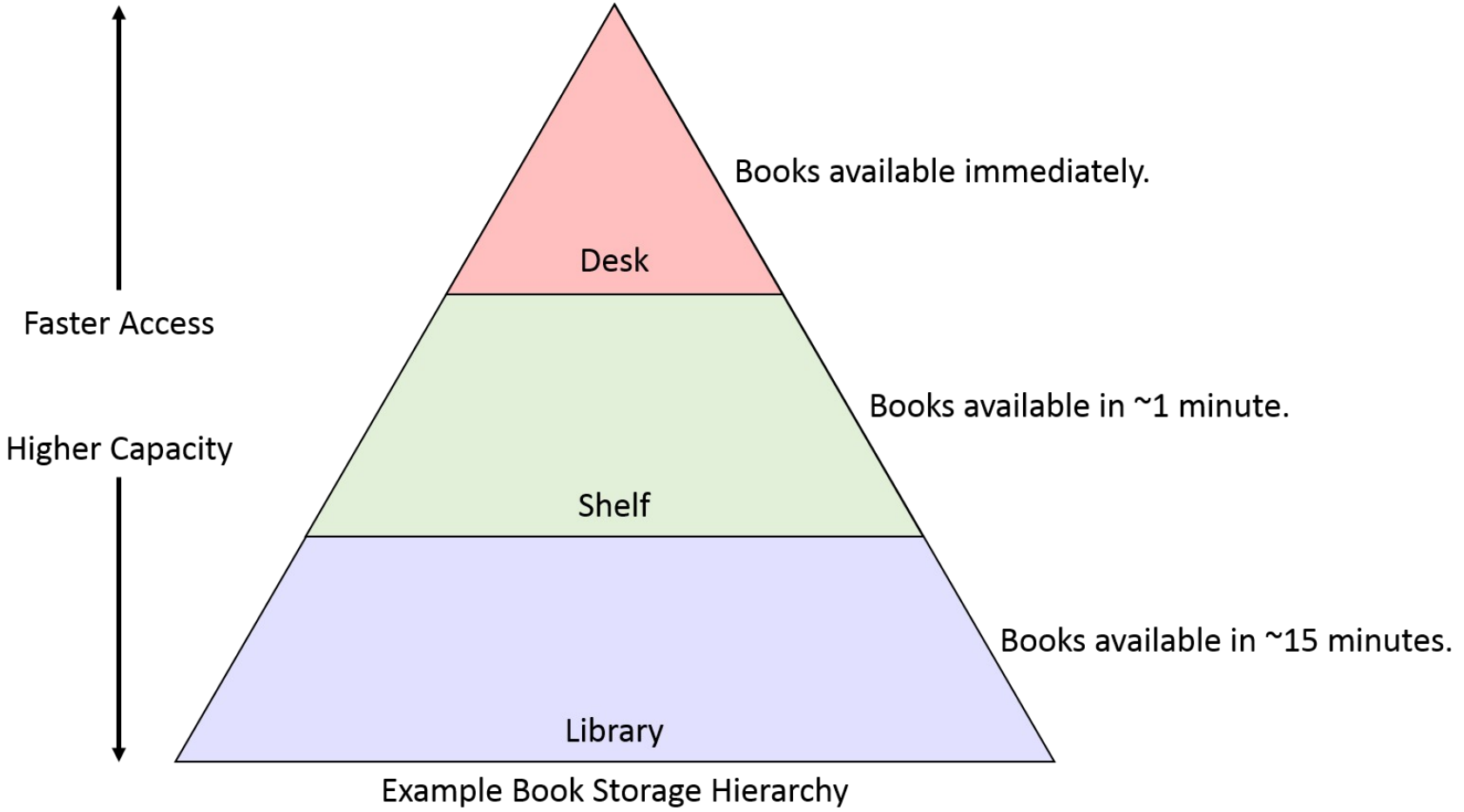
Chapter 7: Large and Fast: Exploiting Memory Hierarchy

Basic structure of memory hierarchy



The Memory Hierarchy

Basic structure of memory hierarchy: Example



Example Book Storage Hierarchy

Locality

- *Locality* is a principle that states, programs access a small portion of their address space at any time. In easier words, programs tend to reuse data and instruction they used recently.
  1. Temporal locality: Items accessed recently are likely to be re-accessed soon. Ex: a Loop.
  2. Spatial locality: Nearby items of the accessed items are likely to be accessed soon. Ex: an Array.

Now students will explain locality with the array……

Hit and Miss

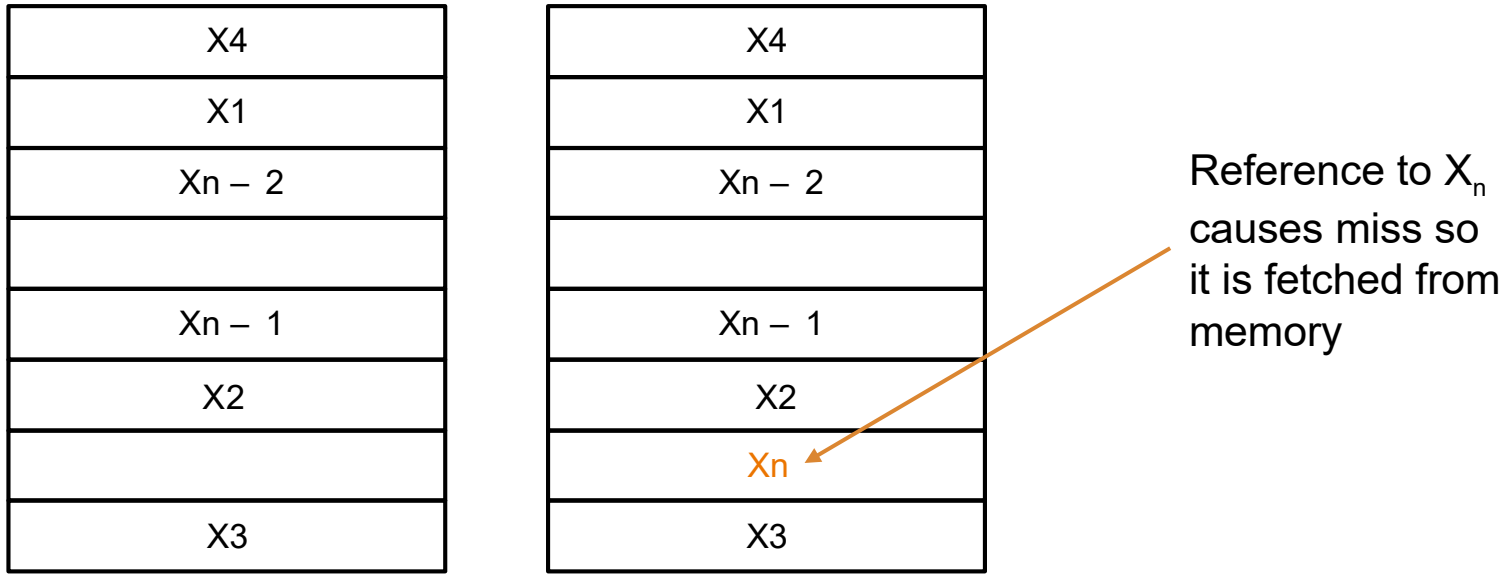
- Focuses on *any two adjacent* levels, called *upper* (closer to CPU) and *lower* (farther from CPU) in the memory hierarchy, because each block copy is always between two adjacent levels
- Memory terminology:
  1. Cache block/line: minimum unit of data present (or not) in cache to move between levels
  2. Hit: data requested in upper levels are present
  3. Hit time: time to determine if the access is indeed a hit + time to access and deliver the data from the upper level to the CPU
  4. Hit rate: fraction of memory accesses that are hits (i.e., found at upper level). Eq: *Hit rate = hits / hits + misses*

Hit and Miss

- Memory terminology:
  1. Miss: data requested is not in upper level
  2. Miss rate: fraction of memory accesses that are not hits. Eq: *miss rate = 1 – hit rate*
  3. Miss penalty/time: time to determine if the access is a miss + time to replace block at upper level with corresponding block at lower level + time to deliver the block to the CPU

Caches

- A cache is a small, fast memory that stores copies of frequently used data from main memory, so the CPU can access it faster.
- By simple example
  - assume block size = one word of data



a. Before the reference to Xn

b. After the reference to Xn

Caches

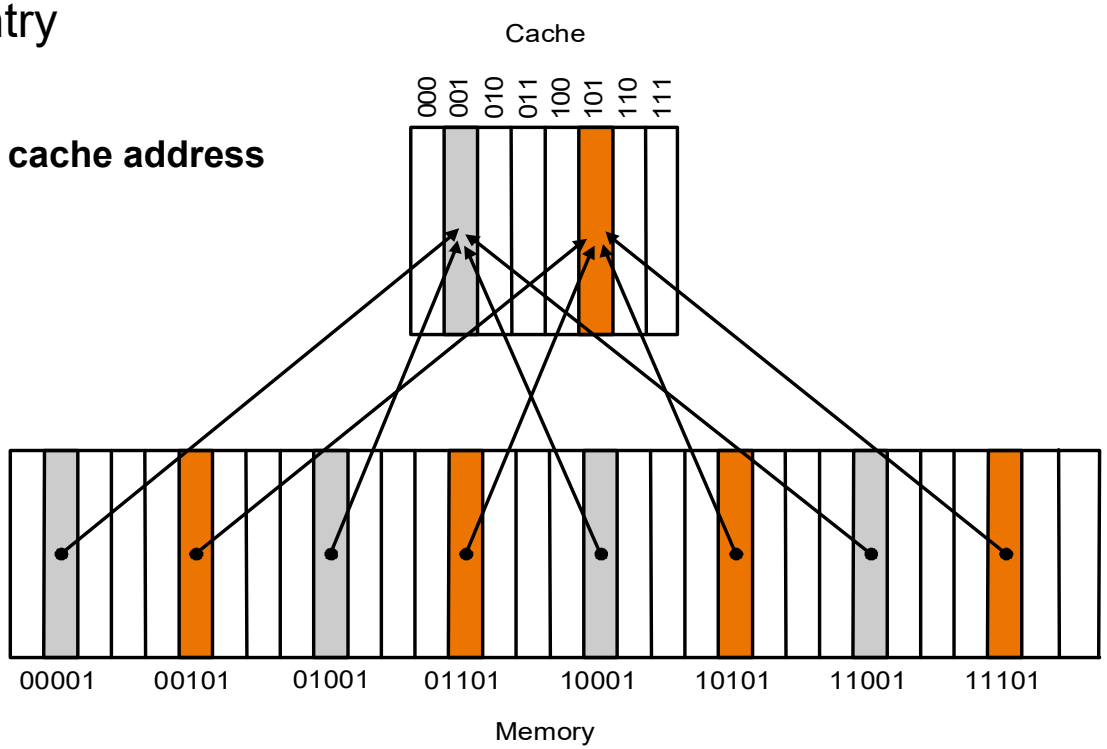
- Issues:
  - *how do we know if a data item is in the cache?*
  - *if it is, how do we find it?*
  - *if not, what do we do?*
- Solution depends on *cache addressing schemes,* such as*, Direct Mapped Cache.*
- Direct Mapped Cache: A direct mapped cache connects each memory block to exactly one cache block using a formula.

Direct Mapped Cache

- Addressing scheme in *direct mapped* cache:
  - cache block address = memory block address *mod* cache size (*unique*). For example, memory block 13 (1101 in binary) would go to 1101 MOD 8 = 5, which is cache block 5 (101).
  - if cache size = 2^m, cache address = lower m bits of n-bit memory address
  - remaining upper n-m bits kept as *tag bits* at each cache block, also need a *valid bit* to recognize valid entry

Like for 8 block = 2^3

So lower 3 bits used for cache address



Cache Read Hit/Miss

- Cache read hit: The data or instruction the CPU needs is already in the cache (fast memory). No actions needed!
- Cache Read Miss: The instruction the CPU needs is *not* in the cache (slow path). Do:
  1. The CPU asks main memory for the instruction.
  2. The CPU *waits* (stalls) until memory sends the instruction.
  3. The instruction is saved in the cache for next time.
  4. The CPU *restarts* fetching the instruction (now it's a cache hit!).
  5. Why stall?: The CPU *must* have the next instruction to know what to do next. It can't guess!

Cache Read Hit/Miss

- Data Cache Read Miss: The data (ex: a variable) the CPU needs is *not* in the cache. Do:
  1. The CPU asks main memory for the data.
  2. While waiting, the CPU *keeps working* on other instructions that don't need that data.
  3. Only when an instruction *needs* the missing data does the CPU *stall* (wait).
  4. The data is saved in the cache for next time.
  5. Why not stall immediately?: The CPU can do other work while waiting, as long as it doesn't need the missing data yet.
  6. Why this doesn't work for instructions: The CPU *must* fetch instructions in order. It can't proceed without the next instruction.

Cache Write Hit/Miss

- **Write-Through Scheme:**
  - Write Hit (data is in cache):
    1. Update the data in the cache *and* main memory immediately.
    2. Ensures memory is always up-to-date (no inconsistency).
    3. Downside: Slow because *every* write goes to memory.
  - Write Miss (data is *not* in cache):
    1. Write the data directly to memory (no need to load it into cache first).
    2. Also store it in the cache (for future use).
  - Performance Trick: Write Buffer
    1. Instead of waiting for memory, the CPU stores writes in a small write buffer (like a waiting area).
    2. The CPU keeps working until the buffer is full.
  - Why Use Write-Through?
    1. Simple, ensures memory is always correct.
    2. Good for systems where consistency is critical (e.g., shared memory in multi-core CPUs).

Cache Write Hit/Miss

- **Write-Back Scheme:**
  - Write Hit (data is in cache):
    - Only update the cache (not memory yet).
    - Mark the cache block as **"**dirty**"** (meaning it differs from memory).
  - Write Miss (data is *not* in cache):
    - Load the missing block from memory into cache.
    - Then update the cache (and mark as dirty).
  - When Does Memory Get Updated?
    - Only when the cache block is replaced (ex: new data needs its space).
    - The "dirty" block is written back to memory before replacement.
  - Why Use Write-Back?
    - Faster (fewer memory writes).
    - More complex (needs "dirty" bits and replacement logic).

Decreasing Miss Rates with Associative Block Placement

- **Direct-Mapped Cache:**
  - How it works:
    - Each memory block can go to only one specific location in the cache.
    - Formula:
      - Cache Block = (Memory Block Address) MOD (Cache Size)
    - Example:
      - If cache has 4 blocks (0,1,2,3), memory block 5 goes to slot 5 MOD 4 = 1.
      - If another block (ex: 9) also maps to slot 9 MOD 4 = 1, it replaces the old one.
  - Advantages:
    - Fast (no searching, just go to one location).
  - Disadvantages:
    - High miss rate because conflicts force replacements.

Decreasing Miss Rates with Associative Block Placement

- **Fully Associative Cache:**
  - How it works:
    - A memory block can go anywhere in the cache.
    - All cache entries are searched in parallel to find the block.
  - Example:
    - Like a bookshelf where a book can be placed anywhere, you check all shelves at once.
  - Pros:
    - Lowest miss rate (no forced replacements).
  - Cons:
    - Slow (high hit time) because searching the whole cache takes time.

Decreasing Miss Rates with Associative Block Placement

- **Set-Associative Cache (Best):**
  - How it works:
    - Cache is divided into sets (groups of blocks).
    - Each memory block maps to one set, but can go to any block within that set.
    - Formula:
      - Cache Set = (Memory Block Address) MOD (Number of Sets)
    - If each set has n blocks, it's called n-way associative.
    - Example:
      - A 2-way associative cache with 4 sets:
      - Memory block 5 maps to set 5 MOD 4 = 1.
      - It can occupy either of the 2 slots in set 1.
  - Advantages:
    - Lower miss rate than direct-mapped (fewer conflicts).
    - Faster than fully associative (only search one small set).
  - Disadvantages:
    - More complex than direct-mapped.

Virtual Memory

- Why Virtual Memory?
  - Problem: Programs need more memory than what's physically available (RAM).
  - Solution: Use the hard disk as extra "slow RAM" and pretend the system has more memory than it actually does.
- Key Idea:
  - Main memory (RAM) acts like a cache for the disk.
  - Programs see a huge virtual address space, but only parts of it are loaded in RAM at a time.
- How Virtual Memory Works:
  - A game thinks it has 16 GB of memory, but your PC only has 8 GB RAM.
  - The OS swaps unused parts to disk, making it seem like 16 GB exists.

Virtual Memory

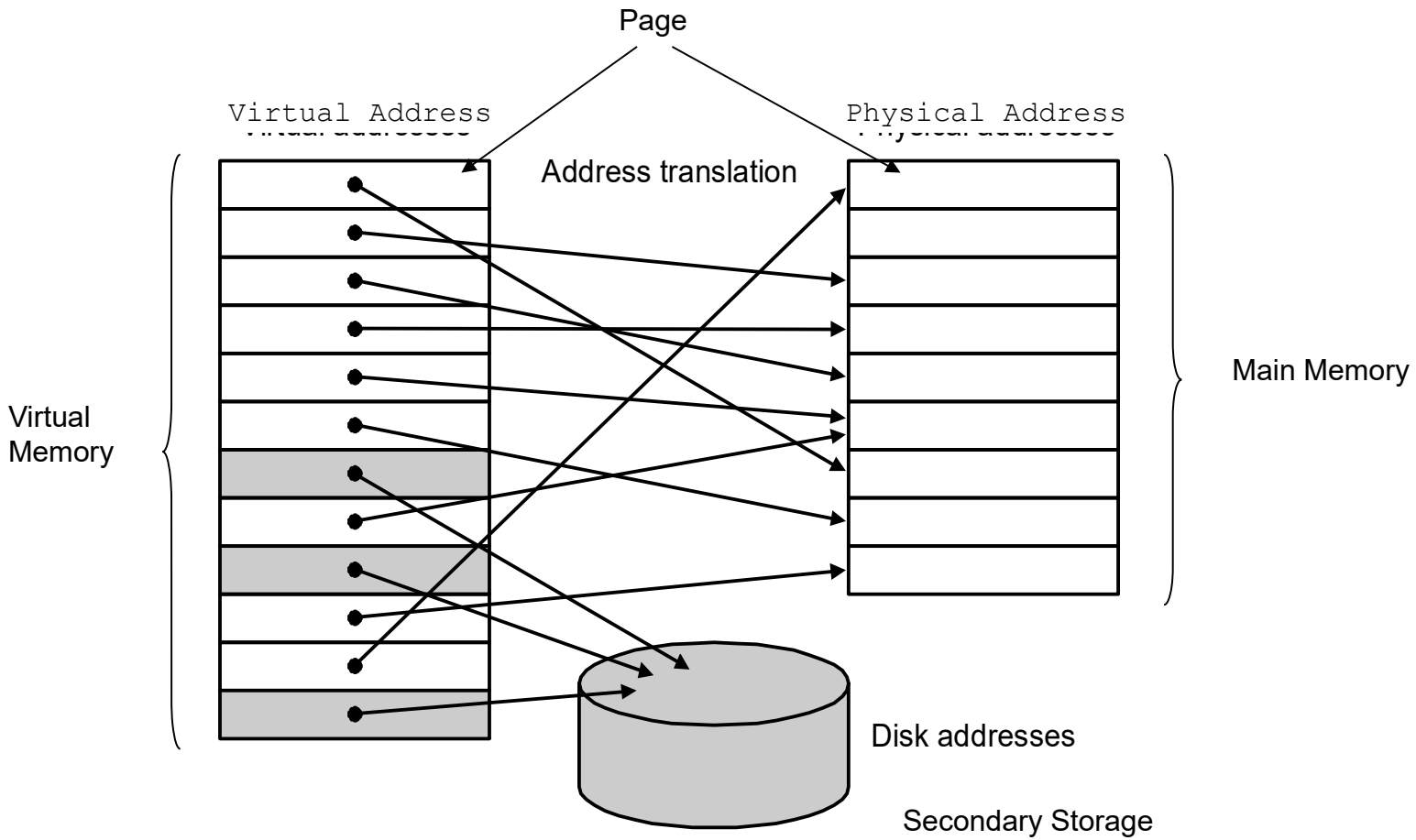
- Pages
  - Memory is split into fixed-size chunks called pages (typically 4 KB).
  - Disk ↔ RAM transfers happen in whole pages.
  - Page Hit: Data is already in RAM (fast access).
  - Page Fault: Data is on disk (slow, must load into RAM first).
- The Page Table: Memory's GPS
  - A page table maps virtual addresses (what programs use) to physical addresses (real RAM locations).
  - If data isn't in RAM (page fault):
    1. OS pauses the program.
    2. Loads the missing page from disk to RAM.
    3. Updates the page table.
    4. Resumes the program (now it's a page hit).

Virtual Memory: Example

Library Books:

- Virtual Memory = All books you *could* read (huge catalog).
- Physical Memory (RAM) = Books on your desk (limited space).
- Disk = Library shelves (slow to access).
- Page Fault = You need a book that’s not on your desk, so go fetch it from the library.

Virtual Memory: A visual



Mapping of pages from a virtual address to a physical address or disk address

Page Fault

- Page Fault: When a program tries to access a page (block of memory) that's not in RAM (it's on disk).
- What happens then?
  1. The OS pauses the program.
  2. Loads the missing page from disk to RAM (slow!).
  3. Updates the page table (memory's address book/GPS).
  4. Resumes the program.
- Why Are Page Faults Slow?
  - Miss penalty: Millions of CPU cycles (disk is ~100,000x slower than RAM!).
  - Solution: Use large pages (e.g., 32 KB or 64 KB instead of 4 KB).
    - Bigger pages = fewer trips to disk.
- Real-World Impact:
  - If your PC slows down when switching apps, it's likely thrashing (constant page faults).
  - SSDs help (faster than HDDs), but RAM is still the best!

Page Fault: How to Reduce?

- Smart Replacement: LRU (Least Recently Used)
  1. Kick out the page that hasn't been used in the longest time.
  2. Implementation:
     a. Each page has a use bit (set to 1 when accessed).
     b. OS periodically clears use bits to track recent activity.
- Fully Associative Placement
  1. A page can go anywhere in RAM (no fixed slots).
  2. Why? Fewer conflicts → fewer page faults.
- Write-Back (Not Write-Through)
  1. Write-back: Only write to disk when a page is replaced.
  2. Write-through (updating disk on every write) is too slow for virtual memory.

TLB: Translation Lookaside Buffer

- Problem: Virtual memory requires checking the page table (in RAM) before every memory access which slows everything down.
- Solution: TLB (Translation Lookaside Buffer), a tiny cache for address translations.
  - TLB Hit (*The translation is in the TLB*): Instant physical address (no RAM lookup).
  - TLB Miss (*The translation isn't in the TLB*): Must check page table (slower), then caches the result.
- Why it matters:
  - Avoids doubling memory access time.
  - Works like a speed cheat sheet for address translations.
- Examples:
  - Page table = Phonebook (slow to search).
  - TLB = Frequently-called contacts list (fast access).

TLB: A visual

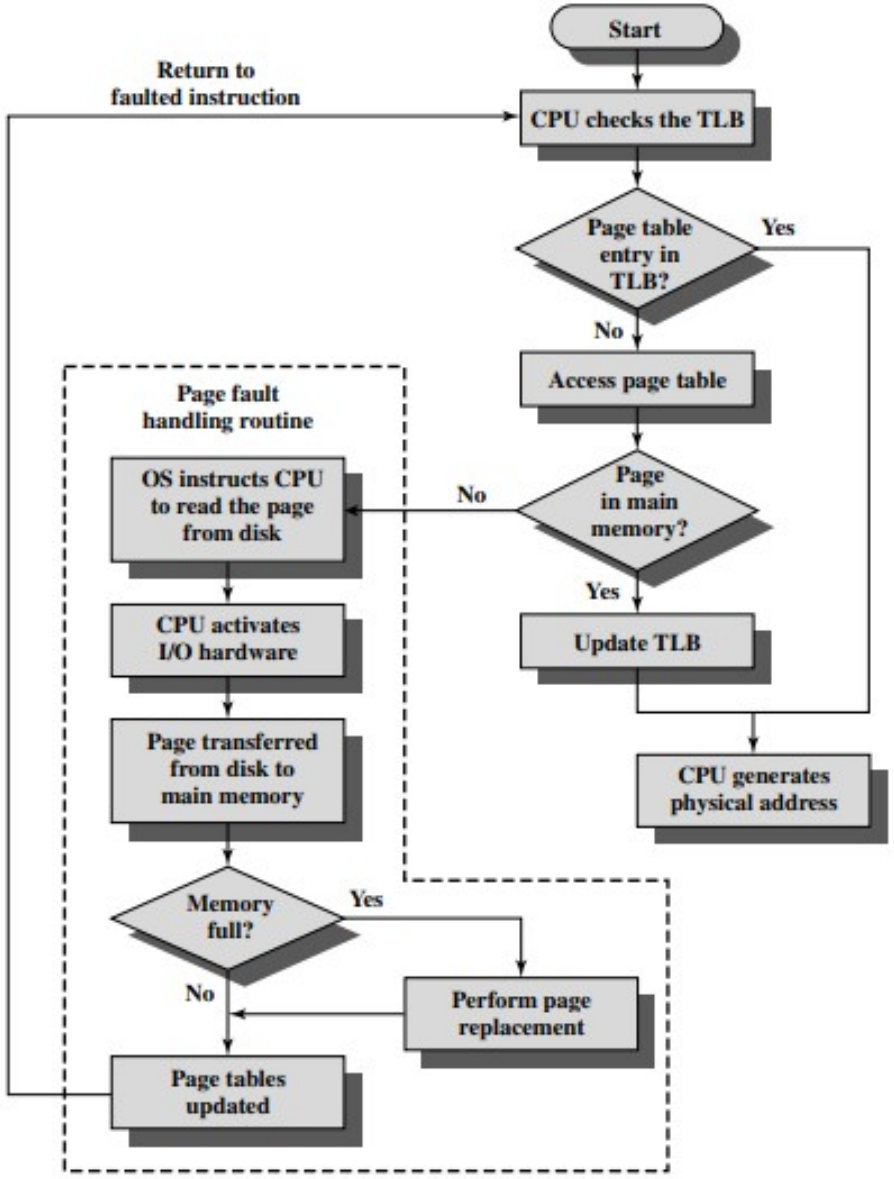
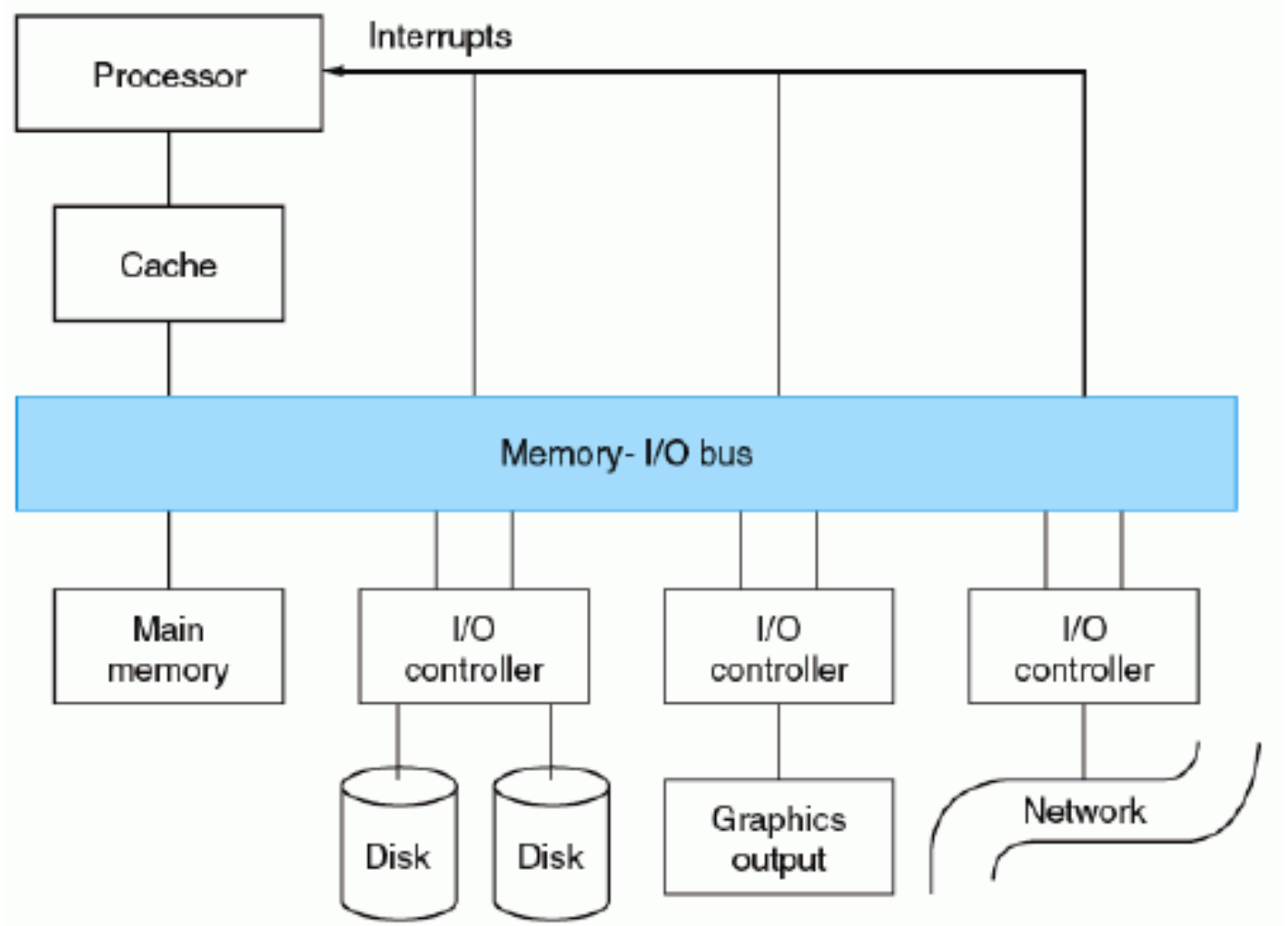


Figure 8.18 Operation of Paging and Translation Lookaside Buffer (TLB)

Chapter 8: Interfacing Processors and Peripherals interrupt DMA

Introduction



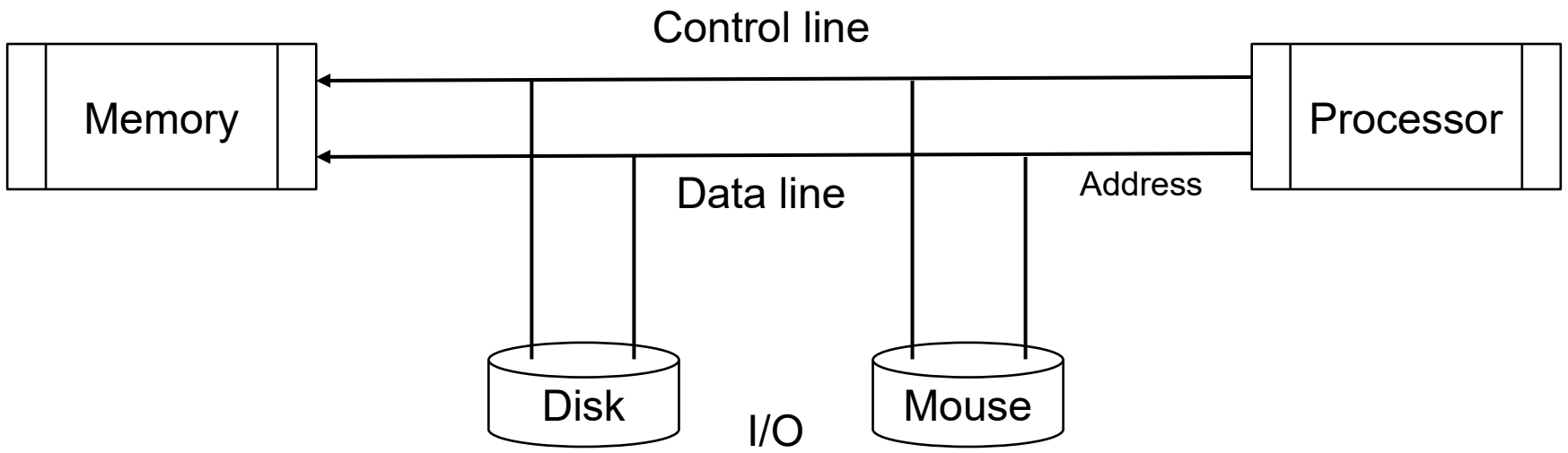
8.1 A typical collection of I/O devices.

Buses: Connecting I/O Devices to Processor and Memory

- A bus is a shared communication link, which uses one set of wires to connect multiple subsystems.
- Bus: set of control lines + set of data lines
- Two major advantages:
  1. Versatility
  2. Low cost
- The major disadvantage:
  - Creates a communication bottleneck.
- A bus transaction includes two parts: sending the address and receiving or sending the data.
- Input operation: inputting data from the device to memory.
- Output operation: outputting data to a device from memory.

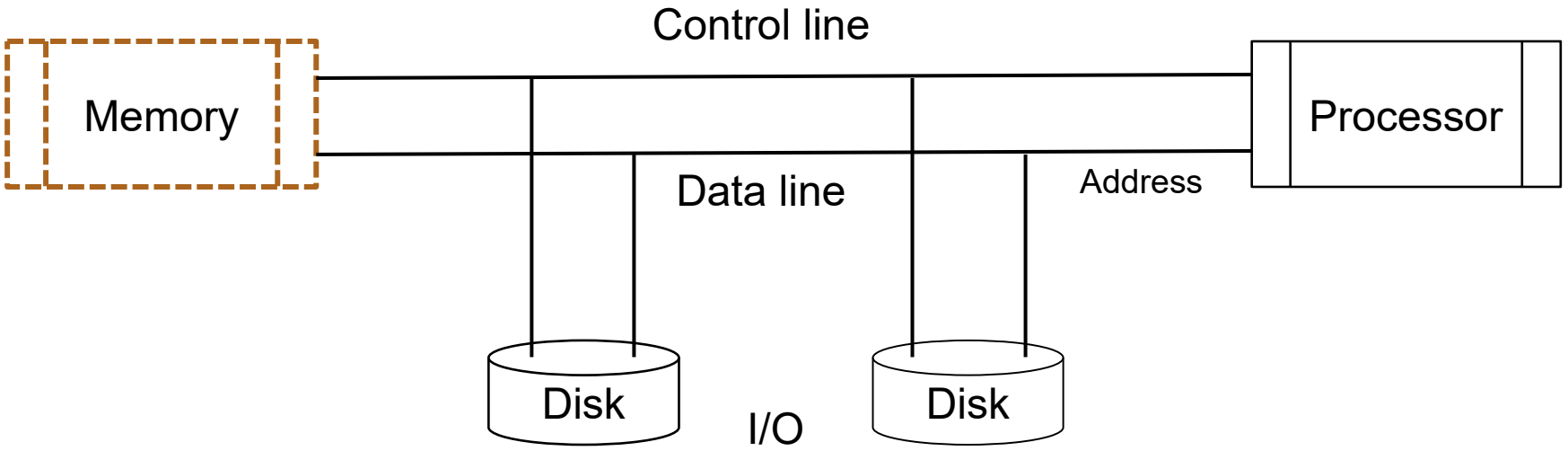
Buses: Connecting I/O Devices to Processor and Memory

- a) 1st step initiates a read from memory.
  1. Control line signals a read request to memory from the processor
  2. Data line contains the address



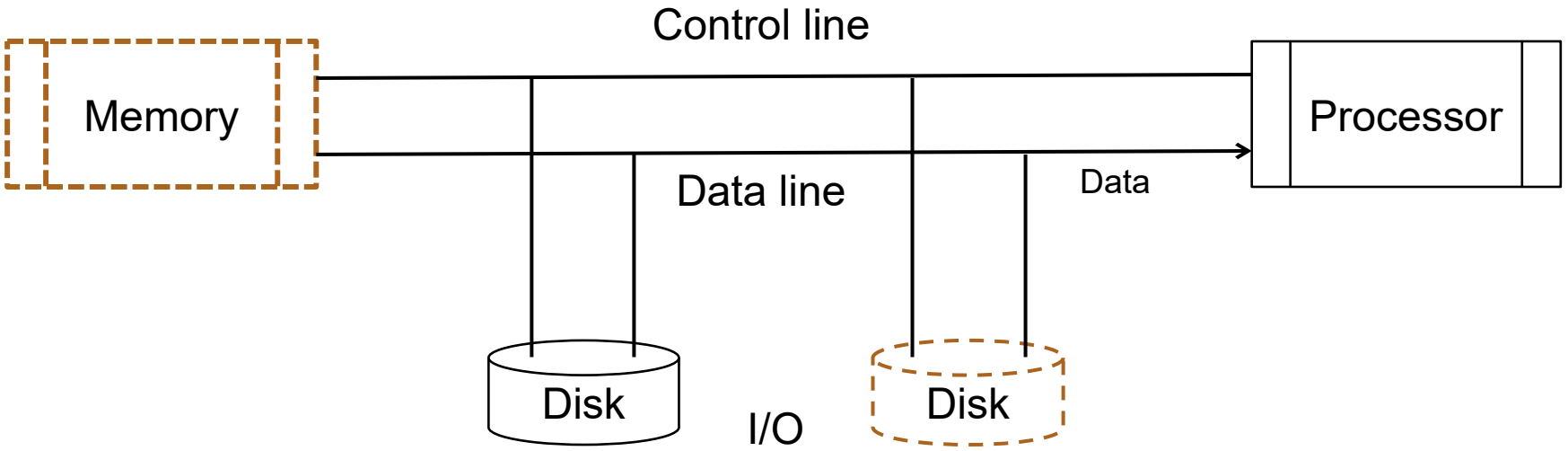
Buses: Connecting I/O Devices to Processor and Memory

- b) 2nd step memory is accessing data



Buses: Connecting I/O Devices to Processor and Memory

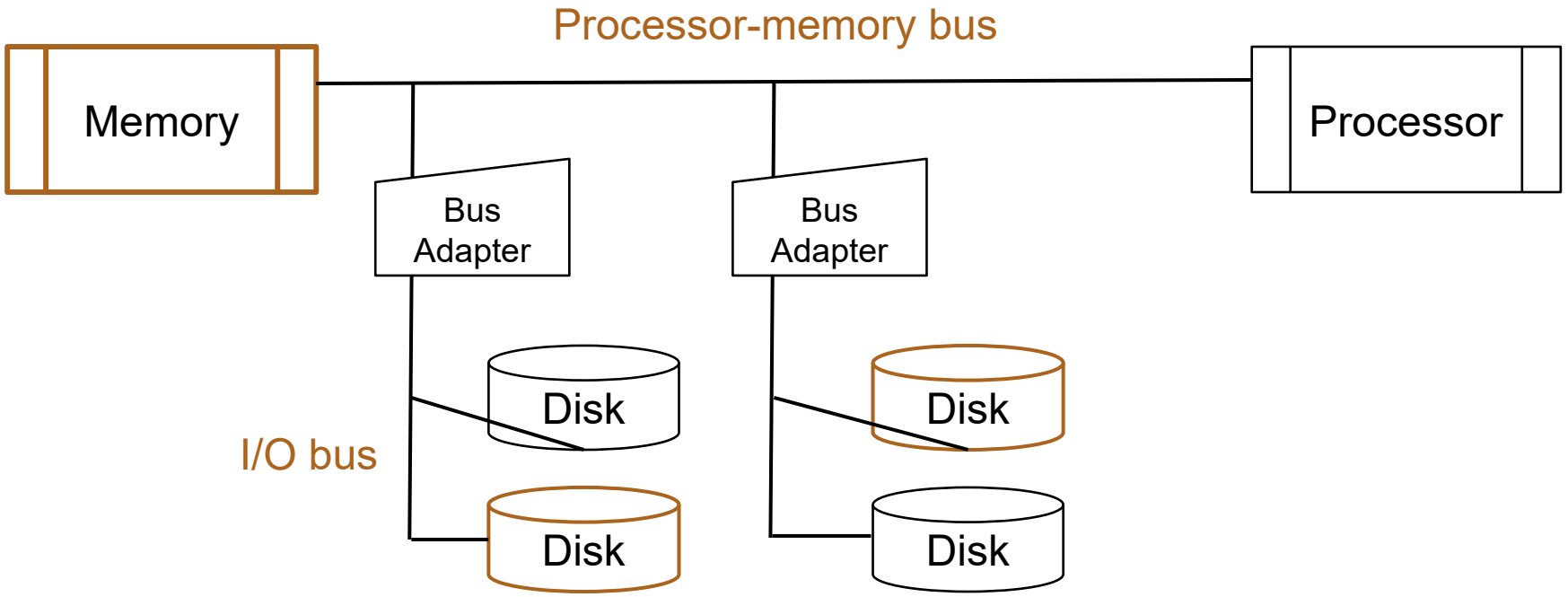
- C) 3rd step memory transfers data using the data lines
- Control lines signals that data is available in I/O device



Types of Buses

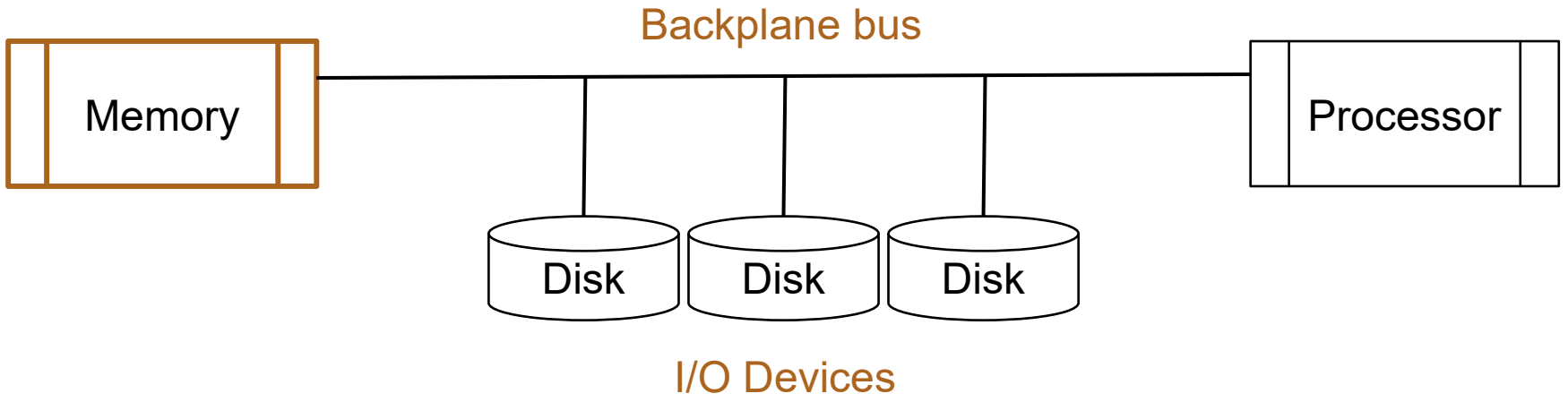
- Traditionally classified as:
  1. Processor-memory buses: Short, high-speed, optimized for CPU-memory communication
  2. I/O buses: Longer, slower, connect peripherals (disks, USB, etc.)
  3. Backplane buses: General-purpose, connect multiple components (CPU, memory, I/O) on a motherboard
- Classification depending on communication schemes:
  1. Synchronous buses: Use a clock signal for timing (PCI - Peripheral Component Interconnect)
  2. Asynchronous buses: Use handshaking (request/acknowledge signals) for timing (ex: USB)
- Processor-memory buses are
  1. Short distance (CPU to RAM)
  2. Generally high speed (matched to memory bandwidth)
  3. Optimized for low latency.

Types of Buses



Types of Buses

- I/O buses,
  1. Can be lengthy
  2. Can have many types of devices connected to them
  3. Often have a wide range of data bandwidth
  4. Typically do not interface directly to the memory (use processor-memory / backplane buses)
- Backplane buses allow processor, memory and I/O devices to coexist on a single bus

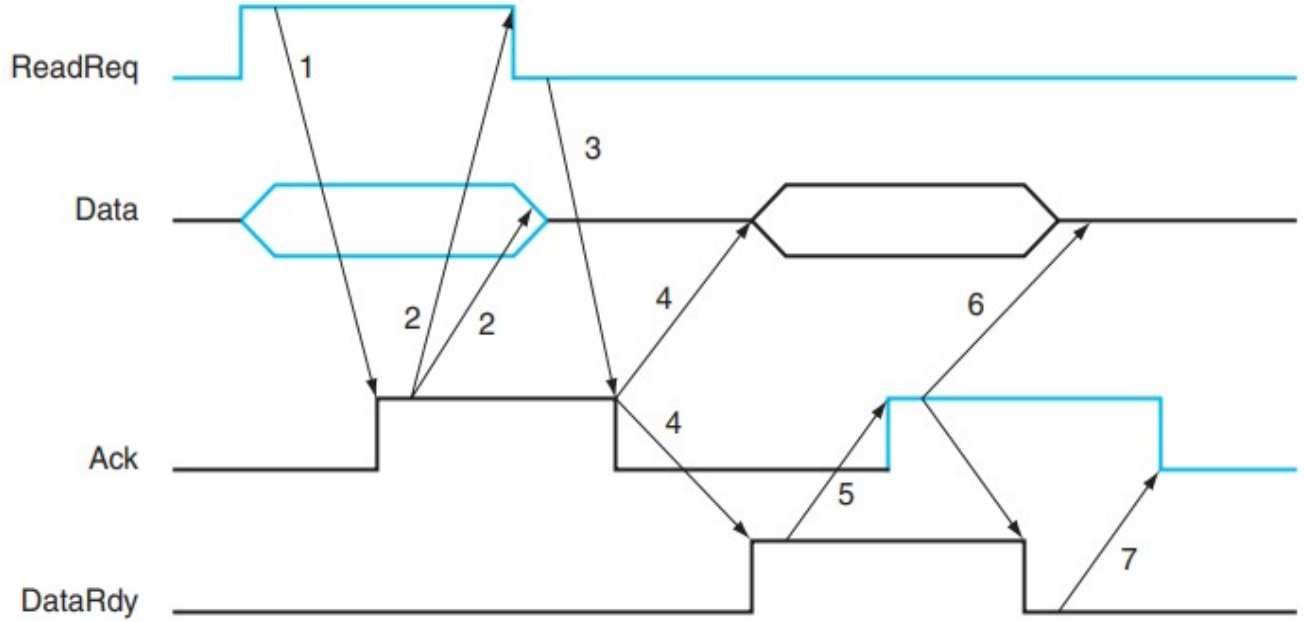


Synchronous Bus vs. Asynchronous Bus

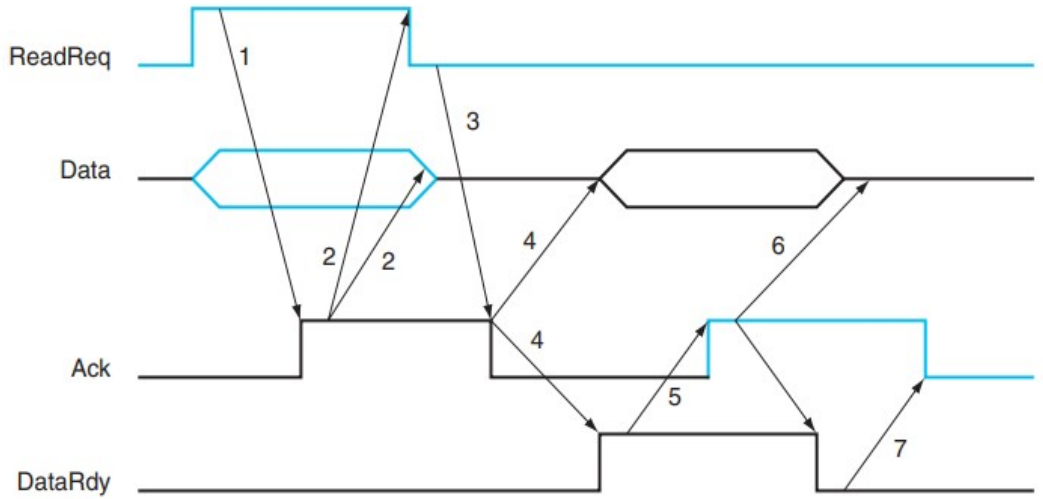
- **Synchronous Bus**: A synchronous bus operates with a common clock signal that synchronizes all data transfers between the CPU, memory, and I/O devices.
  - How it works:
    1. All devices on the bus follow the same clock cycle.
    2. Data is transferred at fixed intervals (clock edges).
    3. No handshaking is needed since timing is predetermined.
- **Asynchronous Bus**: An asynchronous bus does not use a clock and instead relies on handshaking signals (like "Ready" and "Acknowledge") to coordinate data transfers.
  - How it works:
    - The sender initiates a transfer by asserting a control signal (e.g., "Request").
    - The receiver responds with an "Acknowledge" signal when ready.
    - Data is transferred only when both devices agree.

Handshaking Protocol in Computer Buses

- The handshaking protocol is a method used in asynchronous communication to coordinate data transfer between two devices (e.g., CPU and I/O device) without relying on a common clock signal. It ensures that the sender and receiver are synchronized before exchanging data.



Handshaking Protocol in Computer Buses



The steps in the protocol begin immediately after the device signals a request by raising ReadReq and putting the address on the Data lines:

1. When memory sees the ReadReq line, it reads the address from the data bus and raises Ack to indicate it has been seen.
2. I/O device sees the Ack line high and releases the ReadReq and data lines.
3. Memory sees that ReadReq is low and drops the Ack line to acknowledge the ReadReq signal.
4. This step starts when the memory has the data ready. It places the data from the read request on the data lines and raises DataRdy.
5. The I/O device sees DataRdy, reads the data from the bus, and signals that it has the data by raising Ack.
6. The memory sees the Ack signal, drops DataRdy, and releases the data lines.
7. Finally, the I/O device, seeing DataRdy go low, drops the Ack line, which indicates that the transmission is completed.

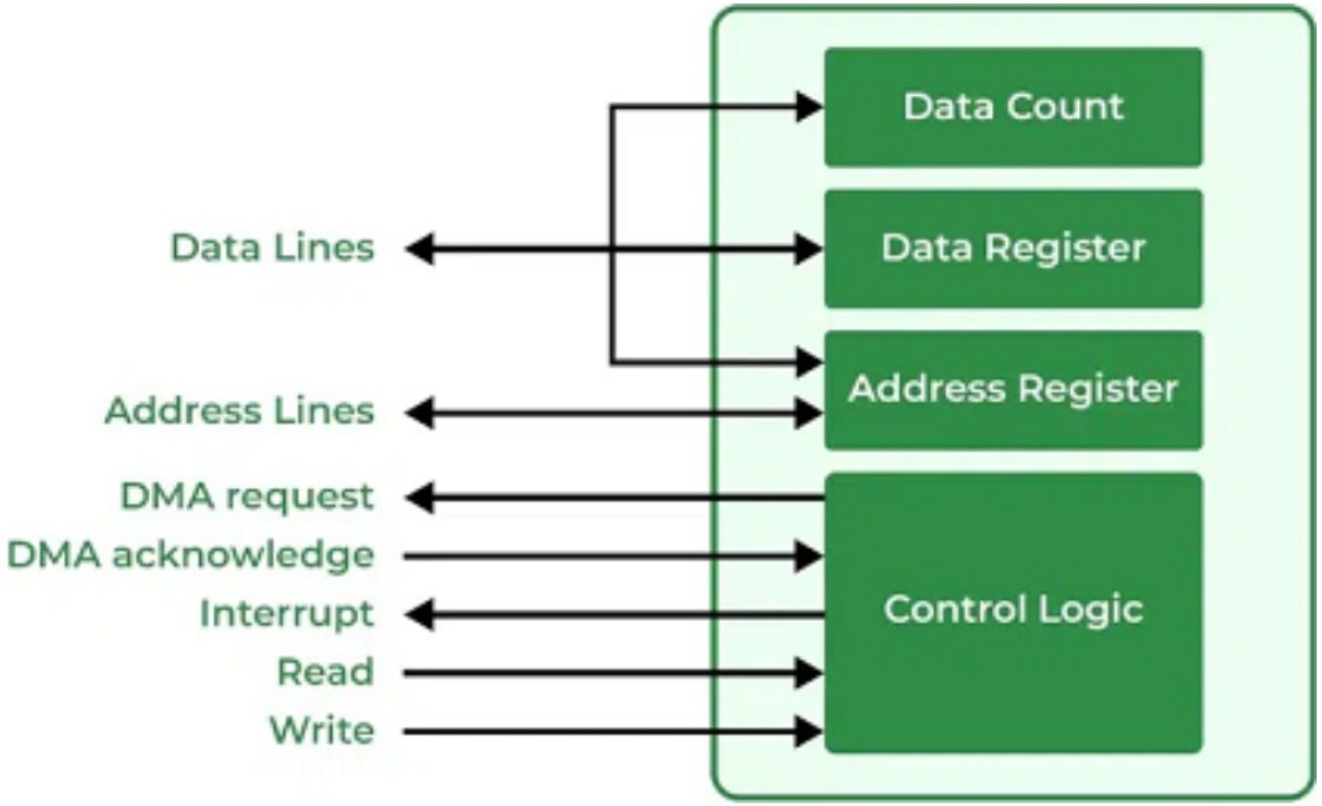
A new bus transaction can now begin.

Interrupt

- An interrupt is a mechanism that allows external devices or internal software to temporarily halt the CPU's current execution, handle a time-sensitive task, and then resume normal operation. Interrupts are crucial for efficient I/O handling, multitasking, and real-time processing.
- Hardware Interrupts: Generated by external devices (keyboard, mouse, disk, network card).
  1. I/O Interrupt: Printer out of paper, USB device plugged in.
  2. Timer Interrupt: Generated by the system clock (used in preemptive multitasking).
  3. Power Interrupt: Low battery, overheating.
- Software Interrupts (Exceptions): Generated by software (OS or running programs).
  1. System Calls: A program requests OS services (read(), write()).
  2. Exceptions: Division by zero, invalid memory access (segmentation fault).

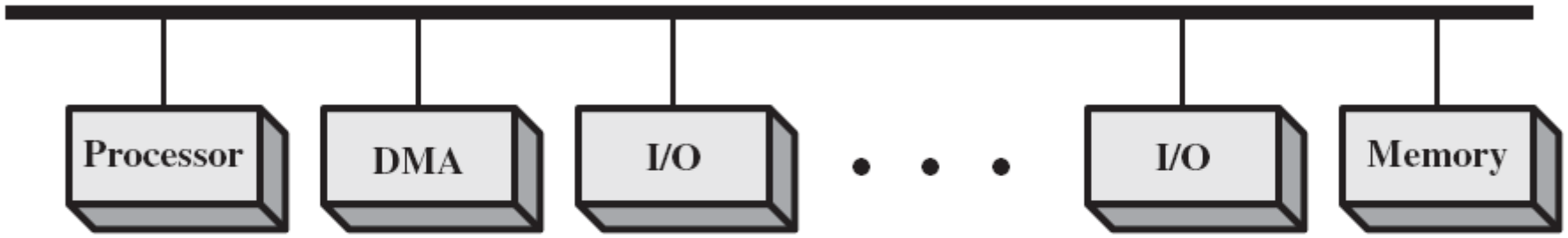
DMA: Direct memory access

- Direct Memory Access (DMA) is a feature that allows peripheral devices (disks, GPUs, or network cards) to transfer data directly to/from memory without CPU intervention.
- Below is a simple DMA block:



DMA Config: Single Bus, Detached DMA Controller

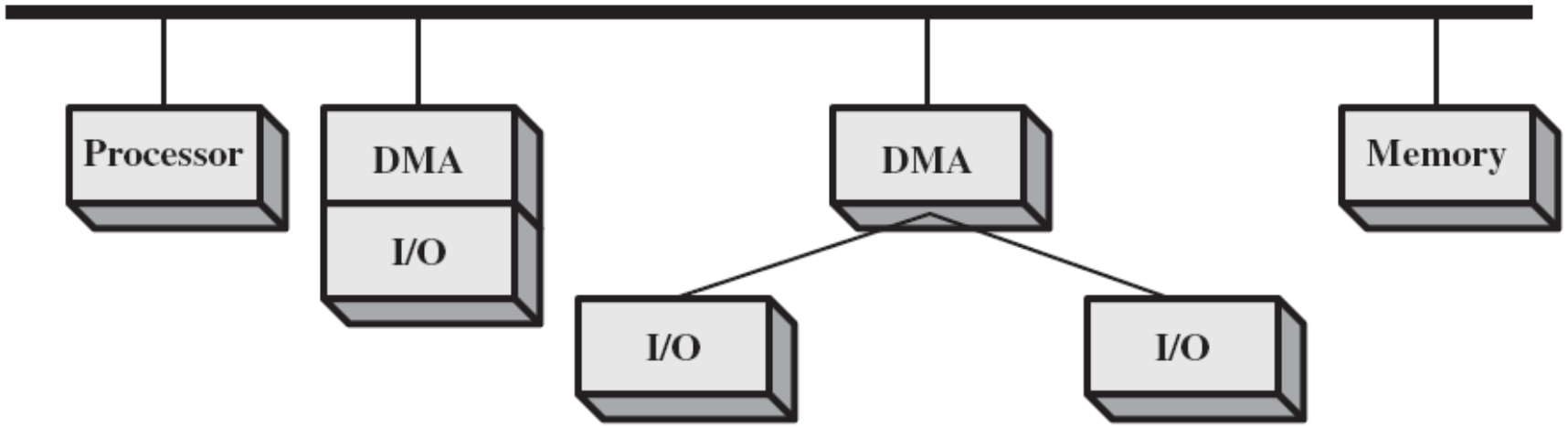
- One Shared Bus: CPU, Memory, I/O, and DMA controller all connect to the same bus.
- DMA as a Surrogate Processor: The DMA controller acts like a mini-CPU to manage data transfers.
  - Simple design
  - Inefficient (bus contention slows CPU)
  - Cheap to implement
  - Only one transfer at a time



Single-bus, detached DMA

DMA Config: Single Bus, DMA Integrated into I/O Module

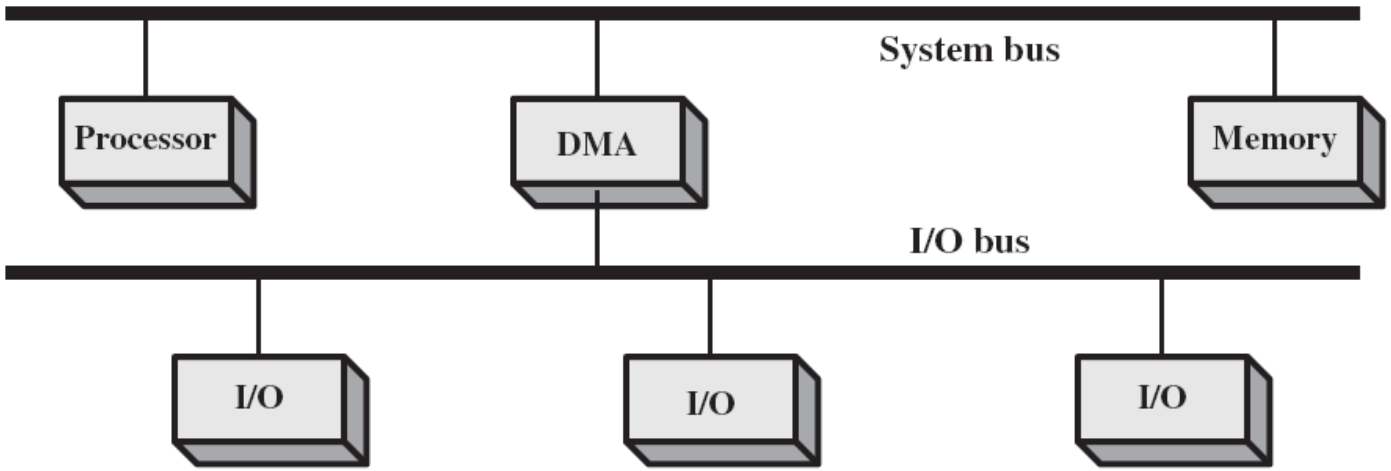
- DMA controllers are embedded inside I/O devices (disk, GPU).
- Each I/O device *can manage its own DMA transfers*.
  - Faster (less bus contention)
  - More expensive (DMA in every I/O)
  - CPU waits less
  - Complex wiring



Single-bus, Integrated DMA-I/O

DMA Config: Separate I/O Bus

- Two Buses:
  1. System Bus: CPU + Memory (high-speed).
  2. I/O Bus: Connects *all DMA-enabled devices* to a single DMA controller.
- Why Better?
  1. CPU rarely pauses (DMA uses I/O bus, not system bus).
  2. Supports many devices without slowing the CPU.



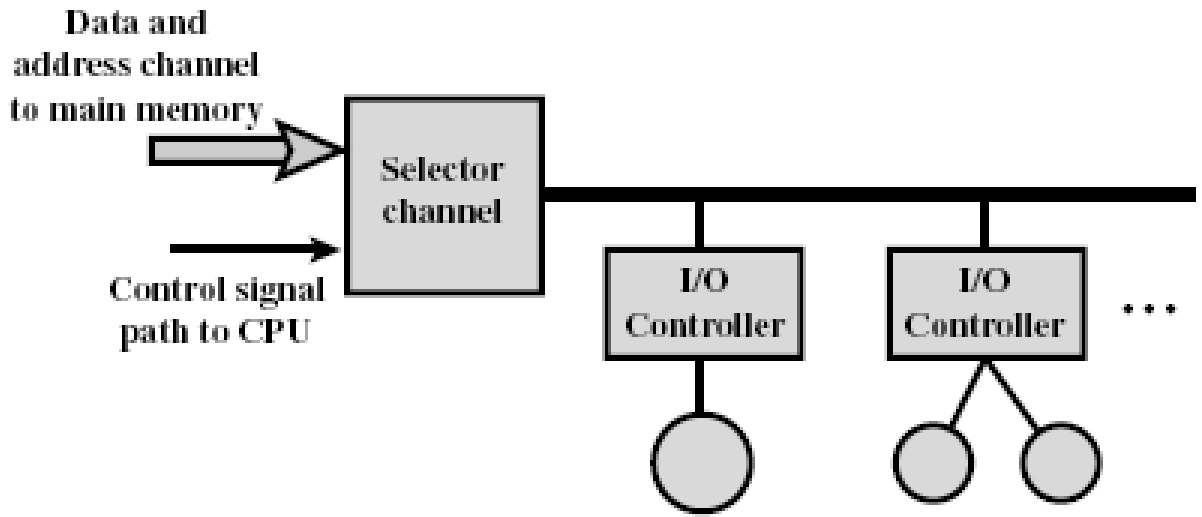
I/O bus

What is an I/O Channel?

- Smarter DMA: An I/O Channel is like a DMA controller with a tiny brain.
  - DMA = Just moves data when told.
  - I/O Channel = Can execute simple programs (called *channel programs*) to manage complex transfers.
- How It Works
  1. CPU gives the I/O Channel a "to-do list" (channel program) in memory.
  2. I/O Channel reads the list and handles all steps of the transfer *automatically*.
  3. CPU is free to do other work.

Types of I/O Channel

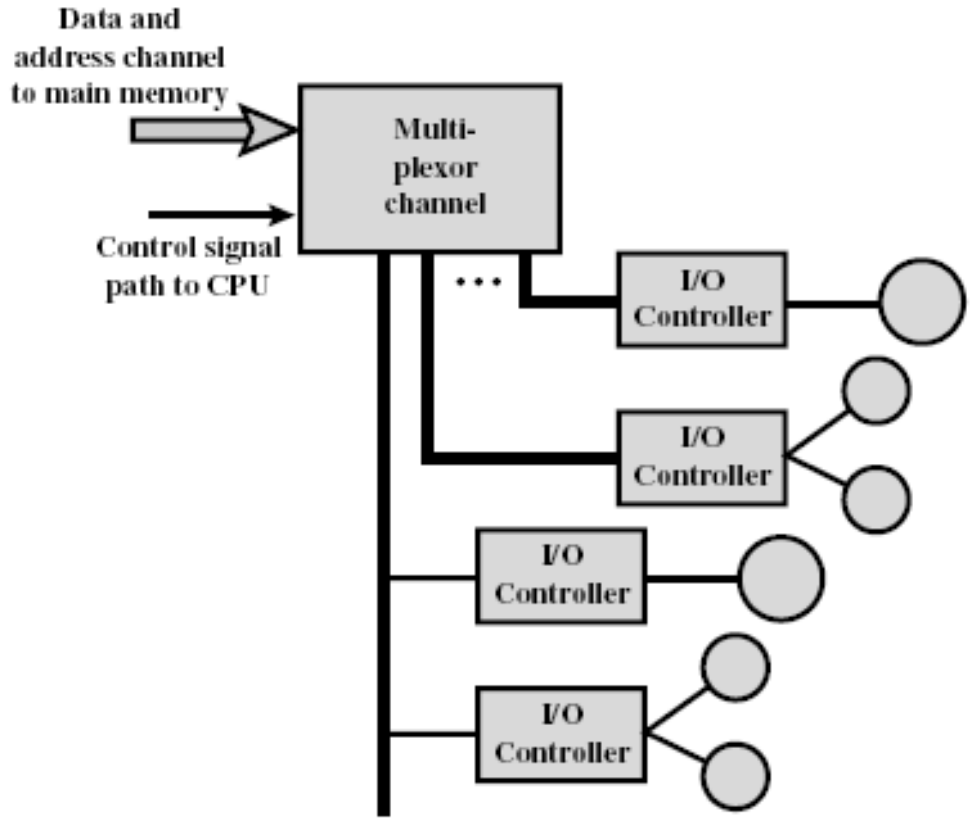
- **Selector Channel**
  - One at a Time: Controls one high-speed device at a time (SSD, GPU).
  - Use Case: Best for devices needing full bandwidth (no sharing).



(a) Selector

Types of I/O Channel

- **Multiplexor Channel**
  - Time-Sharing Pro: Handles multiple slow/medium-speed devices (keyboards, printers).
  - How? Uses *Time-Division Multiplexing (TDM)* to switch between devices.



(b) Multiplexor

The End, Thank you!

Created by following “Computer Organization and Design: Third Edition” book, published by Patterson